

MASTER OF SCIENCE THESIS

Development of a Multi-Touch Interface using a 3D Camera

Author:

Georg Hackenberg (Mat.-No. 283380)

georg.hackenberg@rwth-aachen.de

Supervisor:

Dr. Thorsten Fröhlich

thorsten.froehlich@fit.fraunhofer.de

First corrector:

Prof. Wolfgang Prinz, Ph.D.

wolfgang.prinz@fit.fraunhofer.de

Second corrector:

Prof. Dr. Wolfgang Broll

wolfgang.broll@fit.fraunhofer.de



May 25, 2010

Declaration

I declare that this Master Thesis is my own unaided work, except for the official assistance from my supervisors. It has not been submitted in any form for any other degree or diploma at any university or other education institution. Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references is included at the end.

Location, Date

Signature

Acknowledgements

Foremost I thank my parents for their enduring encouragement and support over the past two decades of general and academic education. I believe, this investment provides me with the proper skills to independently shape my future and beneficially contribute to society. Second, I thank Anisa Stefi for her love and care, and her unconditional support. She was enriching my personal life every day during this period of hard work. Finally, I thank Dr. Thorsten Fröhlich, Dr. Rod McCall and Prof. Dr. Wolfgang Broll for guiding me through my thesis. I enjoyed the freedom they gave me to work independently and implement my own ideas, while relying on their expertise when encountering difficulties.

Abstract

In recent years multi-touch interfaces have become a standard technique in many domains from smart phones, over table surfaces, to regular desktop PCs. The success of multi-touch interaction is an indicator for its superior flexibility and usability over more traditional approaches such as the mouse and the keyboard. In the context of digital 3D content and media similar hand-based techniques have emerged for improving the efficiency and acceptance of user interfaces. However, technical constraints still require utilities such as markers and gloves, or limit the number of users and the detail of the pose estimations.

To overcome the above limitations, a novel lightweight real-time vision algorithm for non-intrusive human hand processing is proposed that is able to detect and track the individual finger locations of multiple users in 3D. Therefore, initially two design choices are explored, one using learning-based techniques on regular color images captured by a stereo camera, a second using geometric constraints on depth data captured by a time-of-flight (TOF) camera. After a preliminary performance evaluation the second is identified as the more promising given its high efficiency as well as the lower number of false-positives. Consequently, a complete vision chain is developed from input preparation, over feature extraction, to object tracking. In particular, a novel feature detector for fingertips and finger pipes is proposed that is derived from the FAST keypoint detector. Further, a graph model is introduced for deriving object topologies and effectively constraining possible palm locations. Tracking is implemented using linear prediction and multi-scale feature-based optical flow. For evaluation the vision component is integrated into a demo application, which implements a basic touch- and gesture-based interaction model using both geometric and statistical constraints over the hand appearances. The demo application is tested in two trials, a pilot study for identifying and fixing minor issues with the preliminary system design and configuration, and a main study for collecting performance data and drawing conclusions about the current state of the technology as well as future possibilities for deployment.

Qualitative analysis of the results suggest that the novel real-time algorithm provides a suitable basis for implementing barehanded 3D interaction techniques on standard hardware. In particular, all subjects were able to complete the given assignments in a reasonable amount of time and over 80 percent considered the experiment *fun*. Further, the interaction model was perceived *intuitive* and *learnable* by more than one half of the users. As the primary tasks for future work the improvement of the tracking component and the accuracy of the pose estimations have been identified in order to increase the stability of the system as well as the expressiveness of the hand gestures.

Kurzfassung

In den vergangenen Jahren haben sich Multi-Touch-Benutzerschnittstellen als Standard in vielen Bereichen von Smartphones über Tischflächen bis hin zu regulären Arbeitsplatzrechner etabliert. Der Erfolg der Multi-Touch-Interaktion ist ein Anzeichen für seine höhere Flexibilität und bessere Bedienbarkeit im Vergleich zu traditionelleren Ansätzen wie der Computermouse und der Tastatur. Im Zusammenhang mit 3D Inhalten und Medien haben sich ähnliche handbasierte Methoden entwickelt, um die Effizienz und Akzeptanz von Benutzerschnittstellen zu verbessern. Jedoch sind aufgrund technischer Einschränkungen immer noch Hilfsmittel wie Markierungen oder Handschuhe erforderlich, oder die Anzahl der Benutzer und der Detaillierungsgrad der Handposenrekonstruktion wird begrenzt.

Um die genannten Beschränkungen zu bewältigen, wird ein neuartiger Echtzeit-Bildverarbeitungsalgorithmus für die Verarbeitung von Handinformationen vorgestellt, der keinerlei Hilfsmittel beansprucht und das gleichzeitige Verfolgen einzelner Finger von mehreren Nutzern im dreidimensionalen Raum ermöglicht. Für diesen Zweck werden zu Beginn zwei Systementwürfe getestet, ein lernbasierter Ansatz für herkömmliche Farbbilder, die von einer Stereokamera aufgenommen werden, und ein geometriebasierter Ansatz für Tiefenbilder, die von einer Infrarotkamera durch Messung von Signallaufzeiten aufgenommen werden. Nach einer groben Leistungsanalyse wird der zweite Ansatz aufgrund dessen hoher Effizienz und der geringeren Anzahl an Falscherkennungen für die weitere Umsetzung ausgewählt. Im Folgenden wird darauf aufbauend eine umfassende Bildverarbeitungskette von Eingabevorverarbeitung über Merkmalsextraktion bis hin zu Objektverfolgung entwickelt. Insbesondere wird ein neuartiger Detektor für Finger und Fingerspitzen vorgestellt, der vom sogenannten „FAST Merkmalsdetektor“ abgeleitet ist. Weiterhin wird ein Graphenmodell für die Ableitung von Objekttopologien eingeführt, welches die effiziente Einschränkung der Handflächenposition erlaubt. Die Objektverfolgung wird mittels linearer Vorausberechnung von Bewegungen und merkmalsbasiertem optischen Fluss gelöst. Für die Beurteilung des Ansatzes wird die Bildverarbeitungskomponente in eine Beispielanwendung integriert, welche ein grundlegendes gestenbasiertes Interaktionsmodell implementiert. Die Beispielanwendung wird in zwei Probeläufen getestet, einer Pilotstudie zur Identifizierung und Behebung von kleineren Problemen der ursprünglichen Systemumsetzung, und einer Hauptstudie zur Sammlung von Leistungsdaten für die Schlussfolgerungen über den aktuellen Stand der Technik und mögliche künftige Einsatzgebiete.

Die qualitative Analyse der Ergebnisse deutet darauf hin, dass der neuartige Echtzeit-Bildverarbeitungsalgorithmus eine geeignete Basis für die Implementierung freihändiger 3D-Interaktionstechniken auf Standardhardware darstellt. Insbesondere waren alle Teilnehmer in der Lage, die gegebenen Aufgaben in angemessener Zeit zu lösen, und über 80 Prozent notierten, dabei Spaß gehabt zu haben. Weiterhin wurde das Interaktionsmodell von über der Hälfte

der Benutzer als intuitiv und leicht erlernbar wahrgenommen. Als Hauptaufgaben für kommende Forschungs- und Entwicklungsarbeiten wurden die Verbesserung der Objektverfolgungstechnik sowie die Verbesserung der Genauigkeit der Handposenrekonstruktion ermittelt, um die Stabilität des Systems und die Ausdrucksfähigkeit der Handgesten zu erhöhen.

Contents

1	Introduction	1
1.1	Motivation	1
1.1.1	Background	1
1.1.2	Scenarios	3
1.1.3	Problems	7
1.2	Objectives	8
1.2.1	Scope	8
1.2.2	Requirements	9
1.2.3	Hardware	11
2	Retrospection	13
2.1	Basic techniques	13
2.1.1	Mathematical models	14
2.1.2	3D vision	16
2.1.3	Image features	19
2.1.4	Vision algorithms	21
2.2	Related work	24
2.2.1	Hand detection	24
2.2.2	Pose estimation	26
2.2.3	Hand tracking	26
2.2.4	Gesture recognition	30
3	Preparation	31
3.1	Overview	31
3.1.1	Learning-based	31
3.1.2	Geometry-based	32
3.2	Learning-based	33
3.2.1	Concept	33
3.2.2	Prototype	35
3.2.3	Performance	37
3.3	Geometry-based	40
3.3.1	Concept	40
3.3.2	Prototype	44
3.3.3	Performance	45

3.4	Conclusion	47
4	Realization	49
4.1	Overview	49
4.2	Input preparation	51
4.2.1	Input filtering	51
4.2.2	Region segmentation	52
4.2.3	Distance calculation	53
4.2.4	Fast classification	54
4.2.5	Pipe segmentation	55
4.3	Feature classification	56
4.3.1	Feature extraction	56
4.3.2	Feature connection	57
4.3.3	Finger extraction	59
4.3.4	Candidate grouping	60
4.4	Object tracking	61
4.4.1	Palm tracking	62
4.4.2	Finger tracking	63
4.5	Gesture recognition	64
4.5.1	Pop up gesture	66
4.5.2	Touch gesture	67
4.5.3	Loose gesture	68
4.5.4	Grab gesture	68
4.5.5	Release gesture	69
4.5.6	Throw gesture	70
4.6	Software Architecture	70
4.6.1	Package structure	72
4.6.2	Data model	73
4.6.3	Interaction components	74
5	Evaluation	77
5.1	Overview	77
5.2	Design	78
5.2.1	Setup	78
5.2.2	Tasks	79
5.2.3	Method	81
5.3	Pilot Study	81
5.3.1	Interaction model refinement	82
5.3.2	Interaction model tuning	82
5.3.3	Tracking algorithm refinement	82
5.4	Main Study	83
5.4.1	Participants	83
5.4.2	Ratings	84
5.4.3	Comments	86

6 Conclusion	89
6.1 Project results	89
6.1.1 Real-time algorithm	89
6.1.2 User study	93
6.2 Future work	94
6.2.1 Usability studies	94
6.2.2 Technical improvements	95
6.2.3 Possible extensions	96
6.2.4 Application research	97
6.3 Thesis summary	97
 A Algorithm: FLOODFILL	 99
 B Algorithm: DISTANCETRANSFORM	 101
 C Sequence: Annotation Tracking	 103
 D Sequence: Palm Tracking	 105
 E Screenshots: Workbench Application	 107

List of Figures

1.1	Excerpt from the movie “Minority Report”.	2
1.2	Examples of augmented reality systems for work support.	4
1.3	Examples of augmented reality systems for knowledge exploration.	5
1.4	Examples of augmented reality systems for gaming.	6
1.5	Cameras used during preparation and realization.	12
2.1	Geometric projection of the pinhole camera model.	15
2.2	Geometric setting of the epipolar geometry framework.	17
2.3	Geometric solution of the depth reconstruction problem.	18
2.4	Neighborhood schemes of the flood fill algorithm.	22
2.5	In- and output of the optical flow algorithm.	23
3.1	Feature classification in the learning-based approach.	32
3.2	Hand model in the geometry-based approach.	33
3.3	Architecture of the learning-based prototype.	35
3.4	User interface of the learning-based annotation tool.	36
3.5	Interface components of the learning-based annotation tool.	37
3.6	First four features around the expected fingertip location.	38
3.7	Tracking results of the optical flow algorithm.	38
3.8	Live results of the automatic classifier.	39
3.9	Components of the geometry-based appearance model.	40
3.10	Geometric modeling of the palm.	42
3.11	Geometric modeling of fingertips.	43
3.12	Geometric modeling of finger “pipes”.	44
3.13	Architecture of the geometry-based prototype.	45
3.14	Performance of the geometric palm model on live data.	46
3.15	Performance of the geometric pipe model on live data.	46
3.16	Performance of the geometric tip model on live data.	47
4.1	Overview of the processing chain.	50
4.2	Result of applying the media filter to the raw input image.	51
4.3	Result of the region segmentation algorithm.	52
4.4	Results of the distance transform procedure.	53
4.5	Result of the circular neighborhood classification procedure.	54

4.6	Example results of the pipe segmentation procedure.	55
4.7	Result of the feature extraction procedure.	57
4.8	Result of the feature connection procedure.	58
4.9	Results of the finger extraction algorithm.	59
4.10	Results of the candidate grouping procedure.	61
4.11	Results of the palm tracking procedure.	63
4.12	State chart of interaction with images.	65
4.13	Illustration of the pop up gesture.	66
4.14	Illustration of the touch gesture.	67
4.15	Illustration of the loose gesture.	68
4.16	Illustration of the grab gesture.	69
4.17	Illustration of the release gesture.	69
4.18	Illustration of the throw gesture.	70
4.19	Overview of the package structure for implementation.	71
4.20	Overview of the data structures for implementation.	73
4.21	Data structures for the fast classification procedure.	74
4.22	Mapping of user interface actions to runtime components.	75
5.1	Experimental setup for the user studies.	78
5.2	Illustration of the introduction task.	79
5.3	Illustration of the 2D puzzle task.	80
5.4	Illustration of the 3D puzzle task.	81
5.5	Personal experience and feedback for the introduction task.	83
5.6	Feedback for both puzzle tasks.	85
5.7	Summary of the answers to the open questions.	87
C.1	Tracking results of the optical flow algorithm (frame 0 – 4).	103
C.2	Tracking results of the optical flow algorithm (frame 6 – 10).	103
C.3	Tracking results of the optical flow algorithm (frame 12 – 16).	104
D.1	Results of the palm tracking procedure (frames t_1 and t_2).	105
D.2	Results of the palm tracking procedure (frames t_3 and t_4).	105
D.3	Results of the palm tracking procedure (frames t_5 and t_6).	106
E.1	<i>Capture</i> tab of the geometry-based workbench.	107
E.2	<i>Inspect</i> tab of the geometry-based workbench.	108
E.3	<i>Live</i> tab of the geometry-based workbench.	108

Chapter 1

Introduction

What are 3D multi-touch interfaces? Why do we need them? What can we expect from them in the near future? This first chapter provides answers to these and related questions. In Section 1.1 the general need for further research in the given domain is provided. Then, in Section 1.2 the objectives of this particular research project are elaborated.

1.1 Motivation

The idea of 3D multi-touch interfaces first gained public interest with the release of the movie *Minority Report* in October 2002. The science fiction film, set in 2054, demonstrated a novel hand gesture-based 3D interface for information exploration, which was used by the police of Washington, D.C. to fight crime. An excerpt from the movie is provided in Figure 1.1 showing the main actor Tom Cruise operating the glove-based environment.

Despite being a topic of Hollywood productions the basic idea has also been picked up by a wide community of researchers due to its potential to fundamentally revolutionize HCI. This development is outlined in Section 1.1.1 by discussing recent directions both of research and product market in the HCI area. A more application-centric approach is followed in Section 1.1.2 by sketching scenarios where 3D multi-touch capabilities could effectively enrich the user experience. Finally, the main problems are highlighted in Section 1.1.3, which are left for researchers to solve until truly robust 3D multi-touch products can be made available to the end consumer market.

1.1.1 Background

Multi-touch interfaces have already become standard HCI elements both for researchers as well as end users. The hype probably started with the introduction of the Apple iPhone¹ in June 2007, which was the first commercial product

¹<http://www.apple.com/iphone/>



Figure 1.1: Excerpt from the movie “Minority Report”.

quality end user device in the multi-touch domain. Since then, many new systems have hit the market in various categories ranging from table surfaces such as the Microsoft Surface [76], to projection-based installations such as the City-Wall [58], to novel classes of desktop PCs such as the HP TouchSmart series². The quick and wide adoption of multi-touch technology is a strong indicator for the flexibility and excellent usability of multi-touch applications in various domains as compared to more traditional interaction devices such as the mouse.

On the downside, as an on-line study shows³, current product quality multi-touch technology is largely limited to two-dimensional surfaces. However, the popularity of three-dimensional content and media has been rising for many years. This development mainly has been fostered by the advancement of hardware components such as graphics cards, integrated gaming consoles or more recently 3D displays as well as sustained end user demand. Consequently, researchers have been trying to integrate both concepts by defining new surface bound multi-touch techniques for manipulating 3D parameters and content such as [20]. Edelmann et. al. show that the result are both robust and intuitive, but interaction is limited to a small set of parameters, in particular the camera position and the perspective. However, for a more diverse and complex set of interactions other techniques are required, e.g. by overcoming the limitation of the two-dimensional surface. For example Grossman et. al. [24] successfully demonstrate basic gestural interaction in three dimensions using a volumetric display and marker-based optical trackers for one thumb and two index fingers. If more accurate hand pose information is required, e.g. more fingers for gesture definition, glove-based approaches find their application such as [23] or the commercial system g-speak⁴ from Oblong Industries, Inc. However, as already

²<http://www.hp.com/united-states/campaigns/touchsmart/>

³<http://www.billbuxton.com/multitouchOverview.html>

⁴<http://www.oblong.com/>

reported in [23], the glove material is of critical importance to the system design due to its potential negative influence on the user's comfort.

This finally leads to the call for barehanded 3D interaction techniques, which provide superior characteristics in terms of comfort, intuition and expressive freedom as opposed to marker-based systems. To achieve barehanded interaction, the tasks of hand detection, hand pose estimation, hand tracking and hand gesture recognition from optical data have to be solved robustly and efficiently at the same time. Recent commercial systems such as the Mgestyk⁵ and Microsoft's Project Natal⁶ have presented the capabilities of true barehanded 3D interaction. However, serious limitations are posed on the number of users or the detail of the hand pose estimations.

Future systems have to build upon the current advancements and improve in the mentioned areas by allowing multiple users to interact simultaneously while relying on highly accurate hand pose information for defining and recognizing a diverse spectrum of hand gestures. As Project Natal demonstrated, in the near future such technology will find its application primarily in the digital gaming world. In the long run, an integration with productivity software for home and office use is also conceivable. However, the success of such integration strongly depends on its usability in terms of efficiency and comfort for the user and has to be set in relation to other emerging approaches.

1.1.2 Scenarios

In today's world of ever emerging digital 3D content and media probably an unlimited number of applications is conceivable for a novel class of intuitive, non-intrusive 3D interaction techniques. In the following, three specific scenarios are highlighted, which have guided the design and implementation of this specific research project. From the individual properties and characteristics of the corresponding system settings concrete technical problems are derived in the next section.

Workspace applications

Augmented workspaces refer to cases where AR/MR technology is introduced into the work place with the goal to improve the overall productivity of the human workers and the work processes. For example, Fiorentino et. al. [22] propose a mixed reality system for improving industrial design processes with a specific application to the automotive industry. A recognized advantage of this approach is the more efficient design work flow due to an integrated set of expert interfaces. On the downside, user input is restricted to a single pen which limits interaction to a single point in space. Hua et. al. [32] extend the ideas of augmented work support to geographically distributed teams, which are a common phenomenon in today's research and development landscape. Interaction is improved by a more diverse set of physical devices including data gloves

⁵<http://www.mgestyk.com/>

⁶<http://www.xbox.com/en-US/live/projectnatal/>

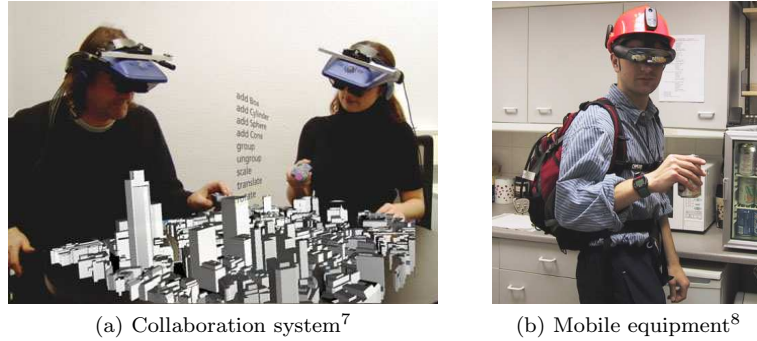


Figure 1.2: Examples of augmented reality systems for work support.

and lens objects, but the results still point to the need for more intuitive input facilities. In contrast to the previous examples, Reitmayr et. al. [61] primarily addressed the problem of mobile interfaces for AR workspace applications, which are a critical component in many outdoor deployments such as excavation or construction sites. In common with the industrial design application, their mobile system features a single pen for direct pointer based 3D manipulation of objects, which is a cheap and fast solution but lacks expressiveness and intuition. Finally, having in mind the sheer unlimited number of possibilities for interaction design, Chastine et. al. [11] conduct a study on the design space for interaction techniques in collaborative augmented environments. The results suggest that pointer-based object referencing methods are an important component, specifically for enabling effective communication between users of physically distributed sites.

In summary, the benefit of augmented workspaces has been recognized in various domains such as industrial design or geographically distributed research and development. Existing systems make use of intrusive input facilities, which are in most cases limited to a small set of interactions. Natural gesture-based interaction with the human hand could improve current systems both in stationary and mobile settings through enabling a diverse set of intuitive pointing and manipulation metaphors.

Educational applications

Augmented learning refers to cases where AR/MR technology is used to communicating knowledge to and reinforcing knowledge on its users. The positive effect of virtual content in education has been recognized in many domains, e.g. Shelton et. al. [69] evaluated AR for teaching spatial phenomena such as our solar system. The study suggests that a considerable increase of the students performance during assessment tasks could be achieved, which is closely related to the way the students interacted with the objects. Similarly, Kaufmann et. al.

⁷<http://www.vr.ucl.ac.uk/projects/arthur/>

⁸<http://www.cl.cam.ac.uk/research/dtg/attachive/ar/>

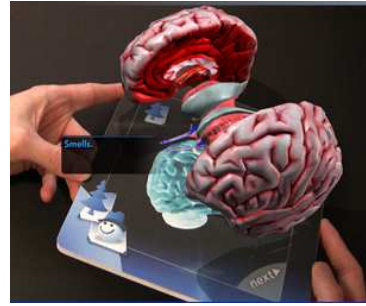
(a) Dinosaur⁹(b) Human brain¹⁰

Figure 1.3: Examples of augmented reality systems for knowledge exploration.

[36] propose an AR setup for teaching abstract geometry in groups of students and teachers. The system facilitates simple pen-based input and experiments show high acceptance and good usability results mainly due to its mature interface design. To go a step further, the idea of collaborative learning is extended to distributed settings by Yuan et. al. [81]. Fast and scalable networking is facilitated by a specific communication protocol, but interaction is still limited to a set of pointing devices. Finally, an economic argument for AR in education is given by Zagoranski et. al. [82] in that virtual replication of experiments is a cheaper and less time consuming alternative to their real world counterparts.

In summary, the benefit of augmented learning has been recognized and evaluated in various domains such as math and physics. Further, the importance of intuitive ways for interaction is highlighted and first mature and usable pen based user interfaces are proposed. Still, non-intrusive barehanded gesture based 3D interaction could improve current systems by enabling a richer, more natural, more plausible way of content manipulation.

Gaming applications

With the ever increasing visual realism of today's computer games, the need for more natural and intuitive interfaces has arisen. As a consequence, almost every major game console developer is publishing motion control capabilities for their platforms in recent years. Probably the first product in this category was the Sony EyeToy¹¹, a digital color camera device for vision-based motion detection and gesture recognition, which was released in July 2003. One technical limitation of the color camera based approach is its reliance on sufficient ambient light conditions for detecting users and body motion. To overcome this problem, newer products such as the Nintendo Wii¹², which was launched in October

⁹<http://tinyurl.com/msaa4n>

¹⁰<http://www.perfectprototype.com/site/archives/360>

¹¹http://www.gamasutra.com/php-bin/news_index.php?story=20975

¹²<http://www.nintendo.com/wii>

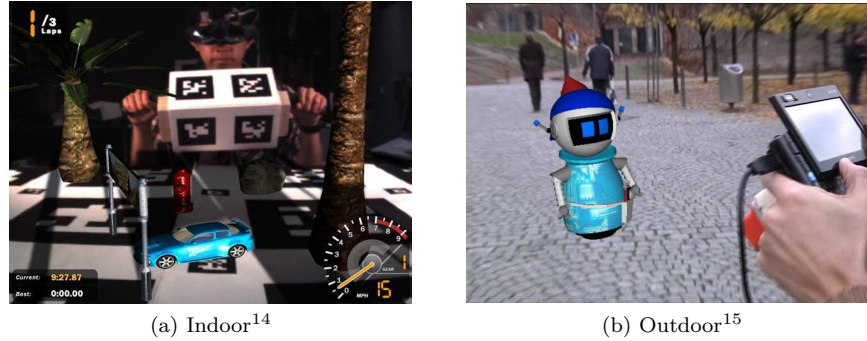


Figure 1.4: Examples of augmented reality systems for gaming.

2006, and the Sony PlayStation Move¹³, which is to be launched in the second half of 2010, favor active light markers on hand-held wireless game controllers for position detection and gesture recognition. Indeed, wireless game controllers are very robust to poor lighting conditions, but interaction might not be perceived as natural and intuitive as a barehanded approach. To overcome the problem of lighting conditions and to deliver a natural and non-intrusive experience at the same time, Microsoft's Project Natal finally introduced active infrared cameras for direct depth sensing and vision-based full-body motion tracking in 3D. The technology has already been demonstrated to support natural avatar control, which enables a whole new dimension of immersive 3D gaming experiences. The product launch is expected in December 2010.

Though featuring a large set of robust technologies and end consumer products, the gaming industry also gives rise to a huge body of research activities. Currently, a major direction in the mobile market segment is the application of AR techniques to deliver virtually augmented experiences on the go. For example, Herbst et. al. [29] propose playful exploration of historical information about cities using GPS-based augmentations with a PDA. Interaction is limited to the buttons on the device which was recognized as a major drawback in the system design. To overcome this problem, Tran et. al. [71] propose vision-based hand interaction using the built-in camera of mobile devices. With this approach a richer experience can be delivered, however, markers are required to support real-time 3D interaction due to the limited hardware resources on the mobile platform.

In summary, a clear trend towards motion based interaction can be recognized in the gaming world. Both intrusive and non-intrusive production quality technologies exist for indoor use with a recent focus on marker-less full-body tracking and direct natural avatar control. The same ideas and principles also find their application in the mobile domain, however, limited hardware resources

¹³<http://us.playstation.com/ps3/playstation-move/index.htm>

¹⁴<http://tinyurl.com/d9huxh>

¹⁵http://www.fit.fraunhofer.de/presse/09-12-16_en.html

and unconstrained environmental dynamics still pose problems, which need to be solved before mobile AR gaming can become a reality.

1.1.3 Problems

Even in the light of emerging technologies such as Microsoft's Project Natal issues remain for research in the area of non-intrusive 3D interaction technology, which need to be addressed in future work. In particular, the following list of open questions has influenced the objective target of this research project:

Interaction design is the first and most fundamental issue that needs to be addressed by all possible applications of 3D interaction technology. Assuming the trend of vision-based motion input, knowing the location and pose of the user does not solve the problem of interpreting his/her intention in the human-computer dialog. On the other side, having a solution for interpreting the user's intention in one concrete application, its suitability to an entirely different scenario might be questionable. In summary, a detailed study on the effects and implications of body motion based input is required to narrow down the design space and give effective guidelines for user and application centric interaction design.

Technology integration forms a second important field for further research. Having integrated full-body tracking into existing gaming consoles to deliver a novel class of gaming experiences for the living room on regular TVs, the question remains how such technology may interplay with other systems such as augmented or virtual reality. Each potential combination might pose a new set of technical challenges and usability issues due to the differences in integration and deployment. In particular, new technologies have to prove robust against intensified environmental dynamics and support significantly larger interaction volumes while maintaining the efficiency and accuracy of existing solutions.

Mobile operation poses the ultimate challenge towards supporting unlimited interaction volumes and gives rise to a novel class of ubiquitous computing applications. While indoor technical challenges generally can be considered solved, outdoor circumstances draw a whole new picture. Most importantly, the availability of computational resources is severely limited as opposed to today's Microsoft XBox and Sony PlayStation gaming platforms, which feature powerful multi-core streaming processors. Novel techniques have to cope successfully with this problem through enabling long-term operation on battery-powered devices while maintaining a high level of accuracy and responsiveness.

This presentation of open issues concludes the motivation for this research project. In the following, the attention is moved to concrete objectives which define the overall scientific contribution of this work.

1.2 Objectives

As discussed previously, despite recent advancements in the gaming industry there are still many problems left to be solved in the area of body-motion based 3D interaction technology. Obviously, within the scope of this work not all problems can be addressed equally well. Therefore, in this section the objectives are narrowed down to allow a fair assessment of the results at the end of the project. Content of the following discussion are the goals respectively non-goals, concrete functional requirements and the hardware setting.

1.2.1 Scope

As previously discussed, existing interaction techniques in the context of digital 3D content and media generally lack usability, while barehanded interaction has the potential to fundamentally revolutionize HCI in this field. To explore this design option, the foremost goal of the thesis is to develop a lightweight real-time algorithm for vision-based non-intrusive human hand processing in 3D. Thereby, resource efficiency is desired to reduce the complexity of the system and allow implementations on standard end-consumer hardware. Alternatively, computing clusters could have been employed as demonstrated by Gump et. al. [25], which deliver high-quality 3D data, but require an entire network of connected computing nodes for solving the task in real-time. A second idea would be to use proprietary hardware components as proposed by de la Hamette et. al. [16], which is generally too expensive in production and distribution. Further, to develop the system prototype a suitable environment for implementing and testing algorithms and their optimizations must be provided. In particular, this environment should include features that support algorithm development and performance comparison. Most importantly a video capturing component must be available to build comprehensive training and test sets. These data sets are required to facilitate the implementation of learning-based methods (e.g. [63, 4]) and statistical approaches such as Particle Filters [34, 33], which are currently expected to provide the most significant improvements.

A second goal of this thesis is to conduct an initial user study on the acceptance of barehanded 3D interaction technology for augmented and virtual reality applications. Therefore a demo application has to be developed on top of the before mentioned real-time algorithm that implements a gesture-based interaction model. The results of the study should be used to draw conclusions about the current state as well as future potentials of the technology. This focus differs from many existing research efforts, which generally aim to study the quality of various tracking/recognition algorithms and their performance optimizations. Instead, this work can be considered a complementary approach to the subject with a fundamental shift from technology to usability. Usability studies form an important part of the scientific contributions in the area of 3D interaction technology as history has shown that computer applications are not only successful due to their functionality but also due to their acceptance among the human users.

1.2.2 Requirements

In this section the general goals of the thesis are broken down to concrete requirements on the individual components of the system. This clear view enables a more systematic evaluation and decision process, which is required to review related work in Chapter 2, compare approaches in Chapter 3 and draw final conclusions in Chapter 6. For clarity, the requirements are grouped by component in the following.

Vision library

This first section summarizes constraints, which have to be met by the core vision library. The library subsumes all components which are required to extract hands from video sequences and track their appearance through frames. Consequently, it forms the basis of all our 3D hand tracking and multi-touch applications.

Real-time operation is clearly *the* most fundamental requirement for this project. Any solution to the real-time 3D hand tracking and gesture recognition problem, may it be as accurate and robust as possible, is not suitable for our use cases if this constraint is not met. Clearly, this imposes a serious limitation on the quality of the hand reconstruction and the tracking capabilities. Fully-fledged high-DOF¹⁶ model-based hand reconstructions have already been presented in the literature [25], though the methods require substantial computational resources, which cannot be provided on standard hardware. As a result, the ultimate goal of this thesis is to develop a truly usable 3D multi-touch prototype with relaxed requirements regarding the quality and robustness.

Limited model refers to the reduced reconstruction detail of the actual hand poses apparent in the image sequences. For example it is not necessary to reconstruct every single joint configuration of the hand, rather landmark points have to be identified robustly. As landmark points primarily the finger tips are considered, which are the most important human features for touch sensation and interaction. Another important piece of information would be the position of the palm and the direction of the arm. Still, any feature should only be used if it can be determined reliably. As discussed earlier, full-DOF model-based estimations are ruled out due to their heavy computational load and its negative impact on operational speed.

Few false-positives means keeping the number of detections that do not refer to an actual hand low. The more often a hand is detected falsely, the less frequently the system will be accepted by its users. Also, the usability of the system will decrease as wrongly triggered user actions influence the interaction session. Ideally, no faulty detections will occur, but this might

¹⁶Degrees of freedom

not be possible to guarantee due to the complexity of the problem. Also, there is usually a trade-off between false-positives and true detections, i.e. reducing the false detection too drastically could diminish the number of correct detections to a level where the system is not functional any more.

Few false-negatives as compared to false-positives means keeping the number of cases low where an actual hand is present in the scene but not detected by the system. Commonly, both false-positives and false-negatives do occur in vision systems, i.e. an entirely fault free system is improbable to be built. So again, the solution is to try keeping this number as low as possible, though, a false-negative will be considered less critical in this study because false-positives have the potential to severely influence the interaction unfavorably. Still, it should be considered that too few actual detections also have the uncertainty to lower the acceptance among the trial users.

Demo application

The second section focuses on the features that have to be available for the user study. The user study is based on a demo application, which implements basic 3D multi-touch interaction techniques on top of the core vision library.

Multiple users/hands are another important requirement, which stands in line with the vast amount of existing multi-touch products. Multiple users have to be supported for enabling collaborative applications like learning environments, meeting rooms or development spaces. Integration of multiple hands per user should instead provide the means for more natural and intuitive interaction. This effect has been studied thoroughly in the literature (e.g. [43, 70]). As a consequence, two-handed gestural commands are considered advantageous in various interaction scenarios, for example, when rotating or scaling objects or arranging items simultaneously. However, one-handed interaction remains useful, e.g. when simply pushing buttons or throwing stuff off screen.

Basic gestures should be recognized on top of the detection, pose estimation and tracking module. For this reason, the recognizable gestures will be limited by the reconstruction detail and robustness, which is delivered by the core vision component. Generally, typical multi-touch gestures should be implemented such as moving objects, rotating and scaling them and moving them off the screen respectively operational 3D space. The idea is not to develop a fully-fledged all-purpose gesture recognition module as demonstrated e.g. with Hidden Markov Models (HMM) [37, 21], but rather to provide a first functional means for evaluating the 3D interaction techniques from the perspective of the user.

Basic application finally denotes a truly usable demo which integrates all the above aspects into a single deployable piece of software. The purpose of

this artifact is twofold: First, it should assist in collecting user feedback for qualitative system evaluation. Second, it should demonstrate the power and possibilities of 3D multi-touch approaches for future applications and markets. I.e. an integrated concept for applying the basic gestures, which have been defined in the previous requirement is desired. Though, the system does not have to serve an urgent purpose, i.e. its operation is not required besides the demonstration scope. Affinity to the use cases presented in Section 1.1.2 is favorable but not necessary.

1.2.3 Hardware

Finally, the question arises which hardware components should be used in the experiments. This applies to both the processing resources (1) such as CPU and main memory as well as the camera device (2). The decisions mainly have been influenced by the desire to prefer general purpose standard hardware over highly efficient, but expensive proprietary solutions. As a consequence, a regular desktop PC with an IntelTM Core 2 Quad 2.83GHz processor and 3.25GB RAM was used as the processing platform (1) for implementing and testing the real-time algorithm. It should be noted that a highly efficient graphics processing unit (GPU) with an array of stream processors is not required.

Regarding the camera component (2) exclusively devices with 3D video capturing capabilities have been tested, namely binocular stereo rigs and time-of-flight depth sensing arrays. The reason is that depth segmentation was identified as one of the major strategies for efficient hand detection under uncontrolled lighting conditions (see Section 2.2.1). Additionally, the depth information provides an important cue for deriving hand configuration parameters such as finger lengths and the palm size, which significantly reduces the amount of ambiguity. However, it should be noted that algorithms exist that are able to cope with monocular video sequences by continuously calculating probabilities for a set of configurations rather than deriving only a single hypothesis [15]. However, this is out of the scope of this thesis.

Finally, the cameras are presented, which have been used during prototyping and evaluation. A low-budget consumer stereo webcam known as Minoru¹⁷ was employed (see Figure 1.5a). This device provides a relatively stable stereo rig with a simple API for accessing synchronized stereo data streams. In contrast, for direct depth sensing the time-of-flight camera SwissRanger SR4000¹⁸ was used, which is displayed in Figure 1.5b. It is one of the few available devices in this category as the technology is quite new and subject to several patents. Nevertheless, the launch of Microsoft's Project Natal (see Section 1.1) supports the hypothesis that the market segment will grow in the near future.

To conclude this chapter, shortly the question should be addressed how the depth information is obtained from those cameras. As mentioned previously, the SwissRanger SR4000 captures a depth map by applying the time-of-flight

¹⁷<http://www.minoru3d.com/>

¹⁸<http://www.mesa-imaging.ch/>

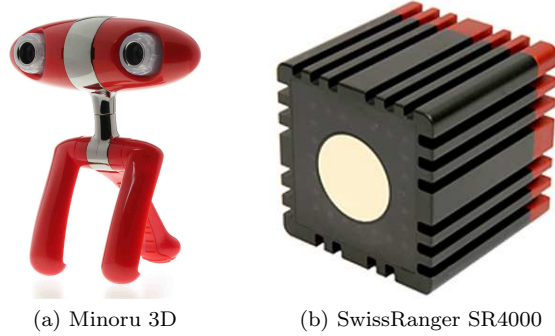


Figure 1.5: Cameras used during preparation and realization.

principle [44]. More precisely, the camera actively sends a modulated light signal within the infrared spectrum and measures the time it needs to be reflected for each image pixel. The reflection time is measured as a phase shift of the original pattern, which limits the maximum distance range perceivable by the device. Our model is configured to work with a distance range between 0.8 and 5.0 meters. For stereo cameras, stereo matching algorithms have to be used instead to retrieve the depth information. This can require heavy computational resources depending on the chosen algorithm and its implementation. More details can be found in the section about basic techniques (Section 2.1).

Chapter 2

Retrospection

This second chapter serves two purposes: First, basic techniques in the area of computer vision are introduced, which are necessary to understand the algorithmic approaches and 3D multi-touch prototype realization in Chapter 3 and 4. Second, related work is presented from the area of human hand detection, hand tracking, hand pose estimation and gesture recognition. The lessons learned from previous research have guided the system design and implementation processes throughout this thesis.

2.1 Basic techniques

The discussion of basic techniques is carried out in four steps: First, in Section 2.1.1 mathematical models for images and cameras are introduced, which serve as the basis of subsequent technical specifications. The models result from the systematic application of mathematical concepts such as sets and mappings to describe and understand the process of analog and digital image formation. In Section 2.1.2 the previous models are applied to the problem of depth recovery from both color and gray-scale image data. Obviously, theoretical and practical depth recovery represents a vital part of 3D vision systems like the multi-touch prototype which is being developed in this thesis, in particular, if stereo cameras such as the Minoru are employed (see Chapter 1). In Section 2.1.3 the focus of the discussion is shifted to the process of feature extraction and classification. The presented ideas have been used extensively throughout the implementation to generate abstractions and interpretations of the image contents. Examples of features include low-level peculiarities such as corners or edges, but also higher-level compound structures such as regions of connected pixels and entire objects. Finally, Section 2.1.4 covers the main algorithms that have been used during realization of the 3D multi-touch prototype (see Chapter 4). The algorithms provide efficient strategies for calculating specific information from the raw visual data, such as the previously mentioned regions of connected pixels or the motion of pixels respectively objects in the image plane.

2.1.1 Mathematical models

Image modeling

At the heart of all considerations stands a mathematical model for representing images of any kind. This model is used subsequently to unambiguously specify the processes that are applied to image data and describing their effects. However, it should be noted that the presented formalism is not the only possible way to introduce and define the corresponding structures. Rather, the formalism is subject to the researchers creative freedom and should serve the primary purpose of effective knowledge transfer.

In the real world images are two-dimensional constructs that carry color information at each point of the canvas. From this description, the two defining domains *location* and *color* can be extracted. Mathematically, location refers to elements of the two-dimensional space domain $\mathcal{S} \subseteq \mathbb{R}^2$. The location is usually measured in some metric like *meters* or *inches*, which is neglected at this point. Instead, color is a more difficult entity for modeling as the many color systems that have been developed in the past unfold. Here, a very basic definition of the color space $\mathcal{C} \subseteq \mathbb{R}$ as a one-dimensional intensity scale is used. This definition obviously applies to any kind of single-channel image data such as gray-scale images, infrared images and many more.

Using these two basic domains the notion of images $\mathbf{I}$ can be defined as a mapping from the two-dimensional location domain $\mathcal{S}$ to the color domain $\mathcal{C}$, i.e.

$$\mathbf{I} : \mathcal{S} \longrightarrow \mathcal{C}$$

Images are modeled as simple mathematical functions which makes tools such as differential analysis applicable. Finally, it should be noted that in case of digital images the location domain is discrete. As a consequence, the mapping $\mathbf{I}$ represent a specific sampling of the original visual signal.

Camera modeling

Having a general mathematical model for images the next questions is how cameras such as the Minoru and the SwissRanger SR4000 (see Chapter 1) project scene points onto it. For both cameras - and many others - the digital image is geometrically equivalent to a 2D sensor array, which is used to capture the local intensity and concentration of light beams bundled by an optical lens.

To describe the position and alignment of the camera in a three-dimensional scene, the camera is modeled as a coordinate system C with origin O and three orthonormal coordinate axes, i.e.

$$C = (O, \vec{x}, \vec{y}, \vec{z}), O, \vec{x}, \vec{y}, \vec{z} \in \mathbb{R}^3$$

Note that for simplicity all scene points will be expressed relative to this coordinate system. Within this camera-anchored coordinate frame the sensor array is modeled as a plane $\mathcal{I}$ such that

$$\mathcal{I} = \{\vec{p} \in \mathbb{R}^3 : \vec{z} \cdot \vec{p} + d = 0\}, d \in \mathbb{R}^+$$

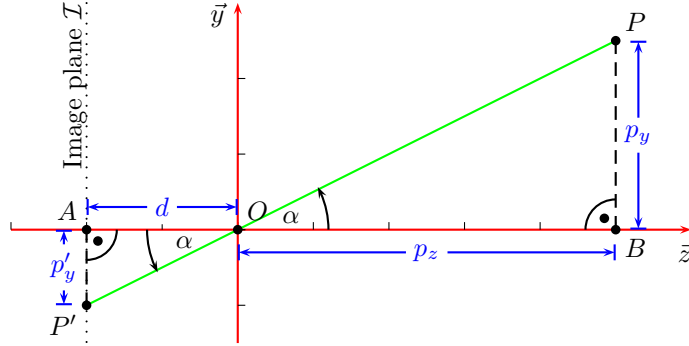


Figure 2.1: Geometric projection of the pinhole camera model.

i.e. the plane I is located parallel to the $\vec{x}$ and $\vec{y}$ axes at distance d in negative $\vec{z}$ direction. This very basic configuration is widely known as the *pinhole* camera model.

The projection of scene points P onto the image plane $\mathcal{I}$ in this model is finally defined by a simple reflection of P through origin O towards $\mathcal{I}$ as illustrated in Figure 2.1. In the diagram the camera axes $\vec{y}$ and $\vec{z}$ coincide with the regular Cartesian paper coordinate axes y and z , the camera axes $\vec{x}$ is omitted since it lies orthogonal to the paper plane. The sensor array $\mathcal{I}$ is reduced to a vertical line at distance d in negative $\vec{z}$ direction. The projection P' is obtained by intersecting line $\overline{PO}$ with $\mathcal{I}$.

At last, through basic geometric equalities the underlying transformation T can be derived in matrix notation using homogeneous coordinates as

$$P' = T \cdot P = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{-d} & 0 \end{pmatrix} \cdot \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

This equation reveals that a basic projective projection matrix with focal distance parameter d is used in the *pinhole* camera model. Other, more sophisticated models exist, e.g. for incorporating radial distortion or depth blur effects. In general, the problem is reduced to a specific set of numeric parameters, which have to be estimated for each camera instance individually using calibration procedures. Closed form solutions for parameter estimation rarely exist, but commonly iterative optimization algorithms are employed. Many implementations exist in standard vision toolkits such as the Open Computer Vision Library (OpenCV)¹, this is why details of the optimization procedures are not covered here. Finally, it should be noted that the SwissRanger SR4000 is shipped with a calibrated camera model.

¹<http://opencv.willowgarage.com/>

2.1.2 3D vision

Stereo vision

Having a basic understanding of the nature of images and cameras, the next question is how to recover depth, i.e. the original distance of projected scene points to the camera. In the case of the SwissRanger SR4000 this is a trivial task since each image pixel inherently carries depth information. For regular color cameras, however, the depth cannot be deduced from the isolated pixel information solely. This limitation becomes clear when considering the continuum of scene points P which map to a given image point P' in the *pinhole* camera model, i.e.

$$\begin{pmatrix} p_x \\ p_y \end{pmatrix} = \begin{pmatrix} -\frac{p'_x}{d} p_z \\ -\frac{p'_y}{d} p_z \end{pmatrix} = \begin{pmatrix} -\frac{p'_x}{d} \\ -\frac{p'_y}{d} \end{pmatrix} p_z$$

This equation reveals that given focal distance d and image point P' all possible scene points P can be deduced as a linear function of their $\vec{p}_z$ component. Obviously, there are infinitely many choices, which makes it impossible to estimate the true depth without placing further constraints on the scene geometry.

To generally cope with the problem of 3D coordinate reconstruction from gray-scale or color images, *stereo vision* techniques have been developed. The basic idea is to capture the scene from two different viewpoints just like the human visual system perfectly exemplifies. Initially, the problem of stereo vision has been formalized in the mathematical framework of the *epipolar* geometry, which captures the basic geometric relationships between scene points and their projections in a system with two observers. For further considerations, a slightly modified version of the *pinhole* camera model is employed, which places the image plane $\mathcal{I}$ at distance d in **positive** $\vec{z}$ direction. This modified version amounts to congruent projection results rotated by 180° around the $\vec{z}$ axis.

The epipolar geometry defines two instances of this camera model with two independent coordinate systems $C_1 = (O_1, \vec{x}_1, \vec{y}_1, \vec{z}_1)$ and $C_2 = (O_2, \vec{x}_2, \vec{y}_2, \vec{z}_2)$ and the respective image planes $\mathcal{I}_1$ and $\mathcal{I}_2$ as illustrated in Figure 2.2. The point P'_1 is constructed by intersecting the image plane $\mathcal{I}_1$ with the line $\overline{O_1 P}$. Analogously, point P'_2 is constructed using the origin O_2 of the second camera coordinate frame. The points E_1 and E_2 are constructed by intersecting $\overline{O_1 O_2}$ with image plane $\mathcal{I}_1$ respectively $\mathcal{I}_2$, which corresponds to a projection of the camera frame origins onto the respective other image plane. Because of the importance of these to points, a special name is given to them: The *epipoles*. The nature of the epipoles E_1/E_2 and the scene point projections P'_1/P'_2 explains the meaning of the lines $\overline{E_1 P'_1}$ and $\overline{E_2 P'_2}$: Reverse projection into space yields that $\overline{E_1 P'_1}$ is the projection of $\overline{O_2 P}$ onto $\mathcal{I}_1$ and $\overline{E_2 P'_2}$ is the projection of $\overline{O_1 P}$ onto $\mathcal{I}_2$. E_1 and E_2 are always part of these lines for an arbitrary point P , which is why they are called the *epipolar* lines. Finally, the lines $\overline{E_1 E_2}$, $\overline{E_1 P}$ and $\overline{E_2 P}$ form a triangle which is called the *epipolar* plane. This plane intersects with both image planes $\mathcal{I}_1$ and $\mathcal{I}_2$, which results in the epipolar lines.

Hartley et. al. [28] show that knowing the basis transformation between the two coordinate systems C_1 and C_2 (expressed as rotation R and translation T),

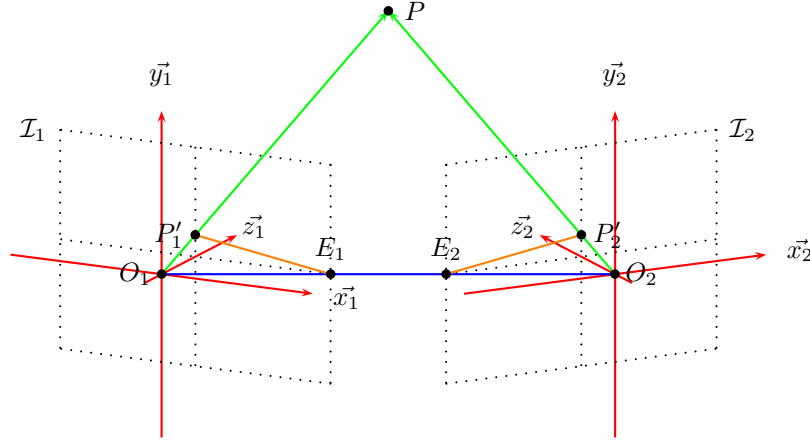


Figure 2.2: Geometric setting of the epipolar geometry framework.

given an arbitrary image point P'_1 , the epipolar line $\overline{E_2P'_2}$ can be reconstructed, on which the image point P'_2 is expected to lie in the second image plane. The formulations amount in the definition of the *fundamental* and *essential* matrices, which model the relationship as linear equation systems. The parameters of the model are usually estimated using iterative stereo calibration algorithms, e.g. OpenCV ships with a working implementation, which has been used for calibrating the stereo web-cam Minoru (see Chapter 1).

Depth estimation

For concrete depth estimation from a pair of time-synchronized color images *stereo matching* algorithms have to be employed. Stereo matching usually exploits the fact that for any projection P'_1 on $\mathcal{I}_1$ the corresponding projection P'_2 on $\mathcal{I}_2$ is located on the epipolar line $\overline{E_2P'_2}$, which can be modeled without prior knowledge of P'_2 . As a result, depth estimation amounts to finding matches along the epipolar lines rather than scanning the entire image.

In the following, without loss of generality a mainly horizontal arrangement of the cameras is assumed. This arrangement allows the alignment of the epipolar lines with the horizontal scan-lines of the two images using image rectification techniques as discussed by Hartley [27]. Consequently, the search along the epipolar lines reduces to a simple iteration along the pixels of a single scan-line, which can be implemented very efficiently.

After having established a match between P'_1 in $\mathcal{I}_1$ and P'_2 in $\mathcal{I}_2$ successfully, the final task is to derive the distance p_z of the original scene point P to the camera centers as illustrated in Figure 2.3. In the setting, the displacement δo_x between the origins of both camera coordinate frames is known as well as the focal distance d , which is assumed identical for both cameras. Then, Q is

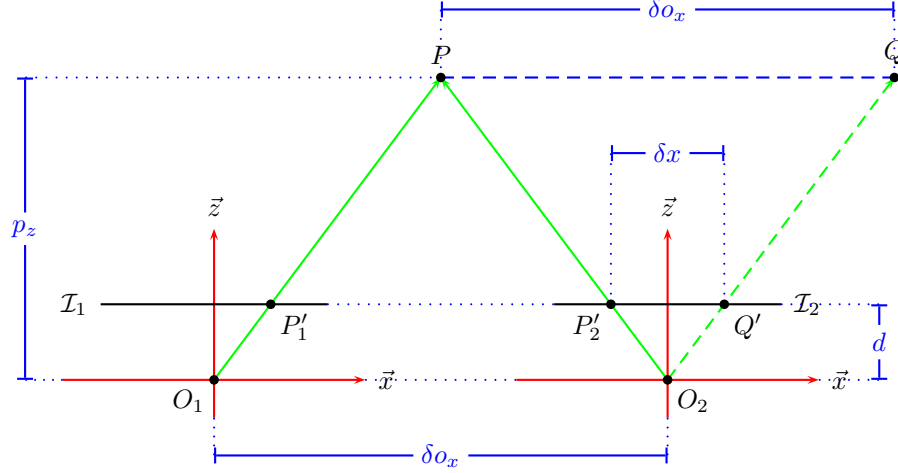


Figure 2.3: Geometric solution of the depth reconstruction problem.

constructed by shifting point P parallel to the $\vec{x}$ axis for δo_x units. As a consequence, the lines $\overline{PQ}$ and $\overline{O_1O_2}$ are parallel and their lengths are identical. In contrast, the point Q' can be found in various ways: The first is to project Q onto $\mathcal{I}_2$, which is indicated by the ray $\overline{O_2Q}$. Alternatively, point P'_1 can be shifted parallel to the $\vec{x}$ axis for δo_x units due to the parallelogram $\square O_1O_2QP$ and the identical focal lengths of the cameras. On that account it can additionally be concluded, that the $\vec{x}$ component of P'_1 in C_1 equals to the $\vec{x}$ component of Q' in C_2 .

Finally, due to the similarity of the triangles $\triangle O_2Q'P'_2$ and $\triangle O_2QP$ the distance p_z of point P can be derived as

$$p_z = \delta o_x \frac{d}{\delta x}$$

This basic relationship between the depth p_z of scene point P and its so-called disparity δx between the projections P'_1 and P'_2 is exploited by many depth estimation algorithms. The two key problems these algorithms have to deal with are point matching and performance boundaries. Obviously, finding the best matches in two distinct images is not a trivial task, especially when optimizing over several scan-lines. Generally, the search space for this procedure is quite large, which poses a serious performance problem. Nevertheless, many good quality approaches exist [74, 80, 77], and lately GPU-based techniques contributed to highly efficient implementations. *Belief propagation*, as for example presented in [79], currently seems to deliver the best result as evaluated by the Middlebury stereo vision benchmark².

²<http://vision.middlebury.edu/stereo/>

2.1.3 Image features

Feature extraction

Feature extraction from raw image data is one of the most fundamental tasks in computer vision and applied consequently in the approaches, which are presented in Chapter 3. In this regard, the term *feature* refers to any kind of information which can be derived from the image contents, e.g. edges as found by the Canny Edge Detector [10], a widely employed algorithm for computer vision applications.

For further considerations a mathematical formalism for describing features and the feature extraction process is introduced. In the most general case, a feature $f \in \mathcal{F}$ is an element of an arbitrary feature space $\mathcal{F}$. The process of feature extraction from image I can be defined as a mapping X^I from the space domain $\mathcal{S}$ to the feature domain $\mathcal{F}$ with

$$X^I : \mathcal{S} \longrightarrow \mathcal{F}$$

This mapping does not have to be defined at every possible image location, i.e. only a subset of pixels is required to map to actual features. This property is heavily exploited by interest point detectors like the Harris Corner Detector [26]. Interest point detectors generally guide the attention of vision algorithms to a small subset of locations, which usually increases the system performance.

For prototyping, a more comprehensive feature detector was employed, which is based on Harris Corner Detector: Speeded-up Robust Features (SURF) [7]. As in the Harris Corner Detector, the SURF algorithm uses the Hessian matrix for calculating a local response for every pixel $P \in \mathcal{S}$. The Hessian matrix holds the responses of convolution with second order Gaussian derivatives with variance $\sigma \in \mathbb{R}^+$ and mean $\mu = P \in \mathcal{S}$. By varying σ , multiple scales are covered by the search, which results in a three-dimensional response volume R with

$$R : \mathcal{S} \times \mathbb{R}^+ \longrightarrow \mathbb{R}$$

Interest points are then selected across the image using a non-maximum suppression scheme, i.e.

$$X_{SURF}^I(x, y) \text{ is defined } \Leftrightarrow \exists s \in \mathbb{R}^+ : R(x, y, s) > R(x + \delta x, y + \delta y, s + \delta s)$$

In the second step, for each interest point P an orientation $\alpha \in [0, 2\pi]$ is derived using the response of horizontally and vertically aligned wavelets. The orientation is used for fitting a rectangular region around the interest point, which defines a local rotated coordinate frame. The frame is then split into patches and for each patch the summed gradient direction $\sum d_{x,y}$ and its polarity $\sum |d_{x,y}|$ is calculated relative to angle α . These values form the entries of the initial feature descriptor f with

$$f \in \mathcal{F} \subseteq \mathbb{R}^{64}$$

The feature vector is finally normalized to achieve robustness against varying illumination conditions and contrast settings.

To conclude the section about feature extraction, it should be noted that several new feature types will be presented in Chapter 3 and 4, which have been designed and implemented in this thesis for detecting human hands in the scene. These include tips, pipes and areas of consistent depth, all of which representing important perceptual concepts of a human hand. However, these features are still in conceptual stage requiring detailed performance studies and feedback from applications. Also, the tip and pipe features are not yet scale independent, but possibilities for improvements exist.

Statistical classification

Having extracted the dominant features from an image the question remains what those features actually represent. The previous definition only states that features $f \in \mathcal{F}$ are distinct elements of an arbitrary feature space $\mathcal{F}$. Usually, the feature space $\mathcal{F}$ is defined as an n -dimensional metric space $\mathcal{F} \subseteq \mathbb{R}^n$ which allows the notion of *distance* as a function D with

$$D : \mathcal{F} \times \mathcal{F} \longrightarrow \mathbb{R}_0^+$$

The distance function allows to express the similarity between features, where low values indicate strong similarity and high values indicate little similarity.

A more powerful technique to approach the similarity problem is given by the framework of *statistical classification*, or *pattern recognition*. Here, features are divided into a discrete set of classes - or labels - $l \in \mathcal{L}$ and training procedures are used for deriving an automatic classifier A with

$$A : \mathcal{F} \longrightarrow \mathcal{L}$$

The training procedures generally depend on an annotated corpora of samples which are modeled as a mapping B of features to classes/labels, i.e.

$$B : \mathcal{F} \longrightarrow \mathcal{L}$$

The classifier A and the training corpora B are defined identically. This is not surprising, when considering, that the purpose of the classifier A is to imitate the training corpora B as closely as possible.

The most important step in constructing statistical classifiers is to choose a functional representation for the mapping A . Generally, a probabilistic maximization function is used, which selects, given feature $f \in \mathcal{F}$, the label with maximum probability, i.e.

$$A(f) = \arg \max_{l \in \mathcal{L}} P(l|f)$$

Another very popular technique that was utilized in the first approach (see Section 3.2) is the *nearest neighbor* pattern classification rule as described in [14]. Strictly speaking, it is a non-parametric model used in statistics when few

training samples are available and the true nature of the underlying distribution is unknown. The general classification rule is as simple as

$$A_{nn}(f) = B(\arg \min_{f' \in \mathcal{F}} D(f, f'))$$

i.e. the label $l \in \mathcal{L}$ of the nearest training sample $f' \in \mathcal{F}$ is selected. It has been shown that the nearest neighbor classification rule delivers optimal classification performance given that the number of training samples approaches infinity. Due to the asymptotic performance and the simple formulation the rule is generally a good starting point for evaluating the capabilities of classification-based approaches.

2.1.4 Vision algorithms

Flood fill

The first algorithm, which is presented in this overview, is called *flood fill*. Flood fill originally was developed for bitmap painting applications with the purpose to fill bounded regions of equal intensity respectively color with a new user selected value. Here a variation of the algorithm is discussed, which relaxes the original equality constraint to a deviation range and redirects repainting commands to a second mask image, with the purpose to implement efficient image segmentation on noisy depth data.

The deviation range is modeled as maximum lower and upper deviation threshold $d_-, d_+ \in \mathbb{R}$, which are used to define the connectivity between neighboring pixels as a boolean predicate CONNECTED as presented in Algorithm A.1. In contrast, the mask M is a simple binary image, i.e. its entries are either zero or one, which serves the purpose of marking those pixels belonging to the filled/connected region. At the beginning of the flood fill procedure, M is required to be initialized to zero at all locations. After termination, the pixels, which belong to the segmented region $\mathcal{R} \subseteq \mathcal{S}$, are marked with one, i.e.

$$\forall P \in \mathcal{S} : P \in \mathcal{R} \Leftrightarrow M(P) = 1$$

Based on the previous parameters, data structures, boolean predicates, constraints and a seed pixel location $(s_x, s_y) \in \mathcal{S}$ the procedure FLOODFILL is finally defined as presented in Algorithm A.2.

The algorithm clearly defines a recursive progression scheme such that in each call to the function (coincides with one iteration) a single pixel is examined for inclusion into the mask M . To prevent infinite loops, first the condition $M(s_x, s_y) = 1$ is tested, which reveals whether the pixel was visited in a previous iteration. If that is not the case, the pixel is added to the mask and all neighbors, which fall into the maximum deviation range, are scheduled for recursive processing. In Algorithm A.2, the pixels along the four cardinal directions are considered as illustrated in Figure 2.4a. This scheme is also called *4-connected*, because the center is connected to four neighbors. Alternatively,

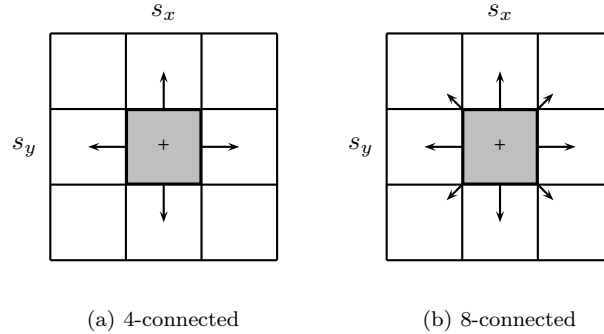


Figure 2.4: Neighborhood schemes of the flood fill algorithm.

some vision systems use an *8-connected* scheme as presented in Figure 2.4b, which has different performance and quality properties.

To conclude the discussion of the flood fill algorithm, it should be noted that for the purpose of image segmentation various other techniques exist, which approach the problem from different directions. For example, Comaniciu et. al. [13] map image features to probability densities and search for modes using the mean shift procedure. A comprehensive review of existing methods is provided in [57], covering learning-based approaches like artificial neural networks, which seem to deliver the most stable results even in noisy environments.

Distance transform

The second algorithm that is presented in this overview is called *distance transform*. Distance transforms find their application in many fields of computer science such as computer vision, robotics or theoretical computer science. The algorithm's basic underlying idea is to transitively calculate the distance of nodes respectively image locations to obstacles. The distance information can be utilized in many ways, e.g. for matching object shapes based on the Chamfer Distance as originally introduced by Barrow et. al. [6].

The representation of obstacle information is generally variable. However, in computer vision applications usually a binary obstacle map l is used with

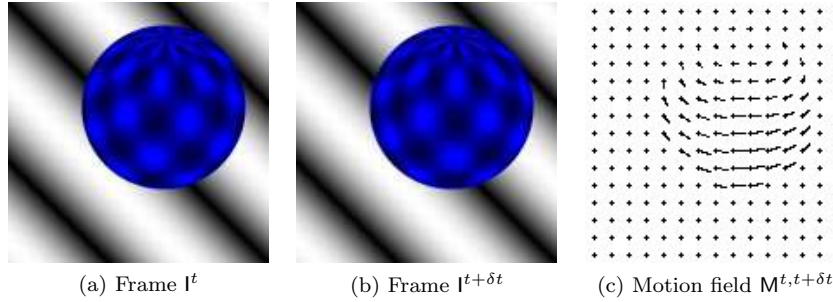
$$l(x, y) = \begin{cases} 1, & \text{if obstacle} \\ 0, & \text{else} \end{cases}$$

The distance transform procedure, which is provided in Algorithm B.1, requires two passes over l to calculate the overall smallest distances to obstacles at each image location. Among the passes the direct neighborhood of pixel $P = (x, y) \in S$ is divided disjointly into a top-left set

$$\mathcal{N}_P^{\nwarrow} = \{(x - \delta x, y - \delta y), (x, y - \delta y), (x + \delta x, y - \delta y), (x - \delta x, y)\}$$

and a bottom-right set

$$\mathcal{N}_P^{\nearrow} = \{(x - \delta x, y + \delta y), (x, y + \delta y), (x + \delta x, y + \delta y), (x + \delta x, y)\}$$

Figure 2.5: In- and output of the optical flow algorithm³.

The first pass uses $\mathcal{N}_P^{\nwarrow}$ to determine the minimal distance to any obstacle, which is located above it or at the same height left to it. The second pass uses $\mathcal{N}_P^{\nearrow} \cup \{P\}$ to determine the overall smallest distance as the minimum of the distance, which found in the first pass (already stored in $O(x, y) \in \mathbb{R}_0^+$), and the smallest distance to any obstacle, which is located below it or at the same height right of it. Throughout the algorithm the distance information is kept in the map O , whose entries are initialized with ∞ . For distance measurements, the distance metric D is used as introduced in Section 2.1.3.

The algorithm is based on the idea of *dynamic programming* because it is reusing the distance measurements from both previous pixels and previous iterations in each pass, which makes it highly efficient for implementation and encourages its use in real-time vision systems.

Optical flow

The last section in this overview is dedicated to *optical flow*. Optical flow techniques aim at the estimation of motion, which is apparent in the image plane, for the purpose of object tracking across the frames of a video sequence. Motion typically occurs for two reasons: Observer movement and/or scene object movement. Generally, it is represented as a two-dimensional vector field M assigning a velocity, also called displacement (per time difference δt), to each pixel of frame I^t as depicted in Figure 2.5c. This vector field associates each pixel of frame I^t to a displaced location in frame $I^{t+\delta t}$ as shown in Figure 2.5.

The algorithms that have been developed for solving the optical flow problem are based on a formal definition of the motion field, which will introduced only briefly: First, two time-consecutive frames I^t and $I^{t+\delta t}$ of the input video stream are required, which are separated by time difference δt . To introduce the temporal dimension t as a functional parameter, the sequence of images (or video stream) is further modeled as a mapping V from a three-dimensional number plane to the gray-value scale, i.e.

$$V : \mathcal{S} \times \mathbb{R} \longrightarrow \mathbb{R} \text{ such that } V(x, y, t) = I^t(x, y)$$

³<http://of-eval.sourceforge.net/>

In contrast, the motion field D depends on the two time points t and $t + \delta t$, for which the optical flow should be calculated. Secondly, its motion vectors are assigned to specific locations in the image plane. Accordingly, M is defined as a mapping from the image locations to 2D displacement vectors with time points as subscripts, i.e.

$$M^{t,t+\delta t} : \mathcal{S} \longrightarrow \mathbb{R}^2$$

The goal of any solution to the optical flow problem now is to derive a plausible solution for the entries of field $M^{t,t+\delta t}$, given the data observed in V .

Probably the most well-known algorithm for solving optical flow was proposed by B. Lucas and T. Kanade [51]. Its basic idea is to require local smoothness of the optical flow field. The method yields a linear system of equations for each pixel, whose solution is given by the two components δx and δy , which are assumed constant for the local sub-window. A critical review of optical flow algorithms is provided in [48]. An evaluation of gradient-based optical flow methods is presented in [54].

2.2 Related work

The related work section covers concrete solutions for processing the appearance of human hands, which can be divided into four categories: Hand detection (1) as presented in Section 2.2.1 deals with the problem of finding hand appearances in images without prior knowledge of their existence and usually marks the first step in a chain of integrated algorithms for utilizing hand information in computer applications. Pose estimation (2), as presented in Section 2.2.2, concentrates on techniques, which allow to derive the pose of the human hand from the appearance information only. Pose usually refers to the configuration of the joints as well as the position and rotation of the hand in general. Hand tracking (3) as presented in Section 2.2.3 focuses on following the appearance of hands through sequences of frames. Advanced algorithms in this category also derive the sequence of poses given an estimate of the initial pose. Gesture recognition (4) as presented in Section 2.2.4 finally interprets the previous sequences of successfully tracked hand appearances as intentional commands or signs for the purpose of human-computer interaction.

2.2.1 Hand detection

As stated previously, hand detection aims at finding hand appearances in images without prior knowledge about their existence. Several techniques exist, ranging from simple color classification and region selection, to textural and structural analysis. The most prominent technique is to model the skin color probability distribution as for example presented by Donoser et. al. [19]. The basic underlying idea is to weight each image pixel according to its skin color likelihood, yielding a grayscale map on which bright regions give clues to possible hand appearances. In the next step, the regions are extracted using image

segmentation techniques such as the Maximally Stable Extremal Region operator, which was introduced by Matas et. al. [53] for robust region based stereo matching.

One problem with this simple color distribution scheme is its susceptibility to changing lighting conditions. As a result, the respective systems only work reliably in controlled laboratory settings, which prohibits their use in location independent settings. To overcome this problem, Rosales et. al. [63] adapt the skin color model using data obtained from a neural-network based face detector such as [65]. A similar approach has been described by Kim et. al. [38] using the popular Ada-boost algorithm [75]. However, the application of this technique prescribes that the user is standing in front of the camera, which entirely rules out head-mounted installations.

A general problem with the previous skin color classification schemes is that background objects with similar color distributions have to be avoided, which might not be possible in unconstrained environments. Therefore, Barhate et. al. [5] enhance the skin color cue with a second motion cue to make the detector more robust. A similar approach was proposed by Kim et. al. [39], who employed difference images and color thresholding to segment moving hands. The resulting binary image is post-processed using morphological operators yielding the final hand blobs. Still, the previous techniques are susceptible to highly dynamic environments with uncontrolled motion in the background.

To make hand detection more robust to everything ranging from background clutter to background dynamics to lighting conditions, Delamarre et. al. [17] propose depth segmentation from dense stereo reconstruction. This approach relies on the assumption that the hand is the closest object in the scene from point of view of the cameras. From depth information the authors additionally expect higher robustness against intra-object occlusions for pose estimation due to the richer visual information. One disadvantage of the approach is the complexity of dense stereo reconstruction, which makes real-time implementations difficult to realize on standard hardware. To overcome this problem, Liu et. al. [49] rely on specialized hardware such as the FingerMouse [16], which was ruled out in Chapter 1 due to its cost and reduced usability.

Finally, a promising approach for hand detection has been proposed by Kolsch et. al. [41], which is based on the popular face detector developed by Viola and Jones [75]. The basic idea is to train a cascade of weak classifiers using the Ada-boost algorithm where each classifier matches a subset of image region for a hand being detected. The classifiers operate on integral images for fast filter response calculation at multiple scales. For correct detection the hand has to be presented in a particular posture. Generally, the cascade can be trained for any possible posture. Which posture is suited best for hand detection depends on the final classifier's performance. Therefore, the authors propose a method for estimating the classifier's performance beforehand based on the Fourier frequency spectrum of manually segmented posture samples.

In summary, various techniques for hand detection exist, ranging from color classification, to depth segmentation, and learning-based approaches. Depth-based segmentation seems to be a good starting point for the prototype due

to its robustness against various environmental influences while requiring no training from annotated corpora.

2.2.2 Pose estimation

The goal of pose estimation is to approximate the hand configuration - which is for example constituted of the individual joint angles - as closely as possible from the image observations. The set of configuration parameters usually depends on the hand model employed and varies in its degrees of freedom (DOF). Due to its efficiency, partial hand pose reconstruction has been used in many real-time systems. The basic idea is to only estimate the location of important landmark points such as the palm or the index finger. For example, Sato et. al. [66] propose to derive the palm position as the centroid of the hand blob. A more stable estimate is given by the point with the maximum distance to all boundary edges as presented by Abe et. al. [1]. In contrast, fingertips can be found using circular masks [56, 46], curvature analysis [68], or palm-edge distance transform analysis [55], which is considered more robust to segmentation noise.

Full pose reconstruction, on the other hand, is a more complex problem due to the increased number of parameters. An elegant solution was proposed by Rosales et. al. [63], who employ a database of synthetically generated pose samples to train a set of specialized back mapping functions into the configuration space. For training, each synthetical image is annotated with the respective hand configuration parameters. Based on the annotated corpora, the mappings are optimized to recover pose information from predefined image features such as the Hu Moments [31]. The individual mapping functions specialize iteratively for a specific subset of feature configurations. The credibility of the estimation is expressed in terms of the match correlation. A problem with this approach is the huge variability in articulation and appearance, which requires an extensive database of training samples.

Finally, model-based tracking approaches, which are presented in the next section, resemble a great part of pose estimation techniques. The basic idea is to integrate tracking and pose estimation with the goal to derive optimal solutions in their temporal relation. The optimal solution is usually found by exploiting the search space of possible configurations and deriving their probability based on matching with image cues.

In summary, partial hand pose estimation seems to provide a good starting point for implementing a real-time vision-based 3D hand processing algorithm. The approach has been evaluated successfully in previous systems and stands out with respect to simplicity and efficiency. Ultimately, the limited information will, however, not be sufficient for implementing a diverse and robust set of hand gestures, and advanced techniques will be required.

2.2.3 Hand tracking

After having detected the hands and estimated their pose successfully, the next task is to track the hand appearances through sequences of frames. More pre-

cisely, the goal of tracking is the establishment of temporal relationships between hand appearances while reducing the search space for potential appearance locations and poses. Generally, tracking methods can be divided into appearance-based and model-based approaches, depending on whether an abstract geometric representation of the tracked object is used or not.

Appearance-based

The most basic appearance-based tracking systems implement data association by minimizing the distance between the segmented hand regions of two time-consecutive frames. For example, this idea is applied by Kim et. al. [39] in the context of their motion-based hand detection framework. Further, they use stereo reconstruction of the hand region centers for 3D mapping of the hand position. Similarly, Schlattman et. al. [67] apply the technique to their voxel-based 3D reconstruction of the user's hands, which is calculated from the visual hull of segmented images in a multi-camera environment. The authors additionally achieve a classification into left and right hand based on an estimation of the elbow location.

One problem with purely region-based tracking methods is the lack of detailed appearance information, which causes frequent tracking failures in more challenging environments. To overcome this problem, Barhate et. al. [5] employ the more powerful EigenTracker framework [8], which incorporates shading information for establishing associations between appearances. Therefore, the algorithm estimates affine transformations between segmented regions and uses eigenspace representations for content comparison. Despite the more reliable region association, background clutter and dynamics are posing challenges due to their uncontrollable influence on part of the region contents.

To make appearance-based tracking more robust, another family of systems takes the motion of objects and their appearances into account. For example, Antonis et. al. [2] assume constant linear motion in the image plane between consecutive frames, which is a close approximation at high video capture frame rates. For a successfully tracked hand the motion is derived as the apparent displacement between two frames, for prediction the displacement is added to the previously known position. The resulting system is robust against occlusion and works well even under camera motion. Consequently, the authors develop a 3D hand tracking system [3] that integrates two basic region trackers and a covariance based region matcher. The stereo reconstruction problem along the epipolar line is solved by applying the stable marriage algorithm with preferences expressed as disparities. In contrast, Kim et. al. [38] propose the use of second order auto-regression of the limb motion to predict the hand position in consecutive frames. This technique relies on the assumption that the user is standing in front of the camera, as commonly observed in human-robotics scenarios. Keskin et. al. [37] finally enhance the linear prediction process with a Kalman Filter [35], which accounts for Gaussian noise in the observation process and delivers smoother approximations of the ground truth.

At last, a very different approach is followed by Kolsch et. al. [42], who

employ correlation based tracking of KLT features [52], which are selected from the bounding box of the hand. The selection is based weights that are calculated from an adaptive skin color probability map and a static spatial distribution mask. If features are lost - a frequent event due to the highly diverse articulated motion of the target object - the next best features are added to the set of tracked items. Generally, the set of tracked features has to comply with the flock property, i.e. features have to maintain a minimum distance from each other while not exceeding a maximum distance from the overall crowd center. This way, a stable spatial distribution of features is maintained, resulting in more robust tracking performance under severe clutter. On the downside, high resolution images are required to be able to find enough distinctive features for building the flocks.

In summary, appearance-based approaches deliver a good balance between accuracy and performance, which suggests their application in location independent real-time computer vision applications. Among the individual approaches, linear motion prediction seems to provide the best results.

Model-based

Instead, model-based tracking techniques are based on the idea of fitting an abstract geometric model of the human hand to the potentially noisy image data as observations. This process usually requires an initial pose estimate to be available, which is iteratively refined to minimize a so-called matching error. The matching error is generally defined in terms of a probabilistic model including image cues such as color, edges and silhouettes. For example, Lu et. al. [50] propose the integration of multiple cues for error formulation and demonstrate improved robustness over single cue systems. Their approach relies on the classification of finger segments using edge detection and curvature analysis. From the displacement of these segments in two consecutive frames, 2D forces are derived, which are used to update the previous model configuration. Additionally, the edge forces are complemented with shading-enhanced optical flow forces, to achieve better spatial coverage. Finally, the the combined 2D forces are iteratively transformed into updates on the model configuration parameters.

A theoretical proof for the convergence of multi-cue model-based matching techniques is provided by Chik et. al. [12]. The authors argue that in almost all cases a unique minimum for their color and silhouette based error metric exists, whereas only rare cases allow multiple minima due to ambiguities in rotation and translation of rigid hand segments. Consequently, it is shown that their optimization algorithm converges to this minimum.

To reduce the ambiguity, some systems rely on stereo and volume reconstructions for the purpose of model fitting. For example, Delamarre et. al. [17] use dense stereo maps to calculate 3D forces directly using the Iterative Closest Point algorithm [83]. It should be noted that a modification to the algorithm had to be implemented since stereo reconstruction does not return closed volumes. In contrast, Ueda et. al. [72] reconstruct the entire hand volume directly using an octree representation. Forces are derived from model vertexes which

lie outside the reconstructed volume. The iteration procedure stops when the model is completely contained within the reconstruction.

One problem with the previous approaches is that only a single hypothesis is supported. To overcome this problem, Gump et. al. [25] propose the use of Particle Filters [33], which allow to approximate the entire posterior probability of the model configuration using a weighted set of samples. For weight derivation a configuration matching error is used, which integrates edge and silhouette cues. For prediction, the samples encode estimates of model dynamics, such as the torque of the joints or the translation of the palm, while a stochastic diffusion component accounts for deviation. To reduce the configuration space, the hand model is restricted to 22 DOF by fixing certain joint angle ratios. Finally, the tracking output is given by the weighted mean of the posterior probability. Due to the high complexity of the procedure, the system is implemented on a particle cluster, which allows near real-time performance.

To make the process more efficient, Zhong et. al. [84] propose the use of elite particles, i.e. highly probable states, for estimating the apparent configuration. After choosing the elite particles a mean shift operation is performed to find the local maximum. Since the mean shift operation alters the sample set, new samples are generated for the next iteration using an importance sampling function as described in [34]. For state prediction, the authors derive a velocity vector for the previous hand configuration and apply it to the sampling set using Gaussian noise diffusion. In contrast, Liang et. al. [47] apply Multi-Directional Simplex Search and Simulated Annealing [18] with the goal to reduce the search space and the number of samples. Their method is based on a discretized KD-tree representation of all possible hand configuration, which is built off-line using pose information captured by a data glove. During prediction, multi-directional simplex search on the discretized space is used to find local maxima. Instead, simulated annealing serves to distribute a small set of samples across the search space in early iterations, which converge to the mode of the distribution successively.

Even though several optimizations for particle filtering exist, it is generally difficult to achieve real-time performance while maintaining accurate estimates of the full posterior probability. To avoid this dilemma, El-Sawah et. al. [21] instead draw pose predications from an integrated gesture recognition module. During system operation, genetic programming techniques are used to maintain a set of the best performing configurations. Accordingly, instead of a single posture, multiple posture hypotheses are kept in the loop. Their respective probabilities are derived using silhouette matching, while assuming Poisson distributed background clutter along the edge normals and normal distributed projection accuracy. However, it should be noted that fingertip markers are used to calculate an initial model alignment hypothesis using inverse kinematics.

In summary, model-based approaches deliver excellent results in terms of the accuracy, with which hands and their poses can be tracked across sequences of frames. However, the respective algorithms are very expensive to compute, which requires complex hardware solutions and prohibits lightweight implementations at the same time.

2.2.4 Gesture recognition

The field of gesture recognition is only touched briefly in this overview since the main focus of this thesis is to develop a real-time multi-finger accuracy hand tracking framework with multi-touch capabilities. The approaches presented here are taken from the body of hand tracking publications, which have been presented in the previous section.

In the simplest case, gesture recognition is implemented using Bayesian classification of image features. For example, Wang et. al. [78] use Fourier Descriptors for deriving rotation, translation and scale-invariant shape features of the previously segmented hand regions. Consequently, gestures are modeled as Gaussian distributions over this feature space. Additionally, a loss function is introduced for adding weight to cases of misclassification. In contrast, Liu et. al. [49] use the Maximum Posterior estimate to derive the most probable hand gesture from samples of a particle filter set. Finally, a gesture transition matrix accounts for transition probabilities in the prediction step.

One problem with the previous approaches is that only static gestures can be supported, where no articulation change or motion is involved. To account for spatio-temporal varying gestures, Hidden Markov Models (HMM) have been proposed both for the purpose of gesture modeling and gesture recognition. Recognition is realized by applying a sequence of hand states to each of these models and calculating the resulting probability of the respective gestures. The most probable gesture is chosen as the output of the module. For example, Keskin et. al. [37] derive velocity vectors from the hand appearances for feeding the HMM. Thresholds are used to divide meaningful input from random hand movements. The threshold model is adaptive as proposed in [45] since recognition probabilities strongly depend on the complexity of the gestures. In contrast, El-Sawah et. al. [21] complete posture labels are fed to the HMM. Posture labeling is achieved using Support Vector Machines, a common method for pattern recognition applications. During model training, random noise is added to the pre-segmented samples to improve the robustness of the system.

As mentioned previously, many more techniques exist. The most robust approaches rely on pattern recognition techniques, which integrate statistical frameworks based on training data. The applicability of any method, however, strongly depends on the quality of the output that is obtained from the hand tracking module.

Chapter 3

Preparation

As discussed in the previous chapter, many approaches exist to tackle the problems of hand detection, hand pose estimation, hand tracking and hand gesture recognition. In the context of 3D multi-touch interfaces, most lack important properties such as real-time performance on commodity hardware, marker-less operation, or the desired third dimension. Nevertheless, important aspects of the existing ideas can be reused for designing a system, which operates in the defined scope. In the following, two particular approaches are presented and evaluated for inclusion in the 3D multi-touch prototype.

3.1 Overview

Currently appearance-based techniques are the most likely approaches to withstand the real-time constraint on standard hardware due to their good balance between accuracy and performance. Still, among appearance-based solutions the number of design choices is large, which is why an initial capability assessment among the alternatives was required. In particular, two techniques have been evaluated: A learning-based approach (1), which is described in Section 3.2, and a geometry-based approach (2), which is described in Section 3.3. In the following, first an overview of the respective techniques is presented, before discussing the details of the prototypes. The chapter concludes with a comparison of the results, which leads to the final design choice for the realization of the 3D multi-touch system.

3.1.1 Learning-based

The learning-based approach has been designed and implemented specifically for the webcam Minoru that has been presented in Section 1.2.3. Since an initial evaluation deemed CPU-based dense stereo matching for robust depth segmentation unfeasible on this platform, the ideas of feature extraction and statistical classification as described in Section 2.1.3 have been applied, instead.

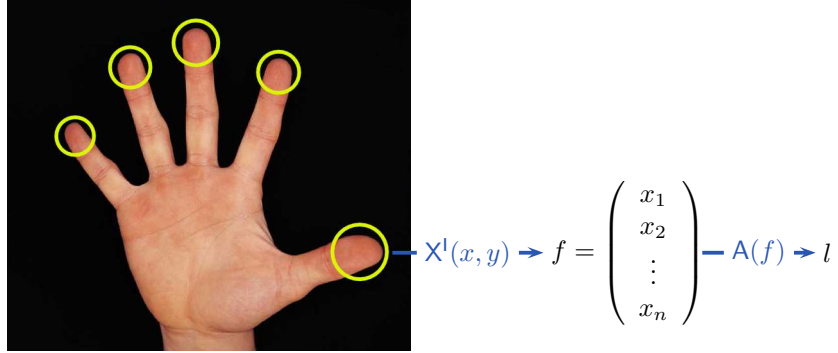


Figure 3.1: Feature classification in the learning-based approach.

The frameworks are adapted for the purpose of developing an classifier, which is able to automatically recognize fingertips, as illustrated in in Figure 3.1. The fingertips are of strongest interest, since they represent the primary reference points for interaction in touch-enabled environments. As a consequence, the feature extractor is required to respond to fingertip locations.

Further, for evaluating the performance of the approach, the system is required to be accompanied by a training tool that enables the annotation of video streams. The purpose of the annotations is to mark those feature points that coincide with actual fingertip locations. In the next step, these features can be used to build a prototype classifier for a live system, which imitates the annotation behavior. It should be noted that during performance assessment it is not of primary importance to implement highly efficient versions of the components. Rather, a qualitative comparison of the approaches should be achieved giving insight into their capabilities in the best case. However, prospects for optimization must be available to ultimately meet the real-time operation requirement.

3.1.2 Geometry-based

In contrast to the learning-based approach, the geometry-based approach applies purely geometric considerations for finding fingertips in depth images as obtained by the TOF camera SwissRanger SR4000 (see Section 1.2.3). As already pointed out in [56, 9, 46], fingertips respectively hands respond well to geometric classification schemes. The basic idea of these schemes is to model the general appearance of hands in images using primitive two-dimensional geometric elements such as circles and rectangles. Usually, respective implementations benefit from the highly efficient geometric constraint evaluation, which is possible due to the simple nature of the underlying model.

An example of a geometric classification scheme is presented in Figure 3.2, where the fingertips and the palm are modeled using circles, and the fingers are modeled using rectangles. This model naturally explains the appearance of a

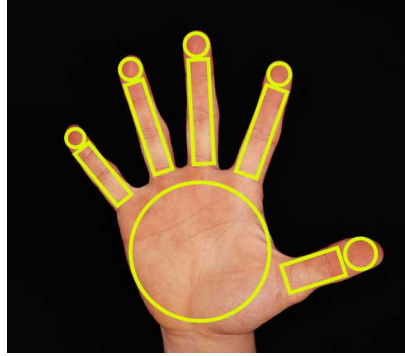


Figure 3.2: Hand model in the geometry-based approach.

fully opened hand, however, more involved hand poses cannot be represented well due to the rigid structure of the fingers. For example, one might think about a grasp gesture with all fingers bended towards the inside of the palm. To account for this type of variability, a modified version of the model has been implemented, which relaxes the rectangular constraint.

Finally, it should be noted that the fully opened hand provides a good starting point for hand detection as evaluated by Kolsch et. al. [41]. Later it will be shown that this idea has been adapted in the context of the geometry-based prototype.

3.2 Learning-based

Having presented an overview of the ideas for vision-based hand processing in the previous section, now the details of the learning-based approach are discussed. First, the concept is reviewed in Section 3.2.1 including the mathematical frameworks and their integration. Second, the architecture of the prototype is explained in Section 3.2.2, before finally discussing the results of the performance evaluation in Section 3.2.3.

3.2.1 Concept

As already pointed out in the overview (Section 3.1), the learning-based approach is based on the ideas of feature extraction and statistical classification, which have been introduced in Section 2.1.3. Accordingly, the recognition process is defined mathematically by means of a feature extractor $X_{candidate}^I$, a training copora $B_{fingertip}$ and an automatic classifier $A_{fingertip}$. These three components form the fundament of the prototype for initial performance assessment.

Obviously, the first task in designing the learning-based system is to decide for a concrete feature extractor, which delivers interest points around fingertip

locations (1) and provides discriminative descriptors for classification (2). After running experiments, the SURF feature detector was selected, which has been presented in Section 2.1.3, i.e.

$$\mathbf{X}_{candidate}^l = \mathbf{X}_{SURF}^l : \mathcal{S} \longrightarrow \mathbb{R}^{64}$$

This choice was mainly guided by three reasons: First, the extractor meets both requirements (1) and (2) sufficiently well, as proofed by the experiments. Second, its implementation is highly efficient and well suited for real-time vision applications. Third, a high quality implementation is freely available in the open source vision toolkit OpenCV.

For recognition the nearest neighbor pattern classification rule, which has been presented in Section 2.1.3, has been employed, instead, i.e.

$$\mathbf{A}_{fingertip} = \mathbf{A}_{nn} : \mathbb{R}^{64} \longrightarrow \{FT, \overline{FT}\}$$

As already mentioned previously, this rule is well suited for assessing the qualitative performance of pattern recognition techniques, which is the main objective at the current stage. Furthermore, the rule is generally used in cases where only a few training samples are available and the true nature of the underlying distribution is unknown. Finally, its implementation is very simple, which makes it a perfect choice for inclusion into prototype applications. It should be noted that actually not a full nearest neighbor classification scheme has been implemented, rather a pure Euclidean distance metric D_{ec} was used for determining the similarity between trained fingertip descriptors $f \in \mathcal{F}$ and scene feature points $f' \in \mathcal{F}$ such that

$$D_{ec}(f, f') = \sqrt{\sum_{i=1}^{64} (f_i - f'_i)^2}$$

This similarity measure serves the purpose of validating the performance of the approach sufficiently well because it also forms the foundation for selecting nearest neighbors in $\mathbf{A}_{nn}$.

Finally, the question arises how the samples for training the classifier are obtained. To build a sufficiently large corpora of annotated training data quickly, an automated annotation procedure is desirable, which requires minimal user input for guiding the feature selection process. The choice fell onto a semi-automatic procedure that allows to select feature points manually in frame $\mathbf{l}^t$ and track their appearance though frame $\mathbf{l}^{t+\delta t}$. Tracking is achieved by means of motion field $\mathbf{M}^{t,t+\delta t}$, which is obtained from an optical flow algorithm as introduced in Section 2.1.4. I.e., assuming position $P^t \in \mathcal{S}$ has been selected as fingertip location in frame $\mathbf{l}^t$, the same feature is expected to appear in frame $\mathbf{l}^{t+\delta t}$ at position $P^{t+\delta t}$ with

$$P^{t+\delta t} = P^t + \mathbf{M}^{t,t+\delta t}(x, y)$$

Using this method, only few corrections are required when the tracking algorithm starts drifting off the correct location. In the other cases, the procedure

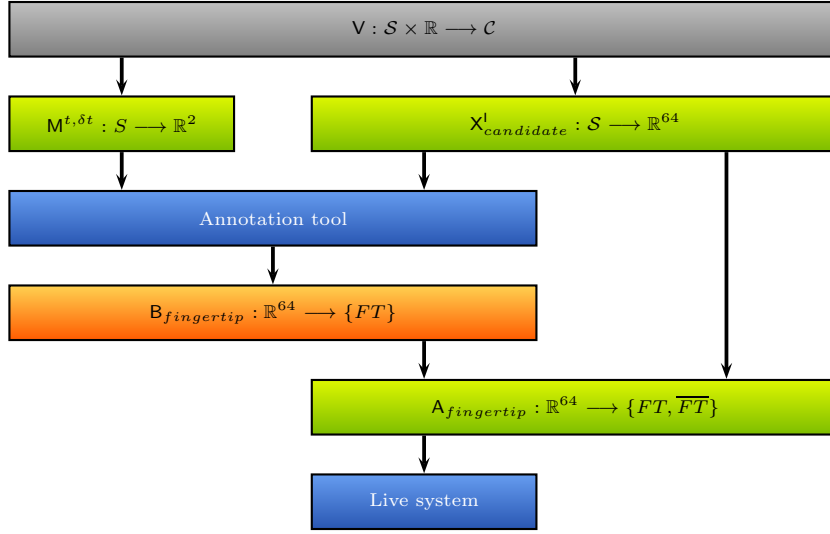


Figure 3.3: Architecture of the learning-based prototype.

of image annotation can be executed quickly by simply approving the tracking results.

3.2.2 Prototype

The focus of the previous section was the review of the general concept and mathematical background of the learning-based approach. In contrast, the current section concentrates on the translation of the general concepts into a concrete software design with the goal to explain the integration of the abstract mathematical formulations into an interlinked set of functional components.

An overview of the software architecture, which was selected for implementing the prototype, is presented in Figure 3.3. As apparent from the diagram, the general structure is layer-based, i.e. information is flowing from top to bottom, but not vice versa. For readability, colors are used to group the components: *Algorithms* are highlighted in green and *user interfaces* are highlighted in blue. The flow of information starts at the video stream V , which is captured from an arbitrary camera C . In the first processing step, this V is fed into both the optical flow algorithm and the feature extractor. The optical flow algorithm is responsible for deriving the motion field $M^{t,\delta t}$, which is used for tracking interest points across frames. Instead, the feature extractor $X_{candidate}^l$ delivers the interest points among which the actual fingertip locations can be found. Both results are streamed into the *annotation tool*, which enables marking and tracking of fingertip features across the entire video stream. The tool is also responsible for maintaining the training database $B_{fingertip}$, which holds all descriptors selected in the past. From this database it is possible to automatically derive the fin-

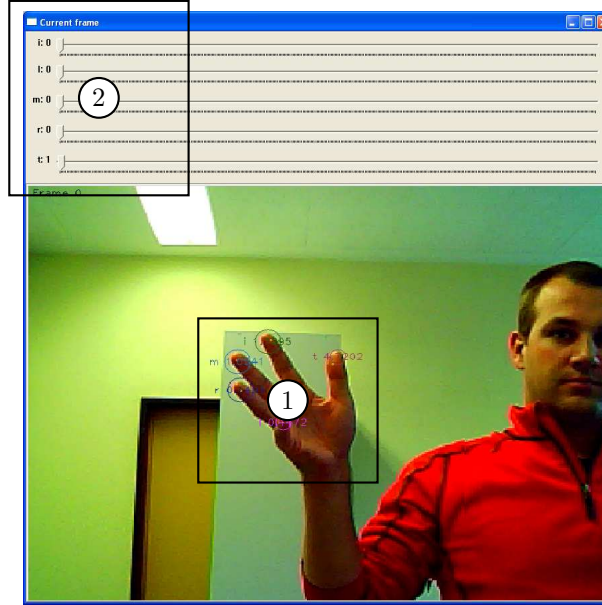


Figure 3.4: User interface of the learning-based annotation tool.

gertip classifier $A_{fingertip}$, which can be applied to candidate features extracted from any image respectively video stream to recognize fingertips. This classifier is finally integrated into a *live system*, which visualizes the probabilities of features to represent fingertips.

The remainder of this section is explaining the user interface of the *annotation tool*, which is central tool of the entire application. As can be seen in Figure 3.4, the interface is divided into two parts: (1) The frame of the video stream, which is processed currently, plus the active annotations and (2) the controls for providing user input. To understand the application work flow, zoomed versions of the highlighted regions are provided as depicted in Figure 3.5a. Figure 3.5a shows the annotations that mark features corresponding to fingertips with circles of varying color. Further, the circles are accompanied by a letter $c \in \{t, i, m, r, l\}$ that indicates the identity of the finger as derived from the first character of the finger name (e.g. t for *thumb*). Finally, for each circle a number is provided, which displays the distance of the circle center to the small, yellow points. The yellow points either represent the location where the user clicked for manually placing the annotations, or where the tracking algorithm expects the features to be after applying the optical flow algorithm. Obviously, these positions might not coincide with actual SURF feature locations directly, which is why the distance measure is used to rank all features with respect to the yellow point. Then, initially, for each frame the closest feature is chosen for creating the annotation.

In some cases, however, the closest feature is not the best match. This,

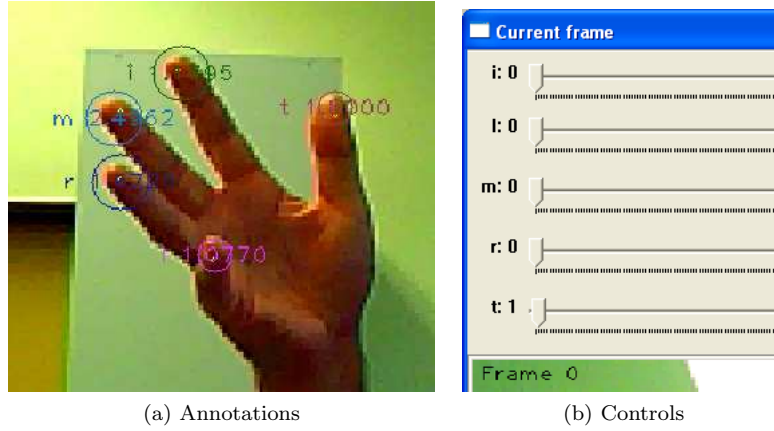


Figure 3.5: Interface components of the learning-based annotation tool.

for example, happens when the tracking algorithm starts drifting of the true location. Therefore, the controls are provided which are depicted in Figure 3.5b. The controls allow the user to select the correct feature from the full set of interest points. As mentioned previously, for speeding up the selection process, the features are sorted according to their distance to the expected location (yellow point). Convenient access to this sorted data structure is provided by means of the sliders whose ticks represent the individual candidates. For example, Figure 3.5b illustrates a situation where the second item from the sorted list is selected to represent the thumb location (value 1). For all other fingers the correct feature location is already given by the first item in the list (value 0).

Finally, a mechanism is provided to step through the video sequence V for annotating the entire stream: A simple keyboard control scheme, which can be handled efficiently parallel to the mouse interaction, has been implemented. The three available keyboard commands and their effects are listed in Table 3.1.

Character	Command	Description
<i>o</i>	Okay	Do not train the features but continue tracking
<i>t</i>	Train	Train the features and continue tracking
<i>s</i>	Skip	Discard the features and reinitialize

Table 3.1: Keyboard commands of the learning-based annotation tool.

3.2.3 Performance

In the following, for assessing the performance of the learning-based prototype two aspects are evaluated: First, the effectiveness and efficiency of the semi-automatic annotation procedure is investigated to help building a sufficiently

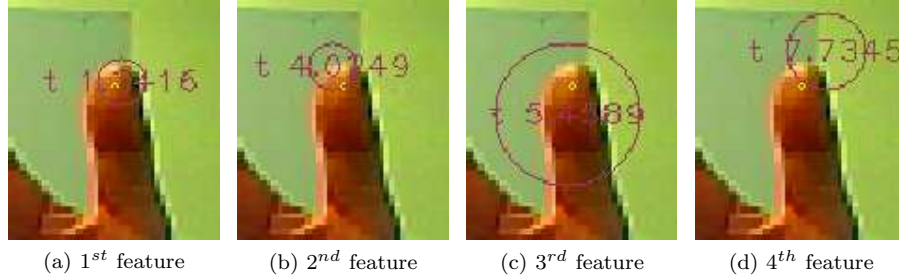


Figure 3.6: First four features around the expected fingertip location.

large corpora of samples for the training phase. Second, the discriminative power of the SURF feature descriptor for the purpose of fingertip classification is examined, since it is not sufficient that the detector reacts to fingertip locations, but also the extracted feature vectors have to be unique enough for recognition.

To evaluate the performance of the annotation procedure, the localization of the SURF feature points around actual fingertip locations has to be investigated. For this reason, the four closest features around a fingertip, which is taken from a random frame of the training corpora, are shown in Figure 3.6. The illustration highlights that multiple valid matches might be extracted from the image, namely the first and the third in the example. The first feature, however, provides the best localization with minimal dependence on the background appearance. Generally, a similar behavior of the feature detector was discovered in most other cases.

Another aspect of the performance of the annotation tool is the effectiveness of the tracking procedure. For illustration, three samples of an autonomous tracking sequence are provided in Figure 3.7 (for the complete sequence see Appendix C). The sequence suggests that the component delivers good results under small hand pose variations, which occur in the first eight frames. In the later frames the little finger starts moving too quickly, which is why the annotation is continuously misplaced. Still, the expected location (yellow point)

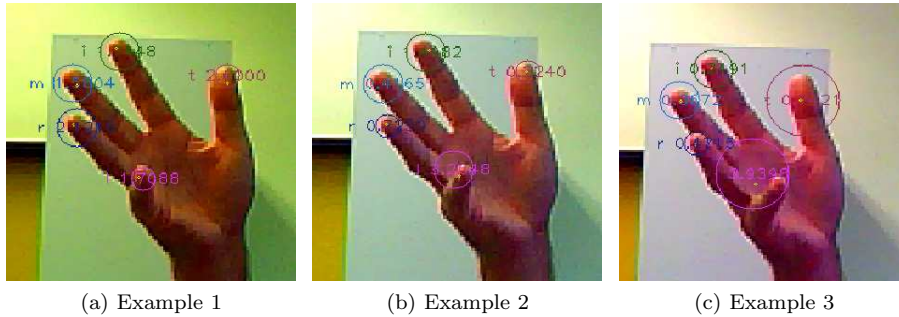


Figure 3.7: Tracking results of the optical flow algorithm.

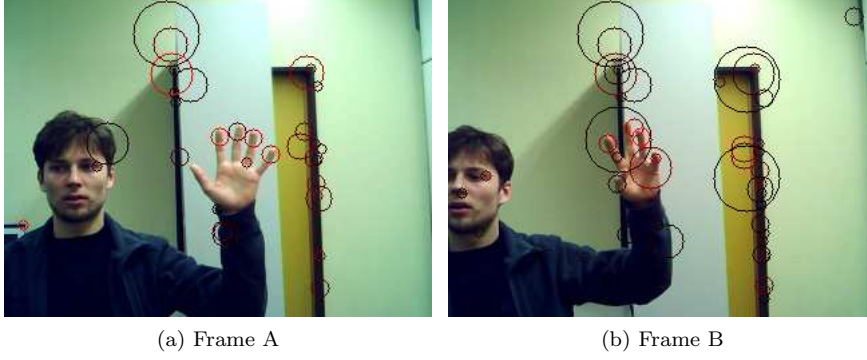


Figure 3.8: Live results of the automatic classifier.

is close to the actual position of the fingertip, which is why the sliders (see Section 3.2.2) can be used efficiently.

To conclude the performance discussion for the learning-based prototype live results of the classifier are presented, which have been taken months after training the system. Two frames of the video stream can be seen in Figure 3.8, each showing a different hand pose given by the same user. Circles indicate those features $f \in \mathcal{F}$ whose distance is close to some training sample, i.e.

$$\exists f' \in \mathcal{F} : B_{fingertip}(f') \text{ is defined} \wedge D_{ec}(f, f') \leq \epsilon$$

The saturation $s_f \in \mathbb{R}^+$ of the color indicates the minimum distance of f to training samples f' as compared to the overall minimum distance of any image features f_{min} to training samples f' , i.e.

$$s_f \propto \frac{\min_{f'} D_{ec}(f_{min}, f')}{\min_{f'} D_{ec}(f, f')}$$

Obviously, in both images at least one of the fingers is marked with a strong red color meaning that the corresponding feature f is close to the training set relative to the other image features. Still, some fingertips are only matched badly, which indicates that the training corpora is not sufficient. Also, there is a considerable amount of outliers like the red circle in the bottom-left region of Figure 3.8a.

Finally, these observations should be set in context with the size and quality of the training database. For this purpose, the number of samples which

Finger	No. samples	Finger	No. samples
Thumb t	2289	Index i	2283
Middle m	2292	Ring r	2290
Little l	2274		

Table 3.2: Statistics of the training corpora.

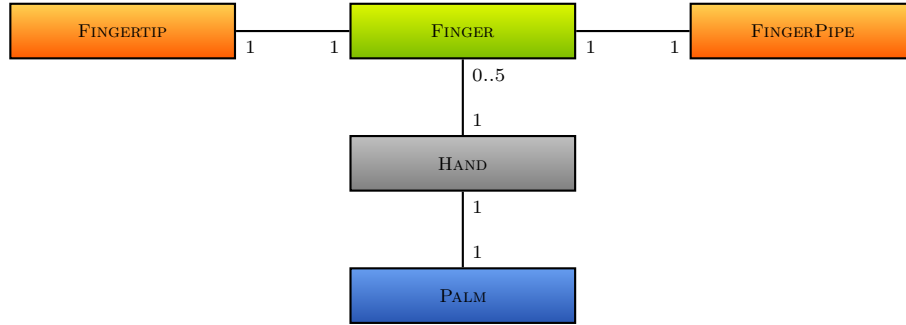


Figure 3.9: Components of the geometry-based appearance model.

have been annotated for each finger type are listed in Table 3.2. The database includes samples from three different subjects, two male and one female, which have been recorded with relatively stable lighting conditions. The process of annotation was only a matter of a few hours due to the efficiency of the annotation procedure. However, it should be noted that some mistakes are included in the database that do not refer to actual fingertip features. The reason is that invalid descriptors have been trained accidentally and the prototype does not provide an undo function at the current stage.

3.3 Geometry-based

The presentation continues with the discussion of the geometry-based approach. On a very profound level, the geometry-based approach is similar to the learning-based approach, which has been presented in the previous section. Their resemblance originates from the fact that both rely on some sort of classification for fingertip recognition. However, in the geometry-based design no training set is involved, instead geometrical constraints on the image contents are derived manually. In the following, first, the concept is reviewed in Section 3.3.1, before highlighting implementation details in Section 3.3.2, and evaluating the performance in Section 3.3.3.

3.3.1 Concept

As already pointed out in the overview (Section 3.1), the geometry-based approach uses a geometric model of the average hand appearance in two-dimensional images. One such model has already been presented in Figure 3.2 using circles and rectangles to represent fingertips, fingers and the palm. For the prototype, a slight variation of this model has been implemented, which is adapted to the characteristics of depth images as obtained from stereo or depth cameras. A translation to regular color images might be possible, but its investigation is out of the scope of this thesis.

The general structure of the geometric model is depicted in Figure 3.9, showing the components and their connections as well as constraints on valid configurations. Obviously, the HAND entity constitutes the central element, which is linked to exactly one PALM and a maximum of five FINGER objects. The PALM instance is the only necessary requirement for a hand to be found in the frame of a video sequence. In contrast, the number of fingers is allowed to vary arbitrarily within the natural limits (zero till five). The fingers themselves are finally decomposed into exactly one FINGERTIP and one FINGERPIPE object. The pipe subsumes the visual appearance of the fingers' bones as a single entity.

Having defined the components of the abstract model, the next step is to specify their mapping onto geometric primitives in the two-dimensional image plane $\mathcal{I}$. As in the original model, the appearance of the PALM entity is defined by means of the maximum inscribed circle of the thenar, which has already shown good performance in other systems [1, 73, 40, 55]. To get to a mathematical formulation of the palm selection procedure a mapping of image location $P = (x, y) \in \mathcal{S}$ to the circle C_{max}^P is required with

$$C_{max}^P = (P, r_{max}^P), r_{max}^P \in \mathbb{R}_0^+$$

where P is used as the center point and r_{max}^P denotes the maximum radius under which the circle still covers an image area of consistent depth. In this context, the consistency constraint is formulated as

$$\begin{aligned} C = (P, r) \text{ is consistent} \\ \iff \\ \forall \alpha \in [0, 2\pi], r' = r - \delta r \in \mathbb{R}_0^+ : \\ |l(x + r \cos(\alpha), y + r \sin(\alpha)) - l(x + r' \cos(\alpha), y + r' \sin(\alpha))| \leq \epsilon \wedge \\ \wedge C(P, r') \text{ is consistent} \end{aligned}$$

with

$$C = (P, 0) \text{ is always consistent}$$

marking a depth segmentation criterion as the pixels of image $\mathcal{I}$ store distance values with respect to the camera. Having the basic mathematical framework, palm candidates are defined as image locations P , which satisfy the following two constraints: First, location P is required to represent a local maximum regarding the maximum inscribed circle mapping, i.e.

$$\forall Q = (x', y') : D_{ec}(P, Q) \leq \delta \Rightarrow r_{max}^P \geq r_{max}^Q$$

Second, the range of valid palm radii is limited by means of a minimum and maximum threshold t_{min} and t_{max} such that

$$t_{min} \leq r_{max}^P \leq t_{max}$$

The valid palm radius range naturally translates to a minimum respectively maximum distance of the user to the camera due to the perspective foreshortening effect. The final palm model is illustrated in Figure 3.10 where the user presents two hands at different distances to the camera.

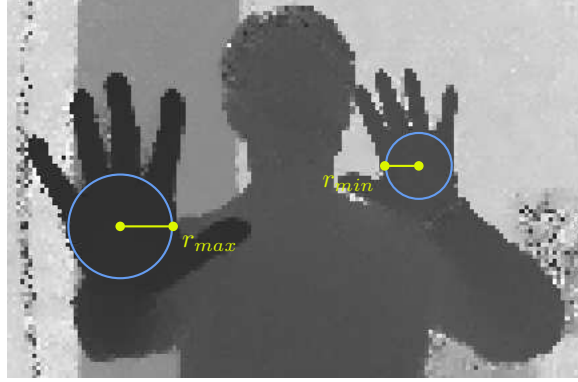


Figure 3.10: Geometric modeling of the palm.

Instead, the geometric representation of FINGERTIP components relies on fixed-size circles, which are used to classify image locations $P = (x, y) \in \mathcal{S}$ according to their circular neighborhood as introduced by Rosten et. al. [64]. The circles are defined as C_{fix}^P with

$$C_{fix}^P = (P, r_{fix}), r_{fix} \in \mathbb{R}^+$$

where, again, P is used as the location of the center point and r_{fix} denotes a constant radius that is the same for every P . Based on this circle a comparison function C_P is defined, which serves the purpose to label each location on the circle according to its depth relative to the center pixel P , i.e.

$$C_P : [0, 2\pi] \longrightarrow \{-1, 0, 1\}$$

such that

$$C_P(\gamma) = \begin{cases} -1 & \text{if } -\epsilon > l(x + \cos(\gamma)r_{fix}, y + \sin(\gamma)r_{fix}) - l(x, y) \\ 0 & \text{if } -\epsilon \leq l(x, y) - l(x + \cos(\gamma)r_{fix}, y + \sin(\gamma)r_{fix}) \leq \epsilon \\ 1 & \text{if } -\epsilon > l(x, y) - l(x + \cos(\gamma)r_{fix}, y + \sin(\gamma)r_{fix}) \end{cases}$$

Obviously, the label 0 represents those locations, which exhibit strong similarity in their depth value compared to the center pixel, -1 and 1 are used if strong deviations are encountered, instead. Based on this comparison function the neighborhood around location P is classified into segments S_n^P with

$$S_n^P = (\alpha_n^P, \beta_n^P, c_n^P), [\alpha_n^P, \beta_n^P] \subseteq [0, 2\pi], n \in \mathbb{N}$$

such that

$$\begin{aligned} \forall \gamma \in [\alpha_n^P, \beta_n^P] : C_P(\gamma) = c_n^P \wedge \\ \wedge \nexists [\psi, \omega] \subseteq [0, 2\pi] : [\alpha_n^P, \beta_n^P] \subset [\psi, \omega] \wedge \forall \gamma \in [\psi, \omega] : C_P(\gamma) = c_n^P \end{aligned}$$

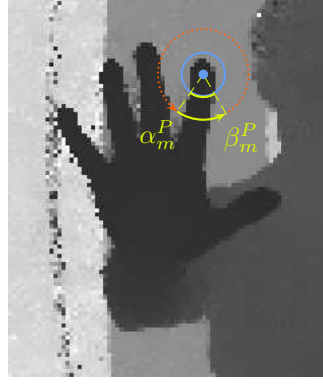


Figure 3.11: Geometric modeling of fingertips.

The set of segments for image location P allows to determine the the location's affinity to common fingertip appearances as illustrated in Figure 3.11. The affinity is expressed as the constraint that only one segment with equal depth is allowed to exist and the segments angular range does not exceed a certain limit, i.e.

$$\exists m \in \mathbb{N} : c_m^P = 0 \wedge |\alpha_m^P - \beta_m^P| \leq \theta_{tip} \wedge (\nexists l \in \mathbb{N} \setminus \{m\} : f_l^P = 0)$$

The angular limit parameter θ_{tip} is used to control the smoothness of the tip structures found by this constraint system, i.e. low values react to strongly exposed and thin tips, while high values respond to flat and thick tips, as well.

Conveniently, for defining the visual representation of the FINGERPIPE components the circular neighborhood classification scheme can be reused. The only difference is that for pipe-like locations $P = (x, y) \in \mathcal{S}$ a different constraint is placed on the set of segments S_n^P : Instead of one segment with equal depth, this time two segments are required as illustrated in Figure 3.12. This altered condition leads to the boolean expression

$$\begin{aligned} \exists l, m \in \mathbb{N} : c_m^P = c_l^P = 0 \wedge \\ \wedge |\alpha_m^P - \beta_m^P| + |\alpha_l^P - \beta_l^P| \leq \theta_{pipe} \wedge (\nexists k \in \mathbb{N} \setminus \{l, m\} : c_k^P = 0) \end{aligned}$$

for selecting the appropriate candidates. Analogously to the previous case, the parameter θ_{pipe} is used to control the thickness of accepted pipe structures where low values indicate thin pipes and high values thick pipes instead.

Finally, two things should be noted about the circular neighborhood constraints for tip and pipe selection: First, the fixed radius r_{fix} requires the user to have a minimum distance to the camera, otherwise the segmentation conditions fail since fingers cannot be covered fully by the circles anymore. A second problem with the segmentation conditions is that fingers are required to have a minimum distance from each other, otherwise neighboring fingers cause

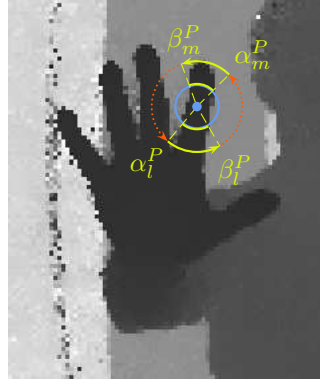


Figure 3.12: Geometric modeling of finger “pipes”.

new segments with equal depth uncontrollably. Overcoming these limitations would require a scale-independent definition, which is based on the entire circle contents rather than the circumcircle only. This task is left to future research.

3.3.2 Prototype

The geometry-based prototype implements the concepts and techniques from the previous section with the purpose to illustrate the power and flexibility of the presented approach. It should be noted that this task requires more effort than the implementation of the learning-based approach presented in Section 3.2. The increased effort is owed to the fact that the learning-based technique strongly relies on existing algorithms and their publicly available implementations. Instead, for the geometry-based technique many components have to be implemented from scratch because the mathematical concepts are not contained in standard vision toolkits.

The structural decomposition and information flow of the system is illustrated in Figure 3.13. Again, the video stream V forms the fundament delivering a time-sequenced stream of images to the vision components. The vision components are divided into three classes: The PREPARATOR, the CLASSIFIER and the TRACKER. The first component is responsible for preparing the images for further processing by extracting rudimentary features from the raw image data. Then, a classification step takes care of grouping these features into finger and hand candidates. As some candidates might originate from noise artifacts, concrete hand instances are not derived directly. Instead, a last tracking component is included that takes temporal information into account for determining stable hand appearances. The hand instances are finally delivered to a workbench application for further processing such as tracing, debugging or profiling.

The user interface of the workbench provides access to three main tabs: The *capture* tab as shown in Figure E.1, the *inspect* tab as shown in Figure E.2 and the *live* tab as shown in Figure E.3. The capture tab offers a video capture com-

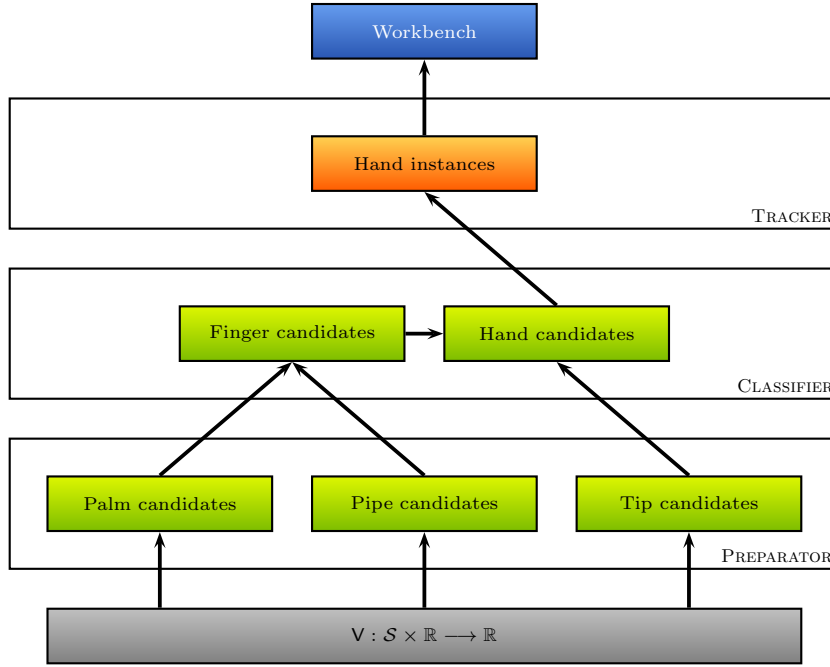


Figure 3.13: Architecture of the geometry-based prototype.

ponent, which allows to record streams of depth images from the SwissRanger SR4000. The inspect tab allows the user to browse the previously captured streams and review the algorithm output for every frame of the sequence. The inspection process is facilitated by a nested set of tabs (see center of Figure E.2), which provide access to intermediate results of the vision components. Additionally, memory statistics are presented at the right border (same Figure) to track memory performance and detect eventual memory leaks. Finally, the same set of widgets - tabbed inspection of intermediate results and memory statistics - is also provided in the live tab to allow real-time performance studies on live data.

3.3.3 Performance

The performance of the geometry-based method is studied by examining the effects of applying the geometric appearance model from the previous section on live data. For this purpose, selected samples from video streams are presented for each component of the model (PALM, FINGERPIPE and FINGERTIP), which are annotated with the results of the respective classifiers. From this presentation the flexibility of the approach becomes apparent, which allows a comparative comparison to the learning-based method presented in Section 3.2.

The annotated set of samples for the PALM classifier is provided in Figure

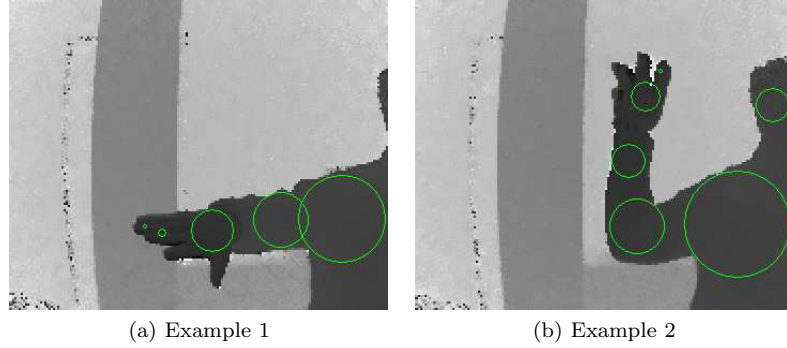


Figure 3.14: Performance of the geometric palm model on live data.

3.14. In the illustration green circles are used to mark locations in the image that correspond to local inscribed circle maxima of regions with continuous depth. From the frames it can be derived that the simple palm model performs well in many cases, especially when a large portion of the thenar is facing the camera. The approach becomes problematic when the palm is held approximately orthogonal to the image plane as, for example, shown in Figure 3.14b. In this and similar cases the image area covered by the palm is too small to reveal a distinctive maximum for the classifier, which makes it impossible to determine an optimal location using the given approach.

The annotated set of samples for the FINGERPIPE classifier is provided in Figure 3.15. In each frame individual image pixels are marked with green color if they match the pipe condition. The pipe condition, as outlined in the previous section, states that the circular neighborhood has to be segmented into exactly two arcs of equal depth as compared to the center pixel. Clearly, this condition works well if the fingers are separated distinctively from each other, as for example the little finger in Figure 3.15a. On the other hand, the approach faces problems when the fingers stand closely together, which causes the

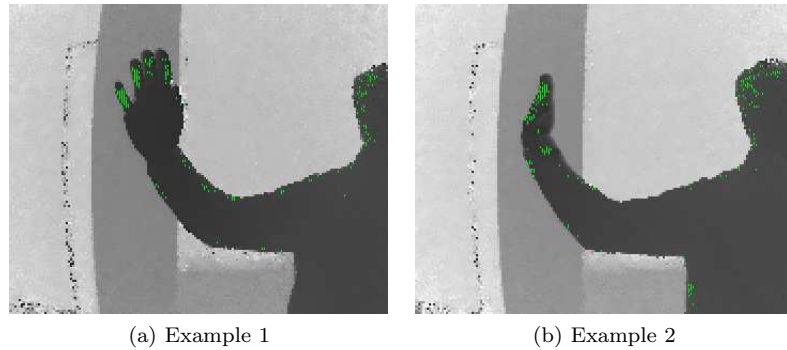


Figure 3.15: Performance of the geometric pipe model on live data.

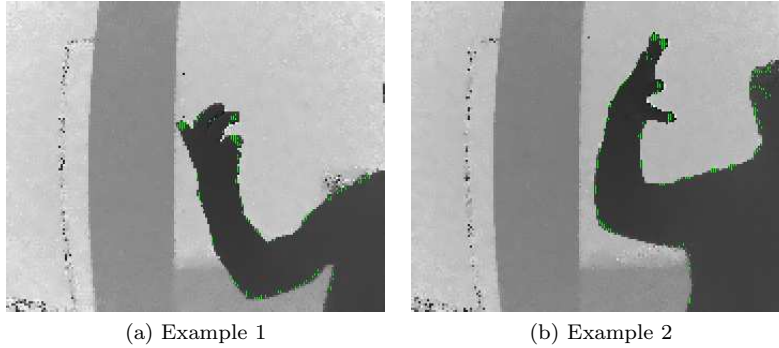


Figure 3.16: Performance of the geometric tip model on live data.

arc segmentation condition to fail due to the overlap with neighboring regions. Another interesting case are situations where most pipes are not clearly visible as in Figure 3.15b. Fortunately, strong pipe-like structures are not detected, but the classifier reacts only sparsely to noise.

Finally, the annotated set of samples for the FINGERTIP classifier is provided in Figure 3.16. Similarly to the pipe case, pixels are marked with green color if they match the tip constraint introduced in the previous section. As a short reminder, the constraint prescribes only one arc segment with equal depth in the circular neighborhood. The samples show generally a good performance of the classifier mainly due to the visually exposed nature of the fingertips. However, analogously to the pipe classifier problems are faced when fingers are overlapping as depicted in Figure 3.16a. This can be attributed to the fact that the maximum angle condition θ_{tip} fails in most parts of the tip region.

In summary, the geometry-based approach showed good classification performance for all components of the abstract hand appearance model in a majority of cases. Still, there are situations where the classifiers fail partly or entirely due to the complexity of the articulation freedom of the human hand and the vast variety of visual appearances which result from this freedom. Obviously, these problems can only be solved with more complex representations and geometric constraints which are difficult to develop.

3.4 Conclusion

Both approaches - the learning-based (see Section 3.2) and the geometry-based (see Section 3.3) - have shown promising results during performance assessment. The strengths of the first lie in its flexibility to capture the visual appearance of arbitrary hand poses under varying lighting conditions, i.e. the qualitative performance of the final classifier becomes a pure matter of the volume of the training data. However, the question remains if the SURF features represent a truly distinctive descriptor for fingertip patches in the image. The samples presented in Section 3.2.3 indicate that there is some potential for confusion,

which could not be eliminated with the given imperfect training set. Another problem is posed by the appearance of the background that cannot be controlled in changed environments since the background around the fingertips is included into the feature descriptors, which ties the corresponding training samples to very specific situations only. Therefore, an improved training procedure might be necessary where hand appearances are segmented automatically to include random background images. The variable background could in turn produce more robust feature spaces in which the dependency on the background is reduced.

Instead, the geometry-based approach works independent of the objects visible in the background of the hand. The only requirement is that the distance between background and foreground objects does not fall below a certain threshold ϵ . The minimum threshold is a reasonable assumption as in most scenarios the user needs a considerable amount of freedom for motion in the virtual or augmented space. Furthermore, the geometric modeling of fingertip and pipe appearances showed fewer confusion potential as compared to the trained SURF classifier. Instead, the danger of false negatives is increased, i.e. image locations that would be classified as tip or pipe by a human but not by the algorithm. A similar behavior was observed for the palm features, where orthogonal rotations towards the image plane caused pure localization of palm centers. Nevertheless, the geometry-based approach seems more promising due to the small number of false positives and a limited number of false negatives. These are both necessary requirements (see Section 1.2.2) regarding the demo application, which is needed for the user study. Also, the runtime performance is excellent because of the simple geometric constraint checking scheme, which is difficult to exceed by more complex feature extraction and classification techniques. These are the main reasons why the geometry-based approach was selected for implementing the multi-touch prototype.

Chapter 4

Realization

This fourth chapter presents the algorithms used for implementing the 3D multi-touch prototype. The discussion starts with an overview of the entire processing chain in Section 4.1. In the following, the presentation continues with detailed information on the preparatory steps in Section 4.2, the feature classification schemes in Section 4.3, the object tracking techniques in Section 4.4 and the gesture recognition in Section 4.5. Finally, the architectural embedding of the functionality into a complete software package is explained in Section 4.6.

4.1 Overview

An overview of the entire processing chain is provided in Figure 4.1. The chain is divided into four major steps: First the raw image data is prepared for the remaining vision tasks. Second, various types of features are extracted and classified from the prepared input. And third, a tracking component takes care of deriving the true hand appearances over time. On top of the tracking output a fourth gesture recognition module is finally responsible for interpreting user intention.

The main purpose of the preparation step (see Section 4.2) is to precalculate required data structures for the subsequent feature extraction and classification algorithms. In a first pass, the raw input image is filtered due suppress noise originating from the camera measurements. From the smoothed distance data coherent objects are segmented in a second pass with the goal to reduce the complexity of subsequent algorithms by confining to object pixels, only. The remaining preparation steps derive the necessary information for applying the geometric appearance model, which has been presented in Section 3.3.1. First, a distance maxima transform is calculated, which facilitates the extraction of inscribed circle maxima with respect to depth discontinuity boundaries. Second, the circular neighborhood segmentation scheme from Section 3.3.1 is applied to calculate feature maps for tip and pipe extraction. And finally, pipes are segmented from the pipe feature map to facilitate the extraction of fingers.

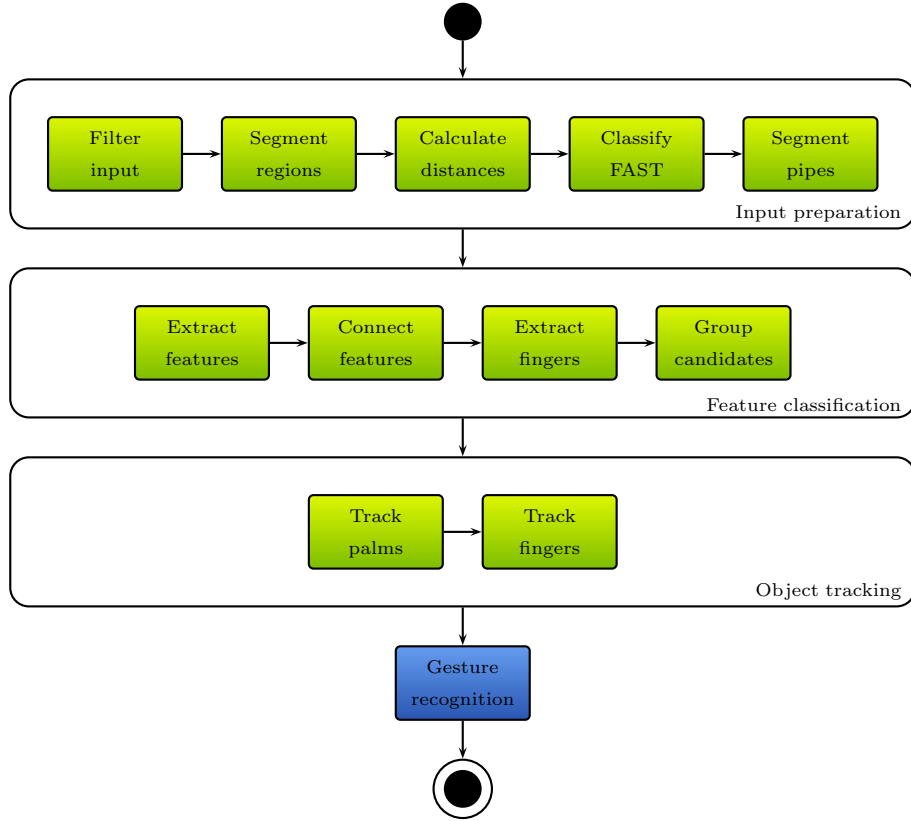


Figure 4.1: Overview of the processing chain.

After preparing the raw image data, feature extraction and classification algorithms (see Section 4.3) are applied with the goal to reduce the visual complexity to a small set of conceptual appearances. In a first step, local turning points are extracted from the distance transform and the tip map. In the next step, a graph is built from the distance features to determine the palm candidates as endpoints of the network topology. In parallel, the tip features are combined with the pipe segments to form finger candidates. Finally, the palm and finger candidates are grouped to potential hand candidates.

After extracting a set of potential hand candidates from the raw feature information, a tracking component (see Section 4.4) takes care of determining the true hand appearances and deriving their spatio-temporal dynamics. The tracking process is divided into two stages: First, a palm tracker is employed to establish the correspondence between previous hand instances and current candidates. Second, a fingertip tracker is used to derive the temporal relation between the individual fingertip locations.

Finally, the hand information obtained by the tracker is fed into a basic ges-

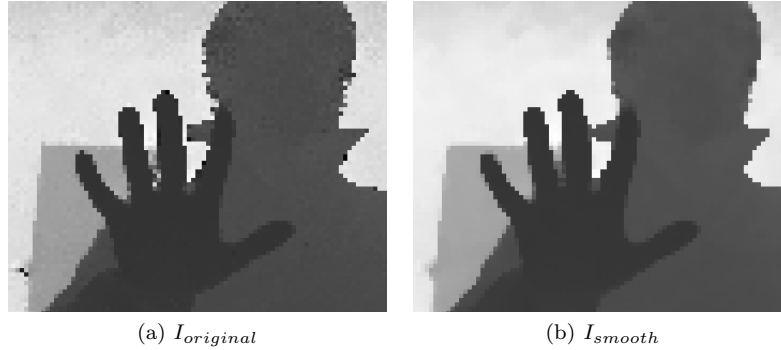


Figure 4.2: Result of applying the media filter to the raw input image.

ture recognition component (see Section 4.5) to derive meaningful user input for controlling a demo application. The set of gestures has been chosen to facilitate well-known interaction metaphors with images such as translation, scaling and rotation [58]. Each gesture is defined in terms of statistical constraints on the respective hand appearances.

4.2 Input preparation

In the following, the individual preparation steps are discussed in detail, ranging from input filtering (Section 4.2.1) and object segmentation (Section 4.2.2), over to distance transformation (Section 4.2.3), to circular neighborhood classification (Section 4.2.4), and pipe segmentation (Section 4.2.5).

4.2.1 Input filtering

As mentioned previously, in a first pass a signal filter is applied to the raw depth image $I_{original}$ to suppress measurement noise, which originates from the sensing process of the TOF camera [44]. Several alternatives have been tested, finally leading to the decision to employ non-linear median smoothing, i.e.

$$I_{smooth}(x, y) = \text{Median of } \{I_{original}(x', y') : (x - x')^2 + (y - y')^2 \leq \sqrt{2}\}$$

One of the main reasons for this decision was that the filter is edge-preserving in contrast to other smoothing techniques such as the Gaussian blur. The minimal 3×3 has been chosen to avoid blending depth values between fingers, i.e. larger window sizes cause the fingers to blur away. For evaluating the median condition, the well-known Quicksort algorithm [30] is used. However, it should be noted that more efficient constant time alternatives exist [59]. Fortunately, due to the low resolution of the camera this choice did not entail serious performance issues.

Finally, the effect of filtering procedure is shown in Figure 4.2. On the left the original image is depicted containing noise, which is best visible around the

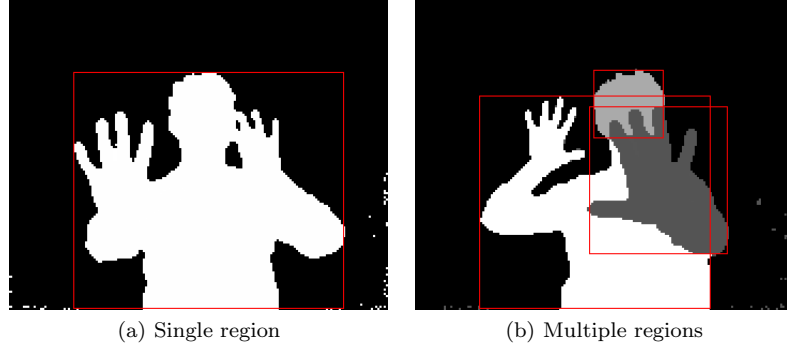


Figure 4.3: Result of the region segmentation algorithm.

head and in the background. Also, the hand appearance is affected, however, the corresponding artifacts are not well recognizable due to linear downscaling of the original 16 bit depth data into the 8 bit gray value channel of the image. The right image shows the results of applying the median filter. As can be seen, the depth data is much smoother at all image locations while all important image features such as the fingertips or the gaps between the individual fingers are maintained.

4.2.2 Region segmentation

Till now, vision algorithms have to be executed on the entire image including each and every pixel. To reduce this processing load, in the next step the filtered image I_{smooth} is segmented into a set of coherent regions $\mathcal{R} \in \mathcal{R}_{region} \subseteq \mathcal{P}(\mathcal{S})$ where $\mathcal{P}(\cdot)$ denotes the power set of $\mathcal{S}$. Many complex segmentation algorithms exist, but fortunately a good quality segmentation can already be achieved by means of the FLOODFILL algorithm, which has been presented in Section 2.1.4. As discussed previously, the procedure determines, given a seed pixel $P = (x, y)$, all image pixels that can be reached transitively. After experimentation, a constant deviation threshold $d = d_{lower} = d_{upper} = 1000$ was selected, which provides a good balance between grouping related pixels and avoiding unnecessary merges between unrelated regions. In contrast, lower values cause the algorithm to cut off important border pixels too frequently, whereas higher values cause frequent merging of overlying objects. After executing the process for all image pixels, each extracted mask M is translated into a region $\mathcal{R}$ with

$$\mathcal{R} = \{(x, y) : M(x, y) = 1\}$$

Obviously, the object is covering exactly those pixels visited by the respective FLOODFILL instance and thus represent a coherent image region.

The effects of the segmentation process are illustrated in Figure 4.3. Here, two independent samples of the same video stream are provided. In each frame red boundary boxes are used to visualize the geometric region properties width,

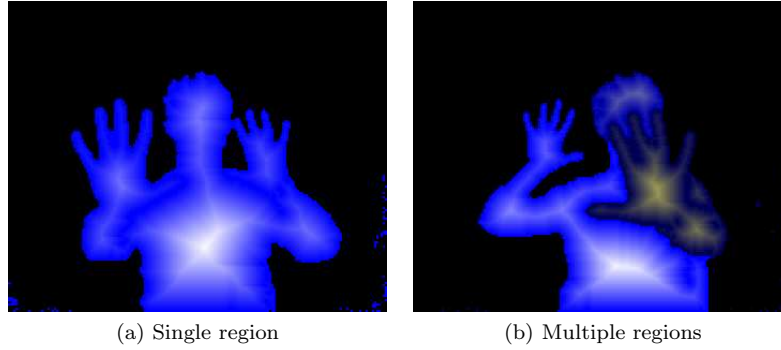


Figure 4.4: Results of the distance transform procedure.

height and position. The actual masks are highlighted in different shades of gray instead. As can be seen in Figure 4.3b, in some cases the algorithm tends to split coherent objects into separate image regions, which is still a valuable result for speeding up the remaining processes.

4.2.3 Distance calculation

Having segmented the prevalent image regions from the scene, the next step is to determine a distance transform $l_{distances}$ as introduced in Section 2.1.4. As previously mentioned, the distance information serves as the basis for selecting palm candidates at later stages of the processing chain. The actual version of the DISTANCETRANSFORM algorithm employed here uses the 8-connected scheme for determining connectivity, i.e. all pixels in a 3×3 neighborhood around the center pixel are considered. However, no direct obstacle map O is used. Instead, a connection between pixel P and pixel P' is derived if

$$D_{ec}(P, P') \leq \sqrt{2} \wedge |l_{smooth}(P) - l_{smooth}(P')| \leq d$$

where $d = 1000$ represents the same deviation threshold as before. This modified version allows to calculate distances not only to region borders, but also to depth discontinuities within the region silhouette. As a result, palms can be detected even if region segmentation delivers only a single entity, which is common if the user holds his hand directly in front of the body.

The effects of the distance transform procedure are illustrated in Figure 4.4. Two independent samples of the same video stream are shown: Figure 4.4a demonstrates a case where the user is segmented as a single entity. In contrast, Figure 4.4b presents a situation where several subregions are created. In both images, the blue color channel is used to indicate the respective region $\mathcal{R} \in \mathcal{R}_{region}$, the red and green channels are used for the distance data instead. As can be seen, the distance transform exposes palm locations equally well in both cases, even if the palms are overlapping the body of the user.

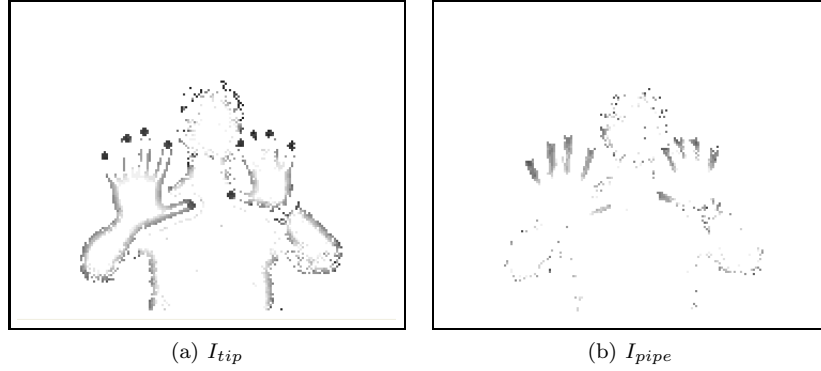


Figure 4.5: Result of the circular neighborhood classification procedure.

4.2.4 Fast classification

The so-called *fast* classification step is probably the most important part of the preparation process. Its goal is to calculate for each pixel $P = (x, y) \in \mathcal{R}$ of each remaining region $\mathcal{R} \in \mathcal{R}_{region}$ the segmentation of the circular neighborhood at distance r_{fix} into the sectors $S_n^P = (\alpha_n^P, \beta_n^P, c_n^P)$ as introduced in Section 3.3.1. The name of the procedure lends itself to the related FAST keypoint detector [64], and is an indication of the high efficiency of the respective algorithm. Through experiments, a radius of $r_{fix} = 4$ was selected, which provides a good balance between coverage of fingertip appearances at various distances while avoiding unnecessary overlap with neighboring regions. In contrast, lower values are susceptible to large hand appearance, while higher values tend to intersect adjacent fingers. Finally, similar to region segmentation (Section 4.2.2) and distance transformation (Section 4.2.3) a threshold of $\epsilon = 1000$ is used for comparison of the center pixel with the circumcircle, which is the obvious choice for this purpose.

To give rise to the subsequent mathematical specifications, in the next step, for each pixel P the set $\mathcal{A}^P$ of arcs with equal depth is derive as

$$\mathcal{A}^P = \{n \in \mathbb{N} : c_n^P = 0\}$$

It should be noted that instead of replicating the sector information, the set contains the indexes of the respective segmentation results S_n^P . Finally, from this information a tip map I_{tip} with

$$I_{tip}(x, y) = \begin{cases} |\alpha_n^P - \beta_n^P|, & \text{if } \mathcal{A}^P = \{n\} \\ 2\pi r_{fix}, & \text{else} \end{cases}$$

and a pipe map I_{pipe} with

$$I_{pipe}(x, y) = \begin{cases} |\alpha_n^P - \beta_n^P| + |\alpha_m^P - \beta_m^P|, & \text{if } \mathcal{A}^P = \{n, m\} \\ 2\pi r_{fix}, & \text{else} \end{cases}$$

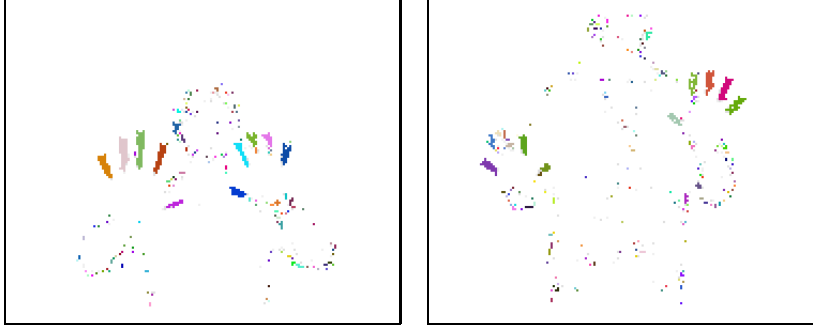


Figure 4.6: Example results of the pipe segmentation procedure.

is derived. Both feature maps facilitate the localization of tip respectively pipe appearances in the image based on the angular coverage of sectors with equal depth compared to the center pixel.

The results of the previous feature mapping is shown in Figure 4.5. The sample for l_{tip} illustrates that tip locations consistently exhibit lower pixel values as compared to the rest of the image. Consequently, tip appearances can be found as local minima in this feature map easily. As the second sample illustrates, in a similar way the pipe locations are exposed in l_{pipe} . In both images random noise can be observed, i.e. random pixels with low intensity that can be filtered out easily.

4.2.5 Pipe segmentation

The main goal of the pipe segmentation step is to find coherent regions $\mathcal{R} \in \mathcal{R}_{pipe} \subseteq \mathcal{P}(\mathcal{S})$ of low intensity in the feature map l_{pipe} . Similarly to object segmentation, which has been presented in Section 4.2.2, pipe segmentation is implemented by means of the FLOODFILL algorithm introduced in Section 2.1.4. However, for iteration the 4-connected connectivity scheme is used, i.e. only the neighbors along the four cardinal directions are considered. Additionally, instead of using a deviation threshold, iteration is limited to a fixed range of valid pixel intensities, i.e. pixel $P = (x, y)$ is only included if

$$0 \leq l_{pipe}(x, y) \leq \frac{\theta_{pipe}}{2\pi} 2\pi r_{fix} = \theta_{pipe} r_{fix}$$

The inequality reflects the maximum arc coverage, which has been defined for pipe pixels in Section 3.3.1. After conducting user experiments, the arc coverage was limited to an angle of $\theta_{pipe} = 0.7 \cdot 2\pi$, i.e. a maximum of 70 percent should be captured by the two sectors of the pipe pixels. The value provides a good balance between the thickness of the selected pipe structures while suppressing noise. Finally, having extracted the masks $\mathbf{M}$ using the FLOODFILL procedure, respective regions $\mathcal{R}$ are created such that

$$\mathcal{R} = \{(x, y) \in \mathcal{S} : \mathbf{M}(x, y) = 1\}$$

Obviously, the regions group those pixels into a common structure of pixels that been visited by the segmentation algorithm.

The results of the segmentation process are illustrated in Figure 4.6. Connected components are labeled with random colors across the entire color spectrum. As expected, fingers respond well to this selection process and consequently leave strong artifacts. Again, small random noise components are contained as well, but they can be filtered easily using a minimum size constraint.

4.3 Feature classification

In the following the individual feature classification steps are discussed in detail from raw feature extraction (Section 4.3.1), over to feature connection (Section 4.3.2), to finger aggregation (Section 4.3.3), and hand candidate grouping (Section 4.3.4).

4.3.1 Feature extraction

The first step during classification is the extraction of atomic features, i.e. features, which can be extracted from the prepared feature maps. In particular, a tip extractor F_{tip} and a distance maxima extractor $F_{distance}$ is employed to facilitate palm and fingertip recognition as described in Section 3.3.1. The extraction process is similar for both tip and distance features: First, the input image I_t (t refers to the feature type) is filtered with a Gaussian kernel G_t with

$$G_t(x, y) = \frac{1}{2\pi\sigma_t^2} e^{-\frac{x^2+y^2}{2\sigma_t^2}}$$

such that

$$I'_t = I_t * G_t$$

The filter is applied with the goal to improve the localization of the feature points. The variance σ_t , respectively the kernel mask size, depends on the concrete feature type: For distance features the kernel size 11×11 was selected, which provides a good balance between the number of feature points and runtime performance. In contrast, smaller values produce more local features while higher values are more difficult to compute. For tips the minimum kernel size 3×3 was selected, instead, due to the overall smaller appearance of the features. In contrast, higher values gradually cause the points to vanish. From the filtered feature map I'_t local turning points $P = (x, y)$ are selected using non-maximum suppression in a 3×3 window, i.e.

$F_t(x, y)$ is defined

$$\Longleftrightarrow$$

$$\forall Q = (x', y') \in \mathcal{S} : 1 \leq D_{ec}(P, Q) \leq \sqrt{2} \Rightarrow I'_t(x, y) \leq I'_t(x', y')$$

As mentioned previously, for tips local minima are selected. For distance features local maxima are selected instead, which reflects the maximum inscribed

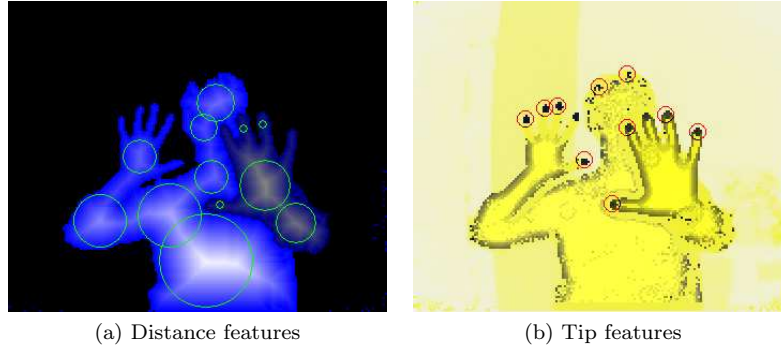


Figure 4.7: Result of the feature extraction procedure.

circle condition as described in Section 3.3.1. Finally, the locations of features are grouped into sets $\mathcal{F}_t$ such that

$$\mathcal{F}_t = \{P = (x, y) \in \mathcal{S} : F_t(x, y) \text{ is defined}\}$$

By means of the set structure the features can be accessed easily in subsequent algorithms for calculating richer information about the appearances.

The result of the feature extraction is illustrated in Figure 4.7. The left image shows the distance features (green circles) overlaid onto the distance transform visualization from Section 4.2.3. Clearly, the palms are represented well with this local turning point scheme. However, other points - e.g. on the arm or on the body - have to be filtered in subsequent steps. In contrast, the right image shows the tip features (red circles) overlaid onto a combination of l_{smooth} in the blue color channel and l_{tip} in the red and green color channel. Again, fingertips are represented well with this scheme, but some locations are missed while others are added. In sum, both features have proven sufficiently stable for determining stable hand appearances and pose information.

4.3.2 Feature connection

In the next step, the distance features are connected to a graph structure G with the goal to derive candidate palm locations as the endpoints of the network topology, i.e.

$$G = (\mathcal{V}, \mathcal{E}), \mathcal{V} \subseteq \mathcal{F}_{distance}, \mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$$

Given it is possible that small distance features are extracted around the finger appearances (e.g. see Figure 4.7a), they are eliminated from the current consideration by means of an empirically selected threshold $r_{min} = 5$ such that

$$\mathcal{V} = \{(x, y) \in \mathcal{F}_{distance} : l_{distance}(x, y) > r_{min}\}$$

The threshold has to be small enough to accept all possible palm features, but at the same time large enough to filter noise around the fingers. Otherwise, it cannot be guaranteed that palms are represented by the endpoints of the network

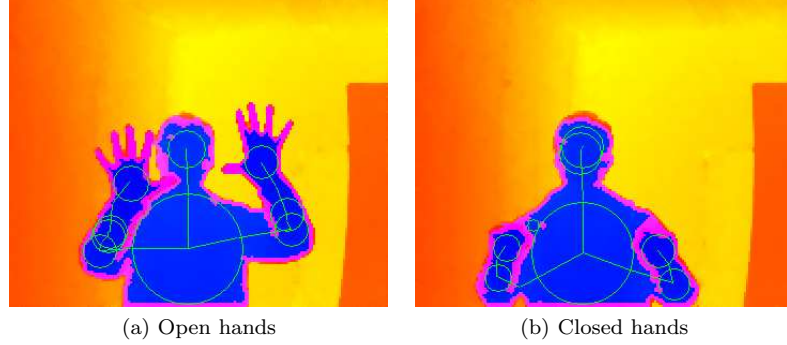


Figure 4.8: Result of the feature connection procedure.

topology. To derive the edges between the vertexes of the graph G , for each $P \in \mathcal{R}$ of each region $\mathcal{R} \in \mathcal{R}_{region}$ the closest distance feature $D_P \in \mathcal{F}_{distance}$ is calculated. The algorithm is based on a regular distance transform $l_{closest}$ that uses distance features as obstacles and remembers the closest obstacle at each location. For distance propagation, the neighborhood of a pixel is defined along the four cardinal directions only to avoid jumping gaps. Further, the connectivity between pixel P and pixel Q is restricted around depth discontinuities, i.e.

$$P \text{ is connected to } Q \Rightarrow D_{ec}(P, Q) \leq 1 \wedge l_{distance}(P), l_{distance}(Q) > 1$$

The second condition had to be added to remove border pixels from the consideration. Unfortunately, depth information is not very reliable along region borders (respectively depth discontinuities), which otherwise causes the distance transform algorithm to give spurious results. Finally, from the closest distance features D_P the set of edges $\mathcal{E}$ is derived as

$$\mathcal{E} = \{P, Q \in \mathcal{V} \times \mathcal{V} : P \neq Q \wedge |P - Q| \leq 1 \wedge D_P \neq D_Q\}$$

i.e. features are connected which have a common border in the distance transform.

The result of the entire procedure is illustrated in Figure 4.8. Two samples are provided, one where the hands are fully opened, and another where the hands are fully closed. The distance features of the set $\mathcal{F}$ and their connections $\mathcal{E}$ are annotated with green circles and lines. The depth image l_{smooth} is printed to the blue color channel, the distance transform $l_{closest}$ to the red and the green color channel instead. In both cases, the object structure seems to be approximated well by the abstract graph. Also, the topology constraint for palm candidate extraction is satisfied. Similar observations have been made for many other samples.

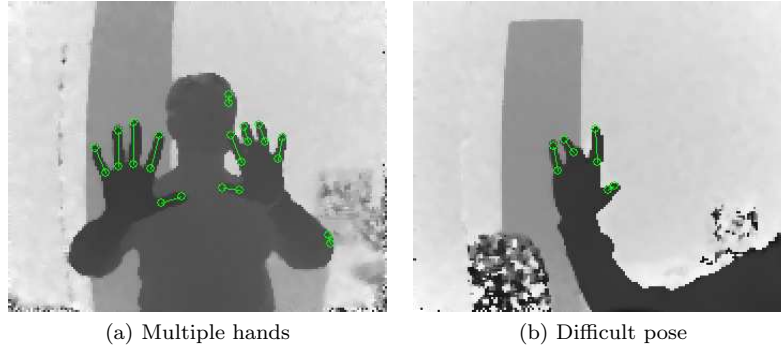


Figure 4.9: Results of the finger extraction algorithm.

4.3.3 Finger extraction

In the next step, the tip features from the previous section and the segmented pipe components from Section 4.2.5 are grouped to finger structures $F \in \mathcal{F}_{finger} \subseteq \mathcal{F}_{tip} \times \mathcal{S}$. Here, the term *finger* does not refer to the finger of a human hand, but to general finger-like appearances in the image. The algorithm first calculates for each tip feature $T \in \mathcal{F}_{tip}$ the set of connected pipe segments $\mathcal{R} \in \mathcal{R}_{pipe}^T \subseteq \mathcal{R}_{pipe}$ such that

$$\mathcal{R} \in \mathcal{R}_{pipe}^T \Leftrightarrow \exists \omega \in [\alpha_n^T, \beta_n^T] : T + \begin{pmatrix} r_{fix} \cos(\omega) \\ r_{fix} \sin(\omega) \end{pmatrix} \in \mathcal{R}, \mathcal{A}^T = \{n\}$$

where $\mathcal{A}^T$ denotes the indexes of the arcs with equal depth compared to the center pixel as introduced in Section 4.2.4. Given only one of the connected pipe segments can serve the purpose of building the finger structure, the largest component $\mathcal{R}_{largest} \in \mathcal{R}_{pipe}^T$ is selected, such that

$$\mathcal{R}_{largest} = \arg \max_{\mathcal{R} \in \mathcal{R}_{pipe}^T} |\mathcal{R}|$$

Finally, the finger structure F is derived from the tip location T and the pipe location $P = (x, y)$ that has the largest distance to the tip, i.e.

$$F = (T, \arg \max_{P \in \mathcal{R}_{largest}} D_{ec}(T, P))$$

The point P is also called the *base* of the finger and represents the location where the finger structure joins a larger component, such as the palm of a human hand.

The effect of the finger extraction procedure is illustrated in Figure 4.9. Two random samples are provided from a random test video stream, extracted finger structures are annotated with green circles for tip and base location and a green connecting line. In the left image two hands are presented simultaneously by the same user, in the right image a difficult pose is presented with a single hand instead. In both cases, the presented finger extraction algorithm maps

the fingers of the human hand very closely. However, noisy combinations are retrieved as well, e.g. at the elbow in Figure 4.9a, which have to be filtered out by subsequent algorithms.

4.3.4 Candidate grouping

The final step during classification deals with the task of selecting the palm candidates $C \in \mathcal{F}_{candidate} \subseteq \mathcal{V}$ from the set of graph vertexes $\mathcal{V}$ (see Section 4.3.2) and determine their grouping with the finger structures $F \in \mathcal{F}_{finger}$ from the previous section. As mentioned previously, the palm candidates C are determined as the endpoints of the network topology of graph G , and additionally a maximum palm radius of 13 pixels is required, i.e.

$$C \in \mathcal{F}_{candidate} \Leftrightarrow \mathbf{l}_{distance}(C) \leq 13 \wedge |\{P \in \mathcal{S} : (C, P) \in \mathcal{E}\}| = 1$$

The radius constraint serves to filter the head of the user, which usually also represents an endpoint in the topology. The selected value presents a reasonable trade-off between maximum palm appearance and minimum head appearance. Smaller values require generally more distance palms, higher values cause confusions with heads instead.

For each candidate C the set of plausible matches $\mathcal{M}_{finger}^C \subseteq \mathcal{F}_{finger}$ is derived as

$$\mathcal{M}_{finger}^C = \{F \in \mathcal{F}_{finger} : |\mathbf{l}_{smooth}(F_{base}) - \mathbf{l}_{smooth}(C)| \leq \epsilon \wedge \mathbf{D}_{ec}(F_{base}, C) \leq \delta_{max}\}$$

where F_{base} denotes to the location of the base of finger F . I.e. those fingers are selected who have a similar distance to the camera as compared to the distance feature, and whose distance to the palm center in image coordinates lies within a predefined range $[\delta_{min}, \delta_{max}]$. Likewise, the reverse mapping $\mathcal{M}_{palm}^F \subseteq \mathcal{F}_{candidate}$ is established to recover all plausible distance features given finger $F \in \mathcal{M}_{palm}$ with

$$\mathcal{M}_{palm}^F = \{C \in \mathcal{F}_{candidate} : |\mathbf{l}_{smooth}(F_{base}) - \mathbf{l}_{smooth}(C)| \leq \epsilon \wedge \mathbf{D}_{ec}(F_{base}, C) \leq \delta_{max}\}$$

where the parameters ϵ , δ_{min} and δ_{max} are identical to the previous definition. As before a distance threshold of $\epsilon = 1000$ is used, but the distance range is limited by $\delta_{max} = 13$ instead. The distance range reflects the maximum palm radius, which is the obvious limit of choice. In contrast, lower values exclude correct choices too frequently, while higher values potentially include structures from adjacent hands.

Having calculated the partitions $\mathcal{M}_{finger}^C$ and $\mathcal{M}_{palm}^F$, a voting procedure determines the best grouping of fingers and distance features. Therefore, first, for each feature C a score s_C is calculated with

$$s_C = \sum_{F \in \mathcal{M}_{finger}^C} \mathbf{D}_{ec}(F_{base}, C)$$

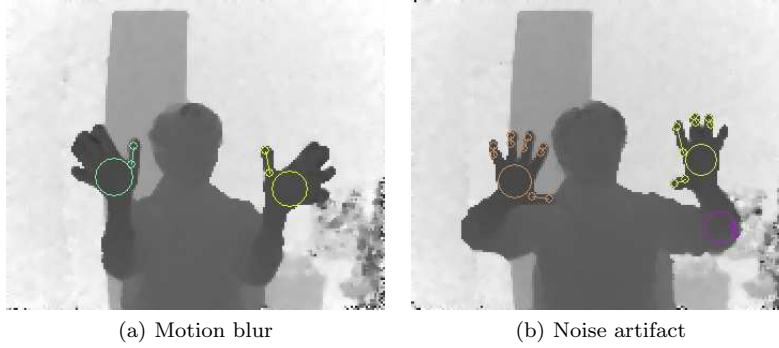


Figure 4.10: Results of the candidate grouping procedure.

summarizing the distances of plausible base points to the palm center. This score serves as the basis for establishing the grouping G between fingers F and distance features as

$$G : \mathcal{F}_{finger} \longrightarrow \mathcal{F}_{candidate}$$

with

$$G(F) = \arg \min_{C \in \mathcal{F}_{candidate}} \frac{s_C}{|\mathcal{M}_{candidate}^C|}$$

I.e. for each finger distance feature C is searched, which has the minimum score value per plausible finger match (also called average score). Finally, all fingers F that map to the same distance feature C are grouped into a set $\mathcal{G}_C$ such that

$$\mathcal{G}_C = \{F \in \mathcal{F}_{finger} : G(F) = C\}$$

The assignment determined the basis for hand detection and tracking as described in the next section.

The result of the grouping procedure is depicted in Figure 4.10. Two independent samples of a test video stream are shown and extracted groupings are highlighted on the depth image I_{smooth} with different colors using the previous annotation conventions. Clearly, all hands are mapped well by this technique even if motion blur prevents the detection of fingers. However, also outliers occur as shown in the right image. Still, the important step of perceptual grouping has been solved efficiently.

4.4 Object tracking

The following sections explain the individual steps for object tracking. The discussion starts with the realization of the palm tracking procedure in Section 4.4.1 and continues with the finger tracking procedure in Section 4.4.2.

4.4.1 Palm tracking

The first step during tracking is to solve for the dependencies between the palm locations $P \in \mathcal{F}_{palm}^t$ in frame t and the palm candidates $C \in \mathcal{F}_{candidate}^{t+\delta t}$ of frame $t + \delta t$. The reason why a restriction to palms is made first is because the palm in general exhibits a more stable appearance than fingers. The palm tracking procedure is based on the optical flow algorithm, which has been presented in Section 2.1.4. A multi-resolution feature-based variant is employed with three layers of the Gaussian pyramid to be able to cope with larger displacements due to fast hand motion. The algorithm operates on the distance transforms $l_{distance}^t$ and $l_{distance}^{t+\delta t}$ respectively. The windows size w_t is determined by the maximal palm radius of the previous frame t , i.e.

$$w_t = 1 + 2 \max_{P \in \mathcal{F}_{palm}^t} \{l_{distance}^t(P)\}$$

The reason is that, if the window size far surpasses the largest palm radius, too much background information is preventing the algorithm from converging, while too few palm coverage is not distinctive enough for finding the correct solution. Finally, a linear prediction model is used to pin-point an initial guess of the palm locations $P_*^{t+\delta t}$ for all hands which have been found in the last two frames t and $t - \delta t$ such that

$$P_*^{t+\delta t} = P^t + (P^t - P^{t-\delta t}) = 2P^t - P^{t-\delta t}$$

If only data from frame t is available, P^t is used as the initial guess instead. After providing the preliminary feature information, the optical flow algorithm converges to the position $P_{**}^{t+\delta t}$ or returns a tracking failure. Finally, for each $P_{**}^{t+\delta t}$ the closest palm candidate $C_P^{t+\delta t} \in \mathcal{F}_{candidate}^{t+\delta t}$ is selected, i.e.

$$C_P^{t+\delta t} = \arg \min_{C \in \mathcal{F}_{candidate}^{t+\delta t}} |P_{**}^{t+\delta t} - C|$$

If the distance between the closest candidate and the tracked location is smaller than the maximal palm radius $r_{max} = 13$, a hand feature $H \in \mathcal{F}_{hand}^{t+\delta t}$ is created, i.e.

$$D_{ec}(P_{**}^{t+\delta t}, C_P^{t+\delta t}) \leq 13 \Leftrightarrow C_P^{t+\delta t} = H \in \mathcal{F}_{hand}^{t+\delta t}$$

The threshold requires the tracked location to lie within the maximal extend of a palm region, which is a natural constraint for the association problem. In contrast, larger values might cause the the tracking to switch between left and right hand accidentally, while lower values are more reliant on accurate tracking output. Finally, for palm candidates $C \in \mathcal{F}_{candidate}^{t+\delta t}$ which have not been tracked successfully, but for which five finger structures have been detected in frame $t + \delta t$, also hand structures are created, i.e.

$$|\mathcal{G}_C^{t+\delta t}| = 5 \Rightarrow C = H \in \mathcal{F}_{hand}^{t+\delta t}$$

This simple constellation scheme defines the hand detection strategy employed in the system.

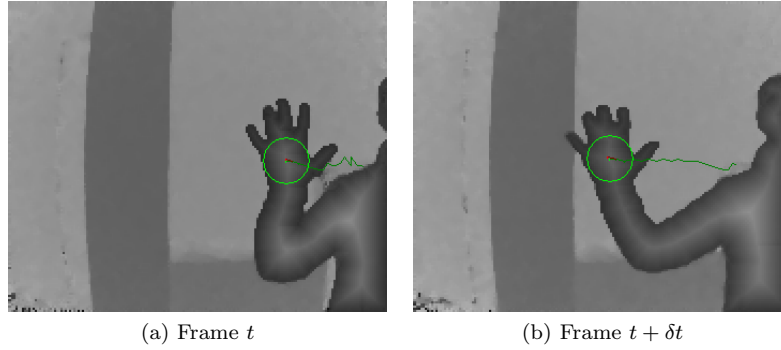


Figure 4.11: Results of the palm tracking procedure.

The result of the tracking mechanism is illustrated in Figure 4.11. Two time-consecutive samples from a test video stream are depicted, which are separated by some δt . Intermediate samples of the same sequence can be found in Appendix D. In all images the palm as well as the palm trajectory is annotated in green and the current prediction in red, while the background is composed of the depth image I_{smooth}^t and the distance transform $I_{distance}^t$. The memory is configured to store 20 consecutive frames of the sequence, which reflects the length of the trajectory. The illustration shows that the prediction mechanism works well for linear motion and tracking succeeds throughout the sequence. However, especially for fast non-linear motion the tracking fails repeatedly, which is a flaw in the current algorithm. The reason is that linear prediction might be inaccurate and the optical flow algorithm only compares the silhouette distances within a small window for deriving matches. Then, if other image locations exhibit similar appearances - such as the elbow or the shoulder of the user in Figure 4.11 - incorrect matches are yielded. To overcome this limitation, a non-optical flow based algorithm might be required that uses higher-level information such as the shape of the object, which has been demonstrated successfully by Barhate et. al. [5], or the relation of the object to other objects, e.g. the network topology of the distance feature graph.

4.4.2 Finger tracking

Finger tracking is very similar in concept and realization to the palm tracking procedure presented in the previous section. Again, the goal of the step is to derive the spatio-temporal dependency between fingers $F^t \in \mathcal{F}_{finger}^t$ of frame t and fingers $F^{t+\delta t} \in \mathcal{F}_{finger}^{t+\delta t}$ of frame $t + \delta t$, but only if the associated palm candidate C^t was also selected to be a hand, i.e.

$$G(F^t) = C^t = H^t \in \mathcal{F}_{hand}^t$$

To account for the new feature type, here, the optical flow algorithm is executed on top of the tip feature map I_{tip}^t with a decreased window size 7. The small

constant window size was selected due to the overall smaller appearance of fingertips to avoid inclusion of background information. Second, the prediction model is adapted to use the apparent motion vector of the palm H^t rather than assuming constant motion to calculate initial guesses for the new fingertip locations $F_{tip}^{t+\delta t}$ such that

$$F_{tip,*}^{t+\delta t} = F_{tip}^t + (H^{t+\delta t} - H^t)$$

The prediction model assumes that better estimates can be found by projecting the palm motion onto the fingers, which has shown good performance in the studies. After obtaining the tracked locations $F_{tip,**}^{t+\delta t}$ the closest finger feature $F^{t+\delta t}$ is selected with

$$F^{t+\delta t} = \arg \min_{F \in \mathcal{F}_{finger}^{t+\delta t}} D_{ec}(F_{tip}, F_{tip,**}^{t+\delta t})$$

Finally, a connection between F^t and $F^{t+\delta t}$ is established if their distance is within the Bresenham circle range r_{fix} , i.e.

$$D_{ec}(F_{tip}^t, F_{tip}^{t+\delta t}) \leq r_{fix}$$

The radius naturally limits the size of fingertip appearances, and has shown good performance throughout the experiments. In general, the size should not be chosen too large to avoid neighboring fingers from being associated incorrectly. Also, the size should not be chosen too small because otherwise any association becomes unlikely.

Obviously, the tracking results for fingertips are similar to the result of the palm tracking due to the similarity of the procedure (this is why samples are not provided here). However, the output is slightly less stable mainly for two reasons: First, the tip feature classifier is much more susceptible to noise artifacts at the current stage, which causes a relatively large number of false-negatives. Second, the motion of fingers can be much more complex since it consists of two independent components: The palm motion and the joint flexion. Overall, the results were sufficiently stable to implement a simple gesture recognition module as presented in the following section.

4.5 Gesture recognition

In the following the gestures are presented that have been implemented for the user evaluation. The gestures are designed to imitate the functionality of the CityWall [58], a large multi-touch interface installed at the city center of Helsinki for exploring Flickr¹ images of the local community. In particular, the 3D multi-touch prototype supports loading of images, selection of existing images for manipulation, translation of images, rotation respectively scaling of images, and image deletion. These interaction techniques have already been

¹<http://www.flickr.com>

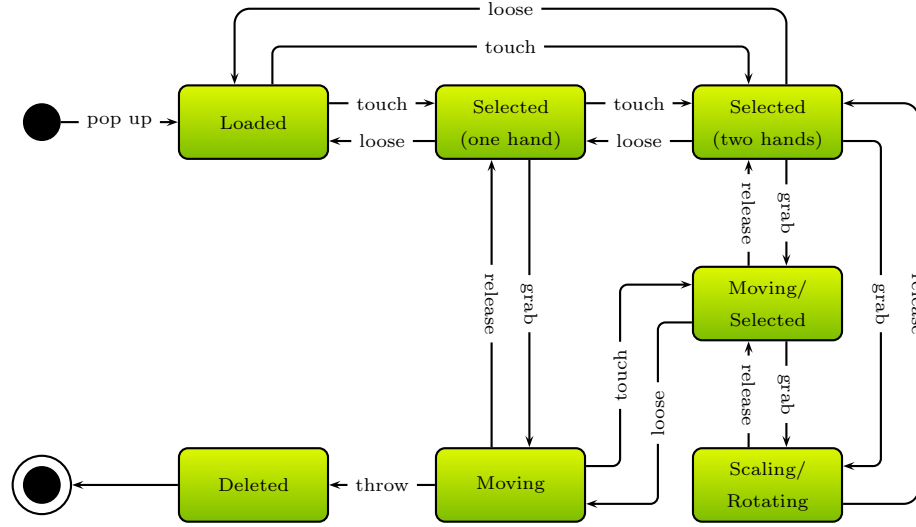


Figure 4.12: State chart of interaction with images.

evaluated positively on the surface-based touch interface of the CityWall, and the success of their adoption to the novel 3D environment is to be investigated in this thesis.

For reference, a state chart diagram of the interaction with images is provided in Figure 4.12. The diagram shows all states that an image can accept during its life time from loading till deletion. The transitions between the individual states are triggered by the following hand gestures:

- *Pop up* for loading images, Section 4.5.1
- *Touch* for selecting images, 4.5.2
- *Loose* for deselecting images, Section 4.5.3
- *Grab* for starting image manipulation, Section 4.5.4
- *Release* for stopping image manipulation, Section 4.5.5
- *Throw* for deleting images, Section 4.5.6

It should be noted that additional transitions exist for cases when hands are lost by the tracker. However, these transitions are omitted to increase the readability of the figure. In turn, the gesture specification is based on a formal definition of the images I with

$$I = (C, \vec{x}, \vec{y}, \vec{z}, w, h)$$

where C refers to the center point of the image in 3D space, $\vec{x}, \vec{y}, \vec{z}$ represent orthonormal basis vectors of an image-local coordinate frame and w, h store the width and the height of the image in the local coordinate frame respectively.

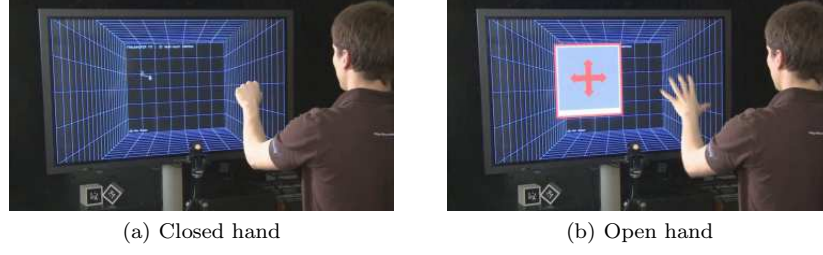


Figure 4.13: Illustration of the pop up gesture.

Finally, it is assumed that the 3D coordinates of image pixels $P = (x, y) \in \mathcal{S}$ can be obtained by a mapping $\mathbf{C}$ with

$$\mathbf{C} : \mathcal{S} \longrightarrow \mathbb{R}^3$$

In the case of the SwissRanger SR4000, which has been used in the experiments, this mapping is already provided by the device driver library. For other systems, such as the Minoru stereo camera, calibration procedures have to be used to derive the information from the disparities obtained from stereo matching and depth estimation (see Section 2.1.2).

4.5.1 Pop up gesture

The pop up gesture is inspired by the idea of popping up objects in space. For hands, this metaphor is translated to a closed hand that suddenly shows all five fingers as illustrated in Figure 4.13. Technically, the gesture is triggered by a free hand $H^t \in \mathcal{F}_{hand}^t$ if several conditions are met. First, number of fingers in previous frames is constrained by

$$\max_{t-12\delta t < t' \leq t-3\delta t} |\mathcal{G}_{H^{t'}}^{t'}| \leq 2 \wedge \frac{1}{9} \sum_{t-12\delta t \leq t' \leq t-3\delta t} |\mathcal{G}_{H^{t'}}^{t'}| \leq 1$$

i.e. the maximum and average finger counts are limited. The thresholds account for noisy finger detections, which might occur even if the hand is fully closed. Second, a stable hand location $\mathbf{C}^t(H^t) = P^t \in \mathbb{R}^3$ is required. Therefore, first the covariance of the location is calculated using

$$\mathbf{V}_{i,j}^{t_1,t_2} = \sum_{t'=t_1}^{t_2} (P_i^{t'} - \mu_i^{t_1,t_2})(P_j^{t'} - \mu_j^{t_1,t_2})$$

with

$$\mu_i^{t_1,t_2} = \frac{t_2 - t_1}{\delta t} \sum_{t'=t_1}^{t_2} P_i^{t'}$$

where $i, j \in \{x, y, z\}$ denote one of the three dimensions of the position. Finally, to implement the stable condition constraint for the palm location, the

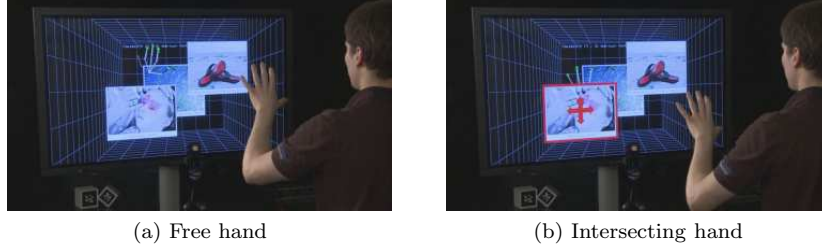


Figure 4.14: Illustration of the touch gesture.

covariance matrix entries are limited by a constant threshold, i.e.

$$\forall i \in \{x, y, z\} : \mathbf{V}_{i,i}^{t-8\delta t, t} \leq (32cm)^2$$

translating to a limited motion of the palm during the last eight frames. The threshold $\frac{1}{10}$ has to be selected small enough to prevent moving hands from triggering the pop up gesture accidentally, since motion blur causes poor detection of fingertips and hands might be interpreted as closed. On the other hand, it has to be large enough to be robust to noise of the depth measurements. Finally, multiple fingers have to be detected in the most recent frames, i.e.

$$\frac{1}{3} \sum_{t-3\delta t < t' \leq t} |\mathcal{G}_{H^{t'}}^{t'}| \geq 3$$

Again, a statistical aggregation of the finger information is used to make the system more robust to unstable finger detections. Finally, if all the previous conditions match for hand H^t , an image I is created and its contents are loaded randomly from Flickr.

4.5.2 Touch gesture

The touch gesture is inspired by the idea of reaching towards objects. For hands, this metaphor is translated to a hand intersecting with the plane of the image as illustrated in Figure 4.14. Technically, the gesture is triggered for image $I = (C, \vec{x}, \vec{y}, \vec{z}, w, h)$ and hand $H \in \mathcal{F}_{hand}$ if the image coordinate frame relative vector $R = C(H) - C$ meets the condition

$$|R \cdot \vec{x}| \leq w + 5cm \wedge |R \cdot \vec{y}| \leq h + 5cm \wedge |R \cdot \vec{z}| \leq 10cm$$

I.e. not a direct intersection between the plane and the hand is required, but rather a threshold is implemented within which the touch event is sensed. The threshold parameters have been selected to provide more flexibility in $\vec{z}$ direction of the image coordinate frame as opposed to the other axes. As a result, the user is required to intersect the target object more closely in $\vec{x}$ and $\vec{y}$ direction, whereas the $\vec{z}$ direction is more tolerant. Usually, the $\vec{z}$ tolerance translates to a decreased need for reaching towards the objects as the images are mostly

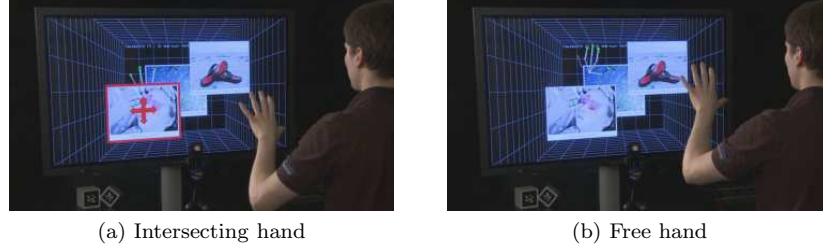


Figure 4.15: Illustration of the loose gesture.

facing the user during operation. Larger values degrade the localization of the selection event, while smaller values require more precise selection.

Finally, it should be noted that, if technically multiple objects are touched with one hand, the first image out of the randomly ordered list of candidates is selected only. More advanced selection mechanisms are perceivable, e.g. by selecting the candidate with the smallest center distance to the palm. However, due to time constraints such selection schemes could not be added to the user interface logic of the demo application.

4.5.3 Loose gesture

The loose gesture represents the direct opposite of the touch gesture, which has been presented in the previous section. Consequently, the gesture is recognized if a hand is moving away from the target object. Technically, again, the same thresholds for the $\vec{x}$, $\vec{y}$ and $\vec{z}$ directions of the image coordinate frame are used to formulate the constraint

$$|R \cdot \vec{x}| > \frac{w}{2} + 5cm \wedge |R \cdot \vec{y}| > \frac{h}{2} + 5cm \wedge |R \cdot \vec{z}| > 10cm$$

If these values are surpassed, the selection between the image I and the hand H is resolved.

4.5.4 Grab gesture

The grab gesture - as the name suggests - is inspired by the idea of grabbing objects with the hand. Breaking the gesture down into its components, it consists of a fully opened hand that is closed to wrap the target objects with the fingers as illustrated in Figure 4.16. Technically, it is recognized for hand $H^t \in \mathcal{F}_{hand}^t$ in frame t if the palm position $C^t(H^t) = P^t \in \mathbb{R}^3$ is sufficiently stable, i.e. given the notation of the covariance matrix $V_{i,j}^{t_1,t_2}$ from Section 4.5.1, the constraint

$$V_{i,i}^{t-4\delta t,t} \leq (10cm)^2$$

has to be met. Again, the stable position is required to avoid spurious triggering of the grab gestures due to moving hands, since motion blur prevents fingertips

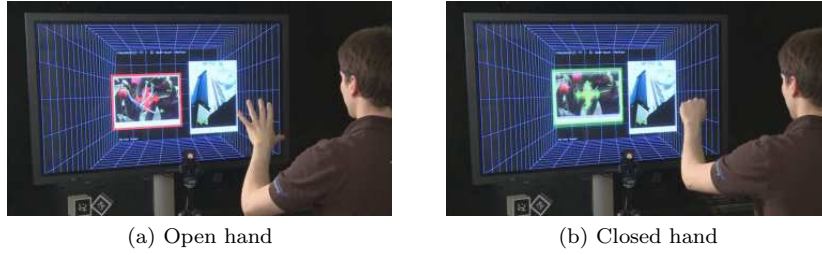


Figure 4.16: Illustration of the grab gesture.

from being detected reliably. However, this time a smaller threshold is required to achieve a better localization of the grab point, which serves as the reference of the move and scale actions respectively. Finally, for the grab event to be actually fired, the number of fingers in frame t is limited by

$$|\mathcal{G}_{H^t}^t| \leq 1$$

i.e. at most one finger is allowed to be detected for the hand. The threshold was not set to zero since for many users still a finger could be detected even if the hand was closed (e.g. a part of the thumb). On the other hand, higher values cause the gesture to trigger too frequently, even if not desired.

4.5.5 Release gesture

The release gesture is the direct opposite of the grab gesture, which has been presented in the previous section. Accordingly, the release event is triggered if the hand of the user is opened again as illustrated in Figure 4.17. Technically, this condition is realized by defining a threshold on the number of finger detections for hand $H^t \in \mathcal{F}_{hand}^t$ over the last five frames, i.e.

$$\frac{1}{5} \sum_{t-5\delta t < t' \leq t} |\mathcal{G}_{H^{t'}}^{t'}| \geq 1.5$$

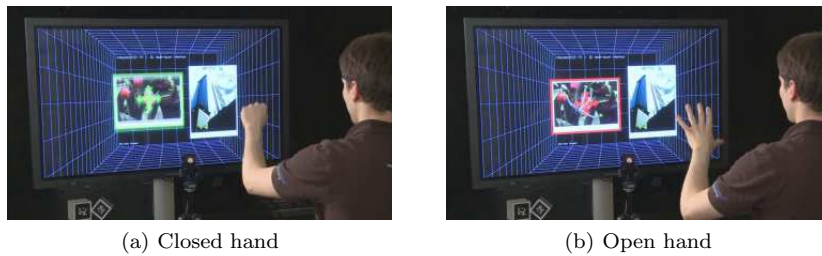


Figure 4.17: Illustration of the release gesture.

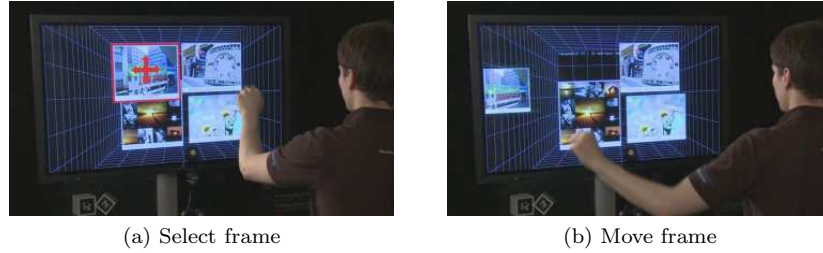


Figure 4.18: Illustration of the throw gesture.

Again, a threshold has to be used to account for rare false-positive finger detections. In contrast, lower values could also be achieved easily by hands that are not opened fully. Higher values rely on more stable finger detections instead.

4.5.6 Throw gesture

Finally, the throw gesture is inspired by the idea of throwing obsolete objects into the trash, which is a common metaphor used by many computer applications. Since fast motion is currently not supported by the tracking algorithm, this action has been translated to moving images across the border of the tracked space as illustrated in Figure 4.18. Technically, this condition is recognized for a hand $H \in \mathcal{F}_{hand}$ by a simple threshold on the palm location $\mathcal{C}(H) = (x, y, z) \in \mathbb{R}^3$ such that

$$|x| \geq 35cm \vee |y| \geq 30cm$$

The threshold is selected such that the hand is reaching the borders of depth image given the average position of the user during interaction. As a result, the interaction area is effectively limited to $70cm \times 60cm = 4200cm^2 = 0.42m^2$ in the current gesture model with approximately one meter tolerance in $\vec{z}$ direction. A larger area respectively volume can be achieved easily by allowing larger and more flexible distances of the user to the camera, while redefining certain parameters of the gesture model. In particular, the borders should then be defined adaptive to the distance of the user/hand from the camera to reflect the frustum structure of the camera capture volume.

4.6 Software Architecture

In this section the software architecture is presented, which has been designed to implemented the previous functionality. The discussions starts with an overview over the package structure in Section 4.6.1, before concentrating on the central data structures for the algorithms in Section 4.6.2 and concluding with the classes for interaction state representation in Section 4.6.3.

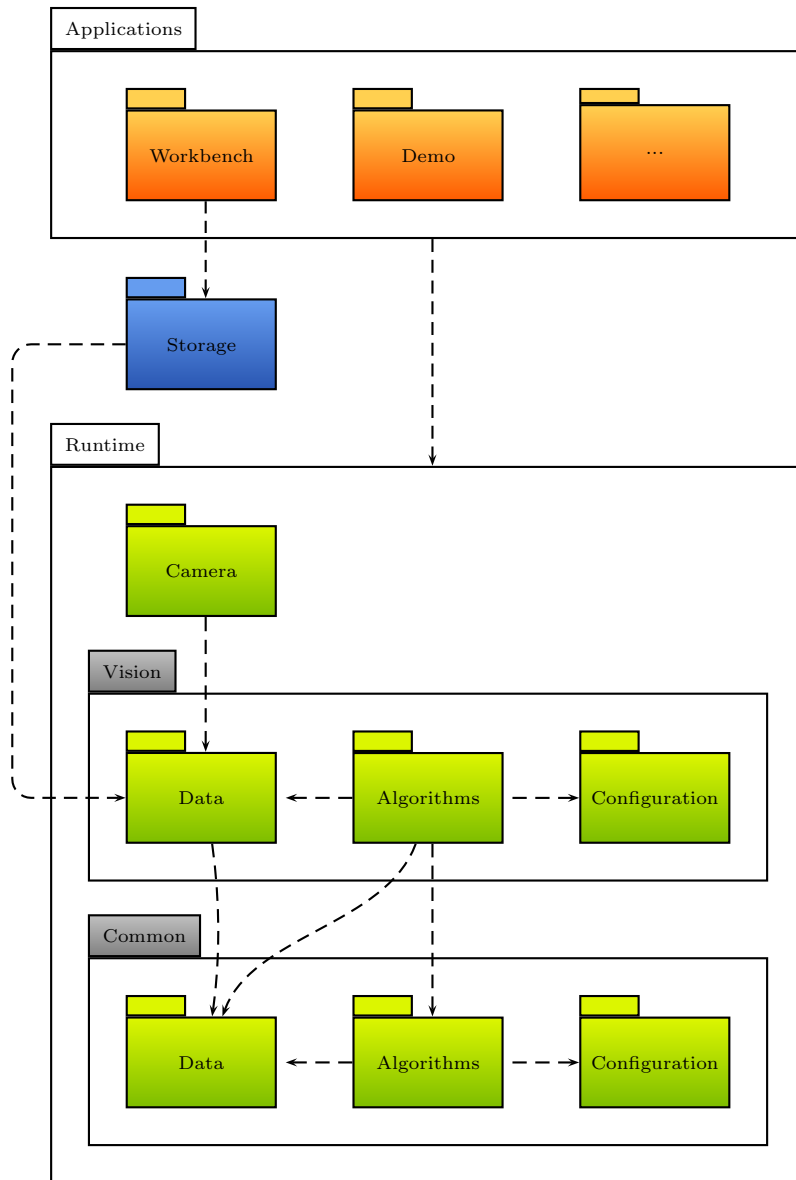


Figure 4.19: Overview of the package structure for implementation.

4.6.1 Package structure

A visual overview of the package structure is provided in Figure 4.19. The general architecture is layer-based with dependencies flowing from top to bottom. The three main packages in this layered hierarchy are the APPLICATIONS, the STORAGE and the RUNTIME. The APPLICATIONS package subsumes - as the name suggests - the applications which are built on top of the core hand tracking and gesture recognition framework. Modules of this layer can be distinguished by a unique graphical user interface (GUI), which provides access to a unique set of system features. In contrast, the other two layers STORAGE and RUNTIME do not provide GUIs themselves, but application programming interfaces (API) instead. The STORAGE package is responsible for managing the file system storage for previously captured and annotated video streams, while the RUNTIME package provides access to the cameras and vision algorithms.

The internal structure of the RUNTIME package is again organized in a layered fashion: At the bottom the COMMON package covers data structures, algorithms and configuration variables which are used commonly for all kinds of problems not related to computer vision. For example, it includes proprietary implementations of linked lists, memory management and sorting of numeric elements, which are all heavily used in the remaining system. The VISION layer is built on top of this basic infrastructure and supplies a similar set of components (data structures, algorithms, configuration), which are tied to vision-related purposes instead. In many cases the module can rely on the underlying functionality to solve its tasks. Finally, the RUNTIME layer contains a proprietary interface for accessing camera devices as part of the CAMERA package. The captured image data is delivered consistently in form of VISION data structures, which are required by the VISION algorithms in turn. This allows a seamless integration of video streams with the tracking framework.

On top of the fundamental RUNTIME package a compact STORAGE module is provided. As previously mentioned, its purpose is to manage the access to the file system for storing video streams, which have been captured from the TOF camera. Consequently, the module depends on the VISION data structures and thus produces output that can be fed into the VISION algorithms directly. To support different research projects with varying dataset requirements (e.g. high resolution model-based hand tracking, integrated full body tracking, etc) simultaneously, the streams are organized into projects, which can be loaded and saved independently. Finally, the data streams themselves are stored in raw ASCII format, i.e. currently no advanced video encoding and compression method is employed because memory efficiency was not a primary goal during implementation, but rather a working proof of concept was required.

The top-most APPLICATIONS layer obviously depends on the RUNTIME package for integrating 3D hand tracking information. Currently, there are two independent applications: The WORKBENCH and the DEMO. The first application was heavily used during development for tracing algorithm performance and debugging its implementation. For this purpose the dependency on the STORAGE package was defined to enable repeated evaluation on test data. The DEMO ap-

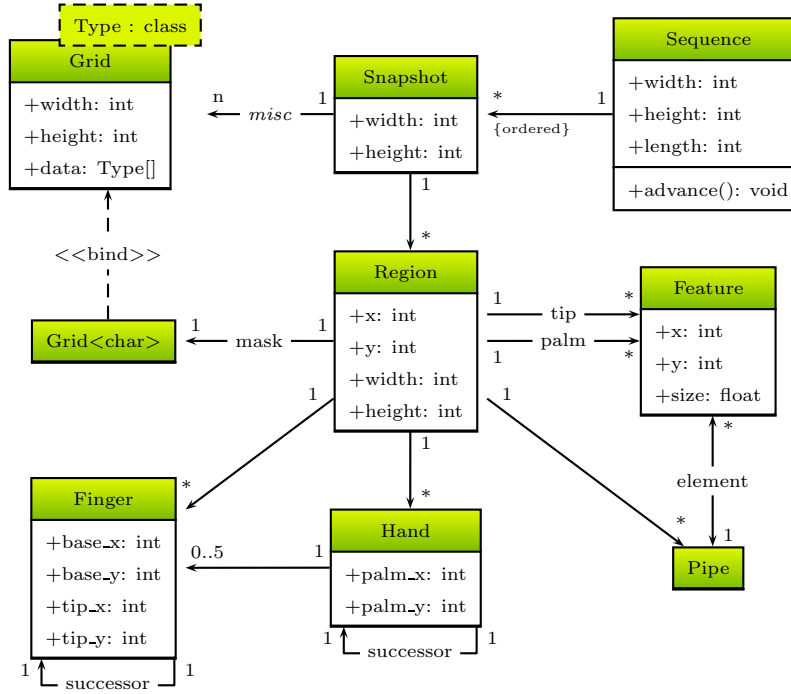


Figure 4.20: Overview of the data structures for implementation.

plication is a fully-functional 3D multi-touch prototype, which implements the gesture recognition techniques from Section 4.5. It was used during a user study to gain a first insight into the psychological effects of bare-hand 3D interaction on the user.

4.6.2 Data model

The data model, upon which the previous algorithms are built, is depicted in Figure 4.20. On the top left, the central information container is presented: The GRID template. A grid conceptually represents a two-dimensional array of elements whose data type can be specified by the template parameter **Type**. Primarily, this data structure is used to store and access raw image data in main memory. However, other use cases are defined as well, e.g. for linking pixels with feature information for constant time access. Most grid instances are managed by a SNAPSHOT object, which groups information extracted from one frame of a video sequence. The snapshots themselves are finally contained in a SEQUENCE object, which puts the snapshots into temporal context with the other frames of the video sequence. SEQUENCE objects work like ring buffers with a fixed number of slots, i.e. if a maximum number of snapshots is reached the oldest will be freed. This way, extracts from video streams can be managed efficiently

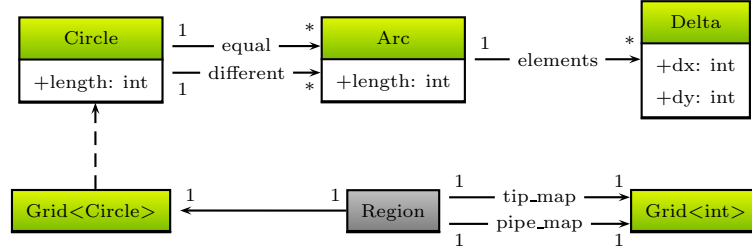


Figure 4.21: Data structures for the fast classification procedure.

in main memory, and tracking algorithms are able to access sufficient temporal information.

The classes GRID, SNAPSHOT and SEQUENCE represent a generic high-level data interface, which can be used independently of the concrete vision task. In contrast, the remaining entities are specifically designed for the individual steps of the employed vision algorithms. The REGION class (center of Figure 4.20) represents coherent image regions, which have been segmented as described in Section 2.1.4. To recover the true stretch of the regions a binary mask is provided, which is realized by the GRID template. Further, atomic features are represented using the class FEATURE, pipes are stored as instances of the class PIPE, finger-like appearances are mapped by the class FINGER and combined hand structures are translated to HAND objects. For tracking, the class HAND and the class FINGER additionally provide links to their successors.

It should be noted that for some steps of the processing chain extensions to the basic data model have to be implemented. Usually, this includes various types of GRID structures to represent the individual features maps, which are extracted from the raw image data. The most important amendment is required by the circular neighborhood classification algorithm, which has been presented in Section 4.2.4. The corresponding data structures are presented in Figure 4.21. Two GRID structures are used for storing the tip and the pipe feature maps. Further, the CIRCLE data structure is added to store the circular segmentation results in ARC instances for each pixel of the GRID<CIRCLE> map. Finally, DELTA information is provided to be able to iterate over the discrete circumcircle.

4.6.3 Interaction components

The final section of this chapter covers the mapping of the gesture recognition specification, which has been presented in Section 4.5, onto concrete runtime components as illustrated in Figure 4.22. The basic idea is that the states of the image interaction model are mapped onto ACTION classes, while the base class defines a public interface for executing the transitions. Subsequently, the actions are processed according to a protocol that defines the concrete order of the method calls, and each concrete class takes the appropriate steps to

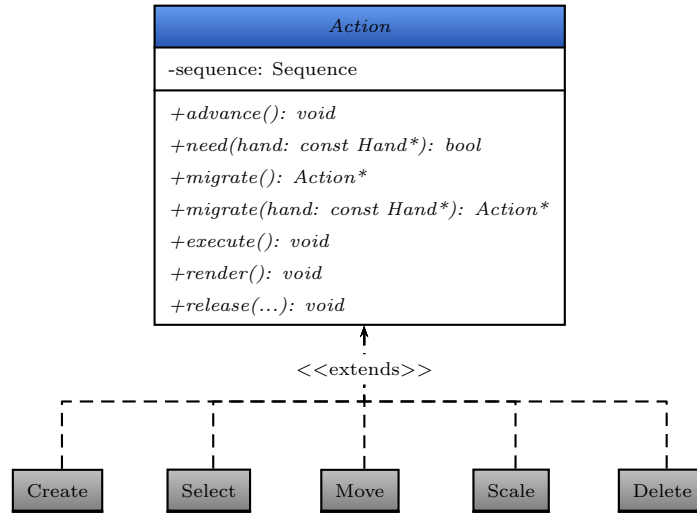


Figure 4.22: Mapping of user interface actions to runtime components.

determine the current state of the system.

The protocol is defined as follows: First, the method **advance** is used to indicate that a new frame has arrived and that the actions should prepare for updated hand information. In the next step, the available hands are distributed among the currently running actions by means of the **need** method. The main purpose of the method is to acquire the successors of already used HAND instances from the previous frame. After this round of distribution, the actions are allowed to **migrate** to new actions, e.g. a SCALE action might become a MOVE action if track of one of the hands was lost. Another common use is to terminate the current action by returning the NULL value. Then, for the remaining actions a second round of hand distribution is started, which allows to **migrate** to new actions that involve another hand. This mechanism is for example used to change from moving to scaling objects in the scene by placing a second hand onto the respective object. Finally, the effects of the actions are applied to the scene contents by calling the method **execute**. This causes the objects to move, scale or vanish depending on the concrete action type.

Chapter 5

Evaluation

In a final phase of the research project, the implementation, which has been presented in the previous chapter, was evaluated for its technical performance and usability among a number of trial users. In this fifth chapter the methodology and results of the evaluation are presented. The discussion starts with an overview of the evaluation phase in Section 5.1, before presenting the design of the evaluation procedure in Section 5.2 and providing the results of the individual studies that have been conducted in Section 5.3 and 5.4.

5.1 Overview

As previously mentioned, the purpose of the evaluation phase was to conduct an initial user study on barehanded 3D interaction technology and to collect user feedback from which final conclusions about the usability of the system and the current state of implementation are drawn. To facilitate user-technology engagement, a set of three tasks has been developed utilizing the interaction model, which has been defined in the previous section. In the next step, the tasks have been integrated into a demo application, which is built upon the RUNTIME infrastructure as introduced in Section 4.6.1. Finally, based on this demo application an experimental setup was configured including the TOF camera, a workstation PC and a projector as illustrated in Figure 5.1. The details about the experiment design - from the task definitions to the hardware setup - is described in detail in Section 5.2.

Based on the experiment design, two disjoint sets of trial users were selected to form a *pilot* study group and *main* study group. The purpose of the pilot study was to recognize major issues in the design and implementation of the system and the evaluation procedure, which have a strong negative impact on the user feedback and hence need to be eliminated prior to conducting the final experiments. In contrast, the purpose of the main study was to collect the final user feedback from which the conclusions about the technology are drawn. The independent experiments and their results are described in Section 5.3 and 5.4.

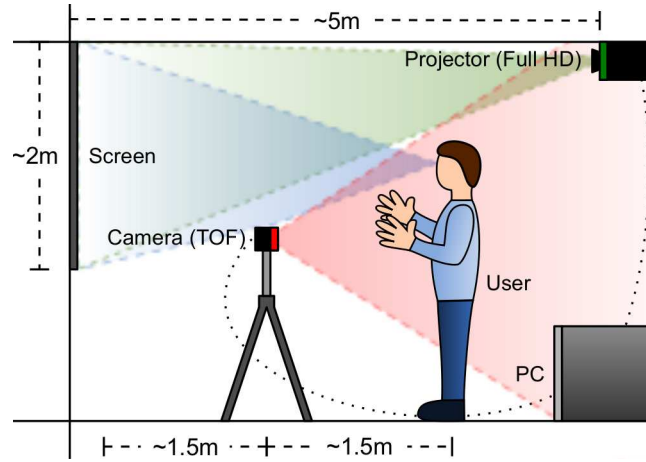


Figure 5.1: Experimental setup for the user studies.

5.2 Design

As mentioned previously, the user experiments are composed of a set of tasks, which are included in a demo application, an experimental setup of hardware components for running the application and a methodology for collecting feedback from the trial users. In the following, the hardware setup is introduced in Section 5.2.1, before describing the tasks in Section 5.2.2 and specifying the methodology in Section 5.2.3.

5.2.1 Setup

The experimental setup has previously been illustrated in Figure 5.1. The user stands approximately 3 meters in front of a silver screen, which is illuminated by a high resolution (1920 x 1080 pixels) projector. The camera SwissRanger SR4000 is mounted on a tripod and placed between screen and user at approximately 1.5 meters to each side. To capture the hand gestures it is facing the user pointing slightly upwards from below the screen. It should be noted that alternatively the camera can also be installed on top of the screen, as commonly known from other motion capture products such as the Sony EyeToy¹. However, the quality of pose estimation strongly depends on a “good” view of the hand, in particular on the angle between camera direction and arm direction. As a consequence, the arms might have to be held considerably above shoulder level, which is an inconvenient pose for longer interaction sessions.

Finally, camera and projector are connected to a regular workstation PC with an Intel Core 2 Quad 2.8GHz processor, 3.25 GB DDR2 RAM and an NVidia Quattro FX graphics card. The workstation takes care of executing the

¹<http://www.eyetoy.com/>

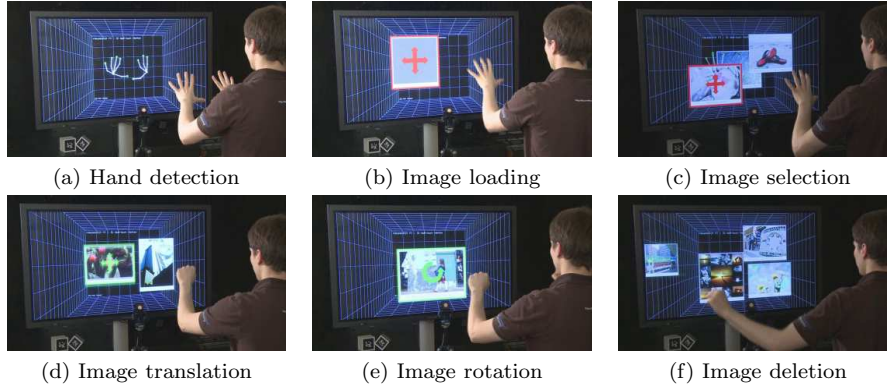


Figure 5.2: Illustration of the introduction task.

vision respectively gesture recognition algorithms (see Chapter 4), and rendering the OpenGL-based user interface in real-time at a maximum of 30 frames per second. In the overall first experiment the user interface was rendered using anaglyph images for creating an illusion of depth. However, since the stereoscopic rendering was perceived as irritating it was disabled in later experiments. Possible explanations for the irritation might be the poor adjustment of the stereoscopic parameters as well as the low color fidelity of the separated color channels. Further investigations into this issue could not be included into the scope of this thesis due to time constraints.

5.2.2 Tasks

Each experiment is divided into three tasks: First, the subject is guided through an introduction to teach the interaction and gesture model and build awareness towards tracking and occlusion problems. Then, two goal-directed tasks have to be completed to collect performance feedback.

First Task: Introduction

The individual steps of the introduction task are depicted in Figure 5.2 in chronological order. Basically, the user is guided through the individual states of the image interaction model, which has been introduced in Section 4.5. Therefore, the respective hand gestures for triggering the state transitions are explained and practiced without hard temporal constraints until the subject feels familiar enough with the system to proceed. Generally, during this stage of the experiment extensive communication between the investigator and the subject are encouraged to help building awareness for the system functionality and technical issues such as tracking limits or depth segmentation constraints. The image material is loaded on the fly from a predefined Flickr query, which is identical for all the subjects. Further, images can be created and deleted as

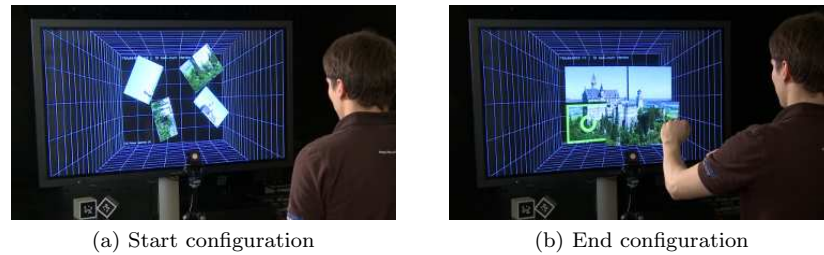


Figure 5.3: Illustration of the 2D puzzle task.

desired to encourage explorative interaction styles with the application.

Second Task: 2D Puzzle

The purpose of the 2D puzzle is to evaluate the usability of the system with respect to tasks that can be executed in a two-dimensional plane only, and therefore allow equivalent implementations on surface-based multi-touch devices. In the start configuration of the puzzle a predefined perturbation of four image pieces is presented as depicted in Figure 5.3a. Each subject is given enough time to get familiar with the configuration and prepare a solution strategy prior to starting execution and time measurement. The solution requires the subject to apply both translation as well as rotation and scaling for reaching the final configuration. Additionally, the depth cue can be used to organize the pieces and gain better access to the target objects, but utilization of the cue is not required. Finally, the time measurement is stopped as soon as the subject finishes a reasonable configuration of the pieces, as for example shown in Figure 5.3b. It should be noted that exact snapping of the elements was not implemented, which is why only a subjective interpretation of the term reasonable by the investigator could be used.

Third Task: 3D Puzzle

Finally, the purpose of the 3D puzzle is to evaluate the usability of the system with respect to tasks that require the manipulation of objects along all three dimensions. In the start configuration of the puzzle four objects are displayed, one static ground plane and three dynamic scene items. The dynamic scene items are distributed similar to the 2D puzzle in a predefined configuration across the operation space as shown in Figure 5.4a. However, this time the individual images are not split into pieces, but the scene placement and relative scale/rotation are altered instead. Again, the time measurements only start after the subjects have made themselves familiar with the problem and developed a strategy for solving the task. The solution generally requires everything from translation along all three dimensions to rotation and scaling. As before, the experiment is stopped as soon as a reasonable configuration is achieved, as for example depicted in Figure 5.4b.

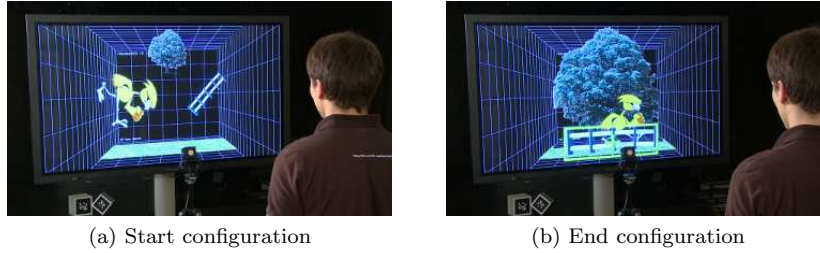


Figure 5.4: Illustration of the 3D puzzle task.

5.2.3 Method

The discussion of the experimental design concludes with the specification of the experiment procedure, which is passed through by every subject individually. First, the investigator provides a narrative introduction into the research project and the goal of the evaluation. Then, a questionnaire is presented to the subject for collecting the user feedback. The feedback is divided into ratings of statements on a 5-point Likert scale for each task as well as open questions for additional comments and explanatory notes. After every task completion the subject is asked to fill the respective rating form with the goal to collect the most recent impressions. In contrast, the comments are provided after finishing the entire experiment to obtain retrospective feedback on the session. Finally, each subject is asked to provide additional personal information, which allows an advanced classification of the evaluation results based on personal attributes and experiences. The entire procedure was designed to not exceed a duration of 30 minutes with each task taking up to 5 minutes and a minimum of 15 minutes for open feedback collection and discussion.

Despite the questionnaire, a second note sheet per subject was maintained by the investigator. The sheet was used to record task completion times as well as general positive and negative observations about the interaction of the user with the system. Both the user comments and the investigator observations leave their mark on the discussion of the evaluation results in the final section of this chapter.

5.3 Pilot Study

The pilot study was divided into three phases: First, the preliminary interaction model was tested for its usability and refinements were implemented to address the major issues as described in Section 5.3.1. Second, the final interaction model was tested for its performance and parameters were tuned to make interaction more efficient as described in Section 5.3.2. Finally, some minor adjustments to the parameters of the core algorithms were made after running a third series of experiments as described in Section 5.3.3.

5.3.1 Interaction model refinement

The first set of experiments was conducted with only a total of three users, one female and two male between the age of 28 and 31. It quickly became clear that the initial interaction model was lacking an explicit grab gesture for triggering modifications such as moving or scaling images. The initial model simply used the intersection of a hand with an image for triggering the respective state changes. Consequently, images were modified accidentally while the user was moving his hand across them for reaching a second object. To overcome this issue, all three subjects recommended to introduce an active gesture for triggering the respective modifications. The choice of the grab gesture seemed natural due to the grab and release metaphor for handling objects in the real world. Secondly, the visualization of the interaction states was improved by distinguishing selection (green frame around the image) from modification (red frame around the image) and translation (cross) from rotation and scaling (circle). The suggestion for improving the visualization was only proposed by one out of three subjects, however, it was expected to improve the efficiency of interaction as the subjects are more aware of the system state at each time. The pictures throughout the previous and the current chapter already show the modified visualization technique.

5.3.2 Interaction model tuning

The second set of experiments was conducted with only a total of two users, two male between the age of 48 and 51. It became clear quickly that the implementation of the grab gesture, which had been included after the previous set of experiments, was too restrictive, which rendered the system difficult to handle by the two subjects. In particular, the range within which the gesture was associated with a certain image was confined to the image's plane only. As a consequence, the subjects frequently missed the target object while performing the manipulation due to insufficient depth awareness. To overcome this issue, both subjects recommended to increase the tolerance for the association between gesture and scene object. Consequently, the range was extended to an entire box volume around the image as opposed to the sole image plane only.

5.3.3 Tracking algorithm refinement

The third set of experiments was conducted with a total of five users, two female and three male between the age of 28 and 55. After the previous modifications to the systems, the overall feedback to the system was much more positive, e.g. four out of five subjects considered the system intuitive as opposed to one out of three in the initial experiments. On the other side, the major problems that have been identified by the final set of pilot study users were issues with the tracking (three out of five) and irritation by the measurement noise (three out of five). Due to time constraints, only one of the two issues has been addressed, namely tuning of the tracking parameters. Most importantly, the graph representation

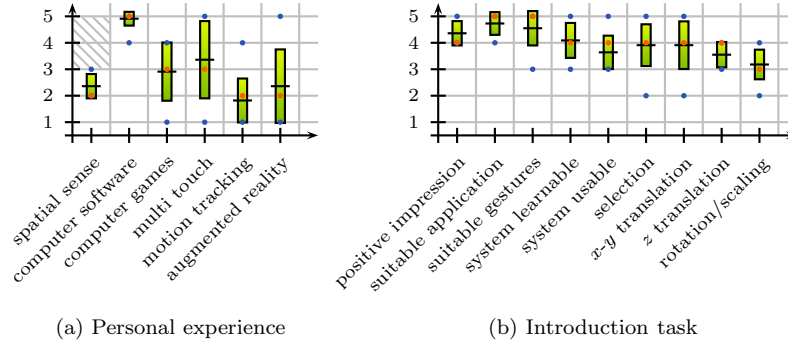


Figure 5.5: Personal experience and feedback for the introduction task.

of Section 4.3.2 was introduced to filter potential palm locations as endpoint of the network topology with the goal to prevent the algorithm from falling off gradually onto the elbow, which was identified as the major problem in five out of five cases. Further, the window size of the optical flow algorithm was changed to be adaptive to the maximum palm radius found in the scene as opposed to a previously fixed value, which was identified as critical in a subsequent technical investigation.

5.4 Main Study

After fixing the major issues in the system design and implementation as discussed in the previous section, the main study was conducted for collecting the final feedback for the conclusions of this research project. In the following, first the group of participants is presented in Section 5.4.1, before providing the detailed statistics of the individual tasks in Section 5.4.2 and highlighting the most important comments in Section 5.4.3.

5.4.1 Participants

In sum, eleven subjects participated in the main study, four female and seven male between the age of 21 and 53, whose personal experience is summarized in Figure 5.5a. The diagram shows the mean and the standard deviation in positive and negative direction (green bar) as well as the minimum/maximum (blue dots) and the median (orange dot) of the answers to the skills and experience questions. The quality of the personal spatial sense could be ranked on a scale between one (vague) to three (pronounced). The remaining questions were targeted on the frequency of technology use and could be ranked on a scale between one (never) over three (monthly) to five (daily).

As the summary shows, the subjects of the trial consistently ranked their spatial senses between normal and pronounced. Instead, almost all participants indicated to use computer software on a daily basis with only one case of weekly

use. Further, computer games and multi-touch applications received a fairly distributed ranking with both a monthly mean and median. In contrast, motion tracking and augmented reality systems appeared to be less popular among the subjects with the mean significantly below monthly use and a median of seldom use. Still, maximum scores lie between weekly and daily encounter with the respective technologies, i.e. also more experienced users are included in the study.

5.4.2 Ratings

In the following, the summary of the direct task feedback is presented. The feedback is divided into several statements that have to be rated by the subject on a 5-point Likert scale with one denoting strong disagreement, and five denoting strong agreement instead. For visualization, the results are provided in Figure 5.5 using the previously introduced scheme for statistical indicators such as mean and standard deviation as well as minimum, median and maximum.

Introduction

The feedback from the introduction task is summarized in Figure 5.5b. Generally, the tendency is positive for all questions except the last, which can be considered neutral among the subjects. In particular, the feedback for the first three questions regarding the first impression as well as the suitability of the application and the gestures appears to be exceptionally good with a mean above four and standard deviations below one. Only one subject rated the suitability of the gestures neutral due problems while grabbing images, which can be reduced to insufficient depth awareness. Further, the learnability and the usability of the system was consistently rated between three and five with the median being four. In comparison, the learnability was ranked slightly better than the usability, which points to an intuitive interaction model and certain problems with the technical realization.

The last four questions are related to the individual components of the interaction model. Selection as well as translation in the x - y plane was rated very similar with a mean of around four and a standard deviation of around one. The minimum score of two was only assigned once by the same subject, which had general trouble using the system due to frequent tracking failures. A first analysis suggested that the specific arm posture of the user in relation to the camera position was the cause of the trouble. Further, the translation in z direction, i.e. towards and away from the camera, was ranked consistently and evenly between three and four showing a general positive tendency. The lower score can be explained with a lack of depth perception as indicated by three out of five subjects, and tracking problems as indicated by the remaining two to subjects. Finally, rotation and scaling received a neutral score with a slight positive tendency. Again, the minimum score of two was only assigned once by the same subject that had encountered frequent tracking problems due to the specific arm posture. In contrast, seven out of eleven users ranked rotation and

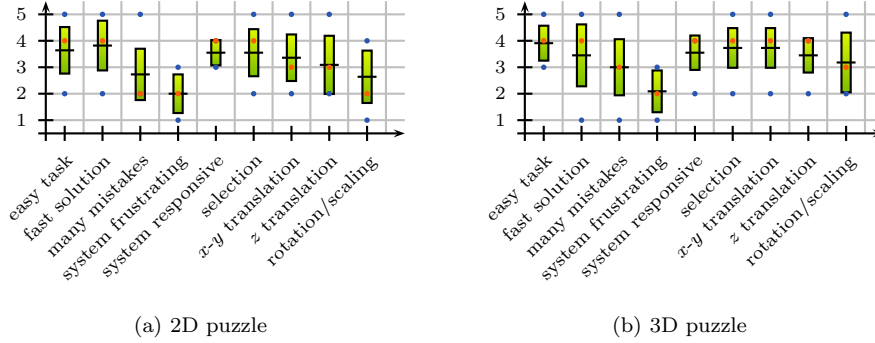


Figure 5.6: Feedback for both puzzle tasks.

scaling neutral. Looking closer, six out of these seven subjects reported tracking issues during rotation, one indicated the issue of occluding hands, which was confirmed to be the major problem from the investigator’s observations.

2D/3D Puzzle

The feedback from the puzzle tasks is summarized in Figure 5.6. The same questionnaire has been used both for the 2D and the 3D puzzle to get comparative data on the subjective impressions of the users. Further, time measurements have been taken to get objective information about how fast the respective problems can be solved using the provided interface. For the 2D puzzle, a mean duration of 169 seconds and a standard deviation of 60 seconds was recorded. In contrast, the 3D puzzle task was executed on average faster with a mean duration of 135 second, while the standard deviation only slightly increased to 69 seconds. To get a better understanding of these time scores, the problems of the four slowest subjects are summarized in Table 5.1.

For the 2D puzzle task (1) all four subjects report problems with tracking during rotation, while (2) three subjects additionally note issues with translation in z direction. From the investigators notes it can be inferred that the first problem is mainly caused by inter-hand occlusions during rotation. Since occlusion is not handled explicitly in the algorithm this constellation inevitably causes tracking loss and interaction break down. As a result, three out of the four subjects provide a negative rating for the *rotation and scaling* action, while only one chooses a neutral score. In contrast, the second issue can be reduced to (a) arms stretched towards the camera in two cases, and (b) problems with depth perception and disorientation in one case. The stretched arm pose has been recognized to be problematic for the tracking algorithm since the entire arm is hidden behind the palm. Consequently, the graph representation from Section 4.3.2 is split into several unconnected components, which in turn leads to multiple topology endpoints and eventually false data associations over time. As a result of the encountered problems, three out of the four slowest subjects

Task	Users	Action	Problem
2D Puzzle	4	rotation/scaling	Hand occlusions
	2	z translation	Stretched arms
	1	z translation	Space disorientation
3D Puzzle	2	rotation/scaling	Hand occlusions
	1	general	Tracking instability
	1	selection	Pose instability

Table 5.1: Problems of the four slowest trial users per task.

indicate personal mistakes during task execution, while two assign a medium frustration value, which is the highest recorded for the 2D puzzle.

Instead, for the 3D puzzle task (1) two out of the four slowest subjects note specific problems with tracking during rotation, while (2) one subject reports general issues with tracking, and (3) another subject encounters difficulties with selecting objects. Again, rotation is causing problems due to hand occlusions, tracking failures and interaction break down. The general tracking problems can mainly be reduced to the stretched arm pose, which was particularly pronounced for the third subject as the investigator’s notes reveal. Finally, the selection issues encountered by the fourth subject originate from spurious triggering of the grab gesture due to insufficient stability of the hand pose. Accordingly, none of the first three subjects assigns a positive score for *rotation and scaling*, while the fourth subject notes a negative score for *selection* instead. As a consequence of the encountered problems, three out of the four slowest subjects additionally note personal mistakes, while another three indicate a medium frustration level, which coincides with the observations of the 2D puzzle.

In summary, the 3D multi-touch interface appears to be equally well suited for both use cases. This hypothesis is confirmed by a final assessment of which task was preferred by the subjects. The opinion was divided equally among the trial users with a tendency towards the respective second tasks (of the random ordering), which points to a learning effect and greater awareness of technological issues. The positive learning effect is also supported by the fact that three subjects (approximately 25%) improved from below average completion time in the first task to above average completion time in the second task, while only one subject changed for the worse due to major orientation problems in the 3D space.

5.4.3 Comments

Finally, the comments of the subjects are summarized in Figure 5.7. For statistical aggregation, the comments have been categorized into a set of positive and negative keywords, and counted across the individual responses. As Figure 5.7a reveals, almost all subjects considered the experiment fun, and many subjects developed an interest in the system. Further, the technology was perceived as an innovative approach to user interface design, which is both intuitive and re-

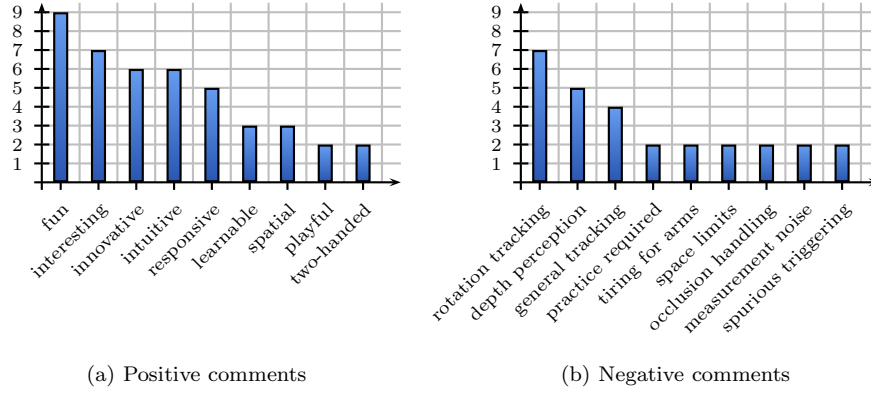


Figure 5.7: Summary of the answers to the open questions.

sponsive at the current time. Among the less common positive comments, the learnability and the spatial context of the interaction as well as the playful and two-handed operation of the system can be found.

Among the most critical problems tracking issues have been reported during rotation by seven subjects, but also in general by four people. Further, five subjects were confused by the visualization of depth, which is rather a deficiency of the demo application than the 3D multi-touch technology itself. Stereo rendering probably would have considerably improved the perception of depth, but it had to be disabled for the previously mentioned reasons. Additionally, two subjects noted that practice is required for handling the system, which stands in contrast to the statement that the system is intuitive to use. However, when looking closer at the data it can be deduced that this comment was noted by same subjects who had major difficulties due to tracking issues. As a result, not the interaction model can be considered problematic but the current technical realization. The remaining comments cover issues like tired arms, operational space limits, occlusion handling, measurement noise and spurious triggering of interface actions.

In summary, the results point to an overall positive impression about the novel barehanded 3D interaction paradigm. In particular, the hand gesture based technique seems to be both intuitive and fun as well as attractive for the users. The technical problems that have been observed mainly relate to the stability of tracking and pose estimation. Despite technical issues, especially the exhaustion of the user should be considered carefully. Though being only reported by two subjects in this study, the problem of operator fatigue has a serious negative potential in more pervasive settings. As a consequence, future interface designs should probably make limited use of extensive hand motion if the average user is to be addressed, otherwise the user's comfort might be considerably disrupted.

Chapter 6

Conclusion

One can conclude that a perfect solution for 3D interaction remains a work in progress. However, important steps have been made towards more intuitive, non-intrusive interaction styles, and the groundwork has been laid for future development. In the following chapter, the progress is critically reviewed: First, in Section 6.1 the project results are reviewed, before listing important remaining issues for future research in Section 6.2. Finally, the the overall results are summarized in Section 6.3.

6.1 Project results

The project results are discussed with respect to the original objectives as defined in the introduction (see Chapter 1). To reiterate, the primary goals of this thesis were to develop a resource efficient algorithm for visual hand processing and gesture recognition which is able to run on standard hardware (1), to conduct an initial user study on novel barehanded 3D interaction techniques (2), and to deliver a software framework upon which future research can build (3). In the following, the results are discussed for each goal individually.

6.1.1 Real-time algorithm

Obviously, the first and most important objective - the implementation of a real-time algorithm for visual hand processing and gesture recognition on standard hardware - was achieved successfully within the scope of this project. As expected, some simplifications had to be made to cope with the complexity of the problem. As a result, some qualitative drawbacks can be recognized as opposed to existing techniques in the area of pose estimation, hand tracking and gesture recognition. In the following, first the core vision algorithm is reviewed, before discussing the advantages and disadvantages of the approach to gesture recognition.

Vision algorithm

Generally, a novel concept for appearance and feature based hand processing on depth images from TOF cameras has been proposed, which allows very efficient implementations and good quality results for reliable hand detection and limited pose estimation using landmark points such as the palm and the fingertips in 3D. The approach is built upon the recent FAST Keypoint Detector [64] and uses Bresenham circles to classify pixels according to their neighborhood into tips and pipes. Palm candidate locations are retrieved from a generic distance transform on the region silhouettes, which takes intra-region depth discontinuities into account. To improve the stability of the process, all possible locations are connected in a graph structure, and palm candidates are confined to represent endpoints in the resulting network topology. This constraint reflects the idea that arms are extremities of the human body, and the hands represent their respective end points. Finally, basic optical flow is employed to solve the problem of temporal association of features, in particular palms and tips, between two time-consecutive frames of the video sequence. To reveal the advantages and disadvantages of the present approach, in the following the individual steps of the algorithm are reviewed individually.

Input filtering using the non-linear Median filter successfully reduces the noise in the depth image while generally maintaining all important structures such as the fingers of the human hand. However, how well these structures are maintained strongly depends on the distance of the human hand as well as the respective size of the finger appearances. In particular, if a finger is located too far from the camera, the filter window of size 3×3 might be covered to nearly one half by the background which in turn causes the finger structures to be blended partially.

Region segmentation using a basic instance of the flood fill algorithm successfully groups the image contents into depth-coherent regions with the goal to reduce processing to restricted image contents only. The employed algorithm is based on a maximum deviation threshold $\epsilon = 1000$, the value was chosen empirically to account for the remaining noise in the depth data. As a consequence, depth discontinuities are required to be larger than roughly 15 to 20 centimeters, otherwise segmentation starts crossing borders. The threshold is a necessity of the current depth measurement quality, i.e. with higher resolution and higher fidelity data the threshold can be decreased, which in turn increases the freedom of the user.

Distance calculation using a depth-discontinuity enhanced distance transform successfully prepares a feature map on which palm location can be found as local minima. The depth discontinuity information is required to find palms even if they are held directly in front of the body and region segmentation groups body, arm and palm into a single entity. For consistency, again the threshold $\epsilon = 1000$ is used, which has the same drawbacks as in the previous case.

Fast classification using a derivative of the FAST keypoint detector for tip and pipe feature mapping probably marks the most important contribution of the novel vision algorithm. Since Bresenham circles are used for traversing neighborhood pixels and deriving the arc segmentation, very efficient implementations can be provided. On the downside, the current implementation requires a constant radius $r_{fix} = 4$ which, in turn, limits the maximum size of the appearance of fingers and their minimum distance to the camera respectively since no multi-resolution analysis scheme is used. Further, again the threshold $\epsilon = 1000$ is used for comparison during arc segmentation with the same drawbacks as previously stated. In particular, the required threshold makes it practically impossible to detect fingers that bend towards the palm as the background distance condition is violated.

Pipe segmentation using the flood fill algorithm and a fixed value range between 0 and $\theta_{pipe}r_{fix}$ successfully finds fingers in the pipe map, which is obtained by the fast classification step. The choice of the parameter $\theta_{pipe} = 0.7$, as opposed to the previous parameters ϵ and r_{fix} , should be independent of the size of the fingers as it is provided in percentage scale. The given value has shown good performance throughout development and user studies.

Feature extraction using Gaussian blurring for improving feature localization as well as non-maximum suppression in a 3×3 window has shown good performance for extracting local turning points in the distance transform and tip feature maps. In particular, palm candidate extraction from local distance maxima has proven to be an efficient and stable strategy in many cases. One drawback, however, is the handling of palms that are rotated orthogonal to the image plane. In these cases, the localization scheme breaks since no distinct distance maximum can be extracted around the actual palm location. In contrast, the extraction of local minima from the tip map is generally more susceptible to noise due to the overall smaller size of the respective features.

Feature connection using an instance of the distance transform algorithm has been demonstrated successfully for deriving the structure of segmented regions and confining the palm locations to the endpoints of the resulting network topology. Most importantly, the proposed technique allows the derivation of contextual information from previously isolated feature points. The implementation uses the threshold $\epsilon = 1000$ for taking depth discontinuities into account, which again marks a drawback with respect to the distance between foreground and background objects. Also, wrist watches occasionally pose problems to the algorithm as the wristband causes random reflections of the infrared signal from the TOF camera.

Finger extraction from the segmented pipes and the tip features provides good performance. In fact, the quality of the presented approach mainly

depends on the quality of the tip and pipe extraction mechanisms, which have been discussed previously. If the underlying data is derived correctly, then the presented scheme has mostly delivered correct results since multiple large pipe structures around a fingertip are generally improbable.

Candidate grouping using plausible matches and weighted selection provides good performance for grouping fingers and distance features to candidate hand structures. The topology criterion for palm locations provides a first valuable constraint for narrowing down the matching space. Plausible matches are mainly based on distance limits between palm center and finger both in image space and in depth space. The limits represent natural constraints for any valid constellation, which is why they are not considered problematic in this discussion.

Palm tracking using linear prediction, feature-based multi-scale optical flow on the distance transform and minimum feature selection has only shown limited performance during the experiments. In particular, the presented scheme cannot handle fast motion sequences well due to the flow algorithm drifting off or entirely missing the actual palm location. Secondly, occlusion is currently not handled explicitly in the current implementation, but has been identified to be a critical component for supporting stable interaction sessions.

Finger tracking uses the same techniques as the palm tracking procedure. In general, the temporal association has shown similar performance to the palm counterpart. However, due to the smaller size of the respective features it is expected that the tracker is overall less stable. It should be noted that, since information from finger tracking has not been used explicitly in the gesture definition, this hypothesis has not been tested.

In summary, the vision algorithm is stable enough to implement a first functional user interface with barehanded 3D interaction capabilities. To keep expectations at an appropriate level, the subjects have to be made aware of the major limitations: Firstly, the hand is required to have a minimum distance to the body, otherwise depth-based segmentation fails to distinguish the corresponding structures. Secondly, the users are advised not to make fast movements as this can result in tracking failures.

Gesture recognition

In order to develop a simple interaction model with 2D Flickr images in a 3D space, a basic gesture recognition component has been developed on top of the core vision algorithm. The formulation of gestures is currently limited to basic statistical indicators such as the mean, minimum, maximum and covariance over temporally connected datasets. The datasets, which are currently processed, include the palm locations as well as the number of fingers detected per hand. More accurate pose models might allow more advanced gesture formulations,

e.g. including the inflection of joints, which cannot be utilized in the current implementation.

During the user study the interaction model received mainly positive feedback by the majority of trial subjects. In general, negative comments were related to technical issues as well as visualization details of the user interface, while the gestures and their specific application for modifying scene contents were perceived both *intuitive* and *fun*. In particular, the two-handed rotation and scaling action has successfully demonstrated the use of multi-touch principles for interaction design in a three-dimensional context. As a consequence, the gesture formulation can generally be considered stable. In contrast, the stability of interaction is mainly limited by the reliability of the tracking component, and interaction usually breaks down between two gestures rather than while executing a specific gesture.

Development platform

To facilitate algorithm development a comprehensive platform for prototyping as well as performance evaluation and comparison was developed. The platform provides a graphical user interface (GUI) including a video capture component, a video sequence inspector and the live video stream (see Appendix E for screenshots). Video capturing is required to record test video sequences upon which the respective vision algorithms are optimized. Usually, each video sequence contains a particular problem such as closed palm tracking, occlusion tracking or similar. The video sequence inspector allows to navigate through the individual frames of each video sequence, while visualizing intermediate data structures and statistics for debugging and profiling the system. Finally, the algorithms can also be applied to the live video stream for validating the real-time performance and testing individual features interactively.

The graphical user interface is built upon a comprehensive vision library including custom data structures and algorithms as well as the general purpose computer vision toolkit OpenCV. The vision components are also included in the demo application as well as other software packages for processing 3D hand information in real-time. In particular, the demo application applies the information in the context of a simple interaction model to demonstrate 3D multi-touch capabilities.

6.1.2 User study

Meeting the second objective - the initial user study - was successful as well. To summarize, a total of two user studies have been conducted for improving and evaluating the performance and usability of a novel proprietary 3D multi-touch interface. The first study was conducted with the purpose of identifying minor drawbacks of the initial system and interface design, which can be addressed easily to deliver a more compelling user experience. In particular, problems with the initial gesture-based interaction model were recognized that required a redesign of the associations between hand gestures and interaction state transi-

tions. Second, a number of tracking issues were encountered due to the optical flow algorithm gradually drifting off the actual palm location. As a consequence, a network topology constraint has been added to prevent highly improbable feature associations between time-consecutive frames. Finally, the visualization of the interaction state required improvement to make the users explicitly aware of state transitions that occurred due to tracking failures rather than explicit hand gestures.

After fixing the highlighted issues, a second study was conducted to collect the final data for relating to performance and usability. Overall, eleven subjects participated that provided largely positive feedback on the usability of the system. In particular, the gesture and interaction model was perceived as *fun* by nine subjects and *intuitive* by six subjects. On the downside, mainly technical issues regarding the tracking and the visualization of depth have been reported. Apart of technical problems, only two subjects reported tired arms, which does not seem critical at first glance. However, when considering that the interaction sessions were limited to a maximum of 15 minutes, it seems possible that longer durations have a stronger negative impact.

As a consequence, it can be concluded that the novel barehanded 3D interaction prototype has the potential to become a standard interaction technique in future applications. In particular, the possibility of natural, non-intrusive, gesture-based interaction along three dimensions is a strong advantage over existing approaches and enables unprecedented comfortable 3D content manipulation. However, in order to reach commercial quality, technical issues need to be resolved in order to increase the stability of the system. Also, the ergonomics of such novel interfaces have to be designed carefully, otherwise the average user will have troubles adopting the respective technology due to other more comfortable choices being available on the market.

6.2 Future work

Despite success in providing a novel barehanded 3D interaction technique, there are a number of open issues that still need to be addressed to improve the usability and stability of the system. This is why in the following remaining usability studies are mentioned in Section 6.2.1, before listing concrete technical improvements in Section 6.2.2, and naming possible extensions in Section 6.2.3, and concentrating on application-specific research in Section 6.2.4.

6.2.1 Usability studies

Due to time constraints some aspects of the novel interaction technique could not be studied within the scope of this thesis. In the following, two important issues are listed that need to be addressed in future work.

Stereoscopic rendering is required to study the usability of the system with respect to a proper depth visualization for 3D content. Therefore, the

stereoscopic parameters such as the position, opening angle and focal point of the virtual cameras have to be tuned to comfort the perception of the user. Thereby, adaptive user-centered projection would probably deliver the best results. Secondly, polarized channel separation has to be introduced to achieve a high level of color fidelity.

Technique comparison has to be carried out with respect to other 3D interaction solutions. One important aspect is for example the selection of distant objects that lie out of the reach of the user's arms. The current interaction model requires the user to walk to that object for starting a manipulation. In contrast, Poupyrev et. al. [60] developed a pointer-based approach using ray-casting, which might be more comfortable and efficient in these situations.

Interface comparison could further contribute to the understanding of the 3D interaction design space. One important aspect is for example the influence of haptic feedback on the usability of the system. The current barehanded approach does not provide any physical sensation of touch, which was not perceived as negative in the context of the demo application. However, the question is how the results are for other scenarios such as the simulation of medical interventions (see Rizzi et. al. [62]).

6.2.2 Technical improvements

The technical improvements are directly derived from the results of the evaluation, which has been presented in Chapter 5. In the following, only the most important issues are listed.

Improved tracking stability is the foremost enhancement required for the current system. Basic optical flow on the raw feature maps does not seem to be distinctive enough for associating data correctly between time-consecutive frames given the complex motion dynamics of the human hand. A size and distance-based match minimization between successive palm locations as proposed by Antonis et. al. [2] might already deliver better performance. Otherwise, more complex techniques such as shape-based trackers [5] or particle filters [33] might be required.

Explicit occlusion handling has shown to be of critical importance for efficient interaction depending on the set of hand gestures that are supported. In particular, the rotation action frequently resulted in the trial users crossing their hands and consequently occluding important visual parts. Successful occlusion handling could be realized by incorporating the arm pose as proposed by Kim et. al. [38], or advanced prediction methods.

Statistical pose estimation from point cloud data will result in more stable, reliable, and accurate pose information. The idea originates from learning-based approaches as for example proposed by Rosales et. al. [63]. Improved

pose data in turn will be the key to more complex interaction designs and more compelling user experiences.

Trained gesture recognition is the direct consequence of improved pose data. The current approach works well with the restricted information that is available, but manual constraint modeling reaches its limits when highly complex pose data is available. As soon as that is the case, learning-based algorithms such as Hidden Markov Models [37] might be required to explore the gesture space more efficiently.

Measurement noise reduction is a critical component for improving the user experience of the system as already pointed out by two out of eleven subjects during the user study. One idea could be to include Kalman Filters [35], which have already proven their value in a number of applications. However, multiple hypothesis support such as Particle Filters [33] might be required to cope with the problem of highly complex motion dynamics.

6.2.3 Possible extensions

The following list contains important extensions that could be made to the system to make the technology more valuable for the market.

Full-body data integration could greatly enhance the design space for interaction with 3D content and media. For example, head detection and tracking could be used to deliver seamless user-centered projections in screen-based virtual reality environments with a single user. In addition, pose information from other body parts could be used to enrich the user experience as already demonstrated by Microsoft's Project Natal¹.

Stereo camera integration has the potential to greatly reduce the cost of the technology. However, color image data introduces new technical challenges that need to be solved efficiently. In particular, depth segmentation cannot be employed without computing dense stereo maps, which unfortunately is an expensive operation. One idea to overcome this issue could be to adapt the hand detection framework to color data and perform 3D matching after feature extraction only.

Mobile deployment will be required to introduce the technology to a variety of scenarios, which require greater efficiency in terms of power consumption. A resource efficient vision algorithm has already been proposed in this thesis. However, a port for mobile devices still has to be investigated. Furthermore, TOF cameras mark a burden for mobile settings at the current stage, which could be resolved by stereo cameras as previously proposed.

¹<http://www.xbox.com/en-us/live/projectnatal/>

6.2.4 Application research

Finally, possible application domains for non-intrusive 3D hand processing and respective interaction techniques are listed.

Data analysis could be carried out in a three-dimensional space using natural barehanded interaction. Especially if highly complex data dependencies are to be analyzed, visualization and manipulation along three dimensions could have a great advantage in organizing and exploring information over existing planar mouse/keyboard/screen approaches such as standard data warehouse and business intelligence applications.

Information modeling could be improved in various domains through an extended operational space and the increased expressive freedom of hand gestures as opposed to a single PC mouse. For example creative activities such as brainstorming, concept development or game character modeling could be supported in a natural way. Also, team collaboration could be facilitated due to independent but integrated interaction sessions of the individual team members both locally and remote.

System simulation could be stream-lined with barehanded interaction to modify simulation parameters and/or trigger simulation events. Interactive 3D simulations could find their application in many domains such as entertainment, education, research, and even industry projects. Natural interaction could improve the efficiency and authenticity of manipulating and exploring the simulation for each respective purpose.

Outdoor interaction with digital 3D content in augmented-reality scenarios currently suffers from poor usability due to the pretension to augment the reality with highly integrated virtual information, while providing only basic input means such as the buttons of a cellphone. Respective systems could be enhanced with vision-based barehanded interfaces to bridge the gap between visualization and manipulation.

It should be noted that many more applications are conceivable in all possible domains. As previously mentioned, the entertainment industry probably represents the strongest force for innovation in the field, but the benefit of barehanded 3D interaction certainly is not limited to computer games.

6.3 Thesis summary

The results of the project can be considered an important step towards a better understanding of barehanded 3D interaction techniques and the possibilities for future 3D interaction design. In particular, a first usability study has shown that 3D multi-touch principles are considered both intuitive and fun, while at the same time current implementations are both sufficiently stable and efficient enough to run on standard hardware with the support of TOF cameras. This

general impression is also supported by the following list of selected comments from the trial users:

“Cool application, it’s really fun!”

“Impressive system with easily learnable features.”

“The gestures were intuitive and conforming to known systems.”

“The interaction takes getting used to, but is easily learnable.”

“A very nice, reactive interface to perform tasks using motion input.”

“I expected that the absence of haptics is disturbing, but it was not like that.”

“The tracking errors are a little disturbing, but do not take the appetite to interact.”

“The experiment was completely positive regardless of the issues with tracking.”

“It looks like an interesting beginning.”

Despite the already usable responsive system, technical issues remain to be addressed to reach commercial product quality including the stability of the tracking component as well as the accuracy of the pose estimations. For this purpose, a comprehensive software framework has been developed which provides all necessary ingredients for implementing and test-driving new ideas. As soon as the core vision component has overcome the current limitations, a new breed of applications can evolve on this novel platform that integrate natural barehanded gesture-based interaction with pervasive digital 3D content and ubiquitous media.

Appendix A

Algorithm: FLOODFILL

In the following, an implementation of the flood fill procedure is provided. The procedure has previously been introduced in Section 2.1.4 as an instance of an image segmentation algorithm.

The implementation uses a boolean predicate function for determining the connectivity between two pixels $P = (x_1, y_1)$ and $Q = (x_2, y_2)$, which is provided in Algorithm A.1. The predicate calculates the absolute difference between the pixel intensities and compares this difference to a predefined deviation threshold $[d_{lower}, d_{upper}]$.

Algorithm A.1 $\text{CONNECTED}(I, x_1, y_1, x_2, y_2, d_{lower}, d_{upper})$

1: **return** $|I(x_1, y_1) - I(x_2, y_2)| \leq d_{lower} \leq |I(x_1, y_1) - I(x_2, y_2)| \leq d_{upper}$

Based on the previous connectivity predicate the actual implementation of the flood fill procedure is defined as shown in Algorithm A.2. The procedure employs a recursive processing scheme, where each iteration considers a single pixel $P = (s_x, s_y)$ and recursions are triggered for connected pixels from the direct neighborhood. Finally, the output is delivered as a mask $M : \mathbb{R}^2 \rightarrow \{0, 1\}$ marking each visited pixel with 1.

Algorithm A.2 FLOODFILL($l, s_x, s_y, d_{lower}, d_{upper}, M$)

```

1: if  $M(s_x, s_y) = 0$  then
2:    $M(s_x, s_y) \leftarrow 1$ 
3:   if CONNECTED( $l, s_x, s_y, s_x - 1, s_y, d_{lower}, d_{upper}$ ) then
4:     FLOODFILL( $l, s_x - 1, s_y, d_{lower}, d_{upper}, M$ )
5:   end if
6:   if CONNECTED( $l, s_x, s_y, s_x, s_y - 1, d_{lower}, d_{upper}$ ) then
7:     FLOODFILL( $l, s_x, s_y - 1, d_{lower}, d_{upper}, M$ )
8:   end if
9:   if CONNECTED( $l, s_x, s_y, s_x + 1, s_y, d_{lower}, d_{upper}$ ) then
10:    FLOODFILL( $l, s_x + 1, s_y, d_{lower}, d_{upper}, M$ )
11:   end if
12:   if CONNECTED( $l, s_x, s_y, s_x, s_y + 1, d_{lower}, d_{upper}$ ) then
13:    FLOODFILL( $l, s_x, s_y + 1, d_{lower}, d_{upper}, M$ )
14:   end if
15: end if

```

Appendix B

Algorithm: DISTANCETRANSFORM

In the following, an implementation of the two-pass distance transform algorithm is provided, which has been introduced in Section 2.1.4. The first pass starts at the top-left pixel $P' = (x_{min}, y_{min})$ and progresses consequently row by row to the bottom-right pixel $P'' = (x_{max}, y_{max})$ of the image for determining top-left obstacle distances. The second pass operates in the reverse order to complement the result with bottom-right obstacle distances.

Algorithm B.1 DISTANCETRANSFORM(l, O)

```
1: for  $y = y_{min}$  to  $y_{max}$  do {First pass}
2:   for  $x = x_{min}$  to  $x_{max}$  do
3:      $P \leftarrow (x, y)$ 
4:     if  $l(x, y) = 1$  then
5:        $O(x, y) \leftarrow 0$ 
6:     else
7:        $O(x, y) \leftarrow \min_{P' \in \mathcal{N}_P^{\nearrow}} \{O(P') + D(P, P')\}$ 
8:     end if
9:   end for
10: end for
11: for  $y = y_{min}$  to  $y_{max}$  do {Second pass}
12:   for  $x = x_{min}$  to  $x_{max}$  do
13:      $P \leftarrow (x, y)$ 
14:      $O(x, y) \leftarrow \min_{P' \in \mathcal{N}_P^{\nwarrow} \cup \{P\}} \{O(P') + D(P, P')\}$ 
15:   end for
16: end for
```

Appendix C

Sequence: Annotation Tracking

In the following, a sample tracking sequence is provided to illustrate the performance of the optical annotation tracking procedure (see Section 3.2 for details).

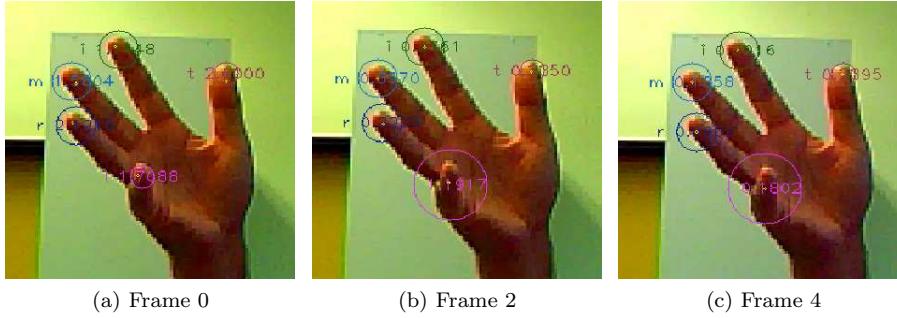


Figure C.1: Tracking results of the optical flow algorithm (frame 0 – 4).

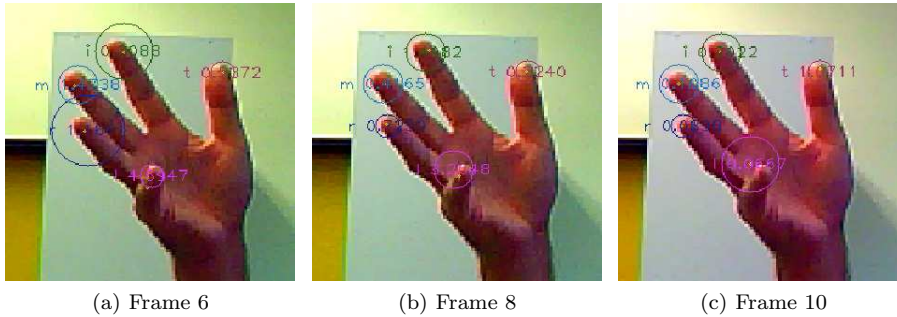


Figure C.2: Tracking results of the optical flow algorithm (frame 6 – 10).

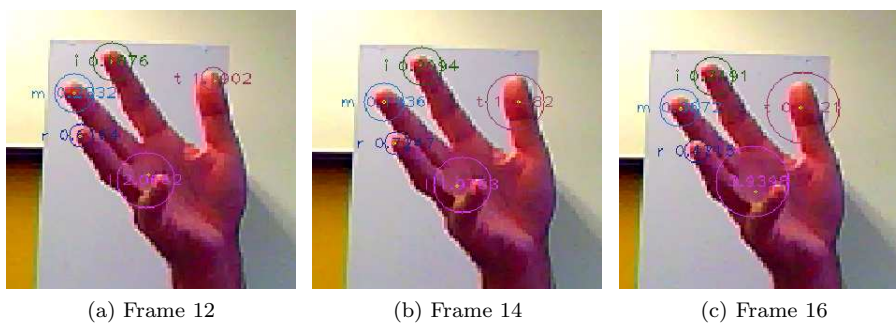


Figure C.3: Tracking results of the optical flow algorithm (frame 12 – 16).

Appendix D

Sequence: Palm Tracking

In the following, a sample sequence is provided to illustrate the performance of the optical flow based palm tracking algorithm (see Section 4.4.1 for details).

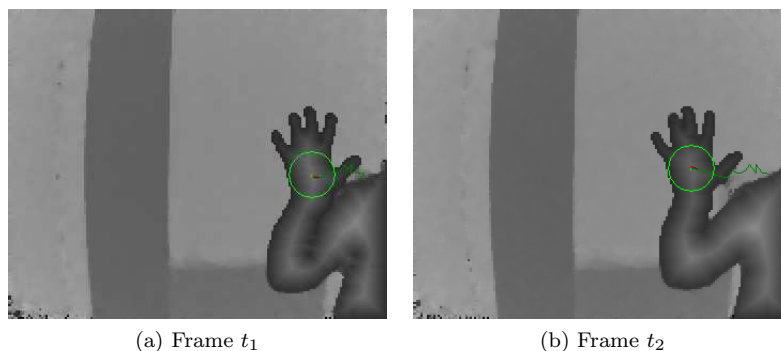


Figure D.1: Results of the palm tracking procedure (frames t_1 and t_2).

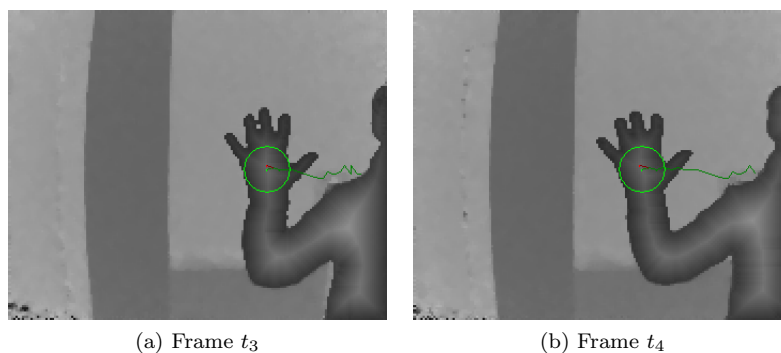


Figure D.2: Results of the palm tracking procedure (frames t_3 and t_4).

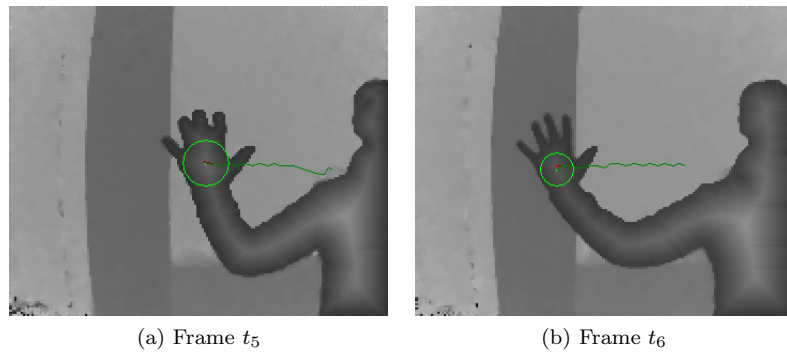


Figure D.3: Results of the palm tracking procedure (frames t_5 and t_6).

Appendix E

Screenshots: Workbench Application

In the following, screenshots of the geometry-based workbench application are presented (see Section 3.3 for details). Figure E.1 shows the capture screen, which allows to record and save test video streams for off-line algorithm debugging, profiling, and performance evaluation. Instead, Figure E.2 shows the inspection screen, where the test videos can be accessed and results of the algorithm can be viewed. Finally, Figure E.3 shows the live screen, which allows to apply the algorithm on live data from the connected TOF camera.

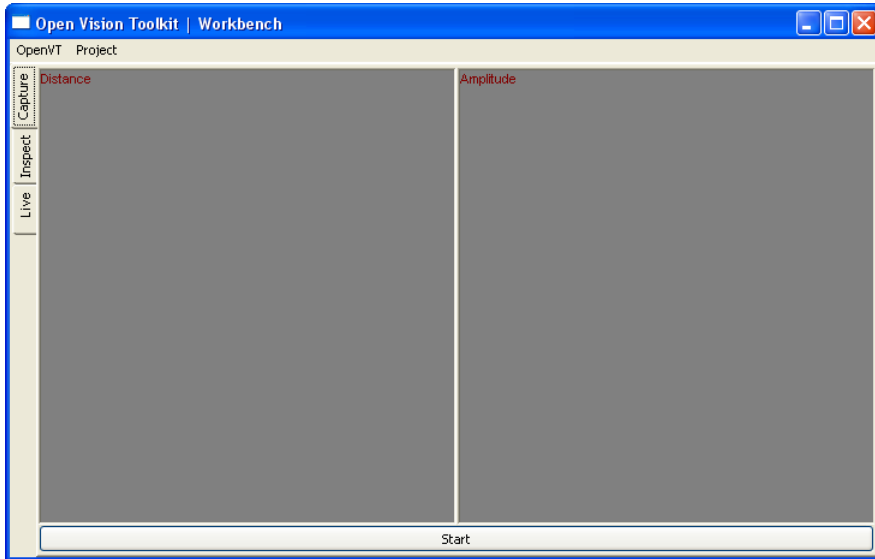


Figure E.1: *Capture* tab of the geometry-based workbench.

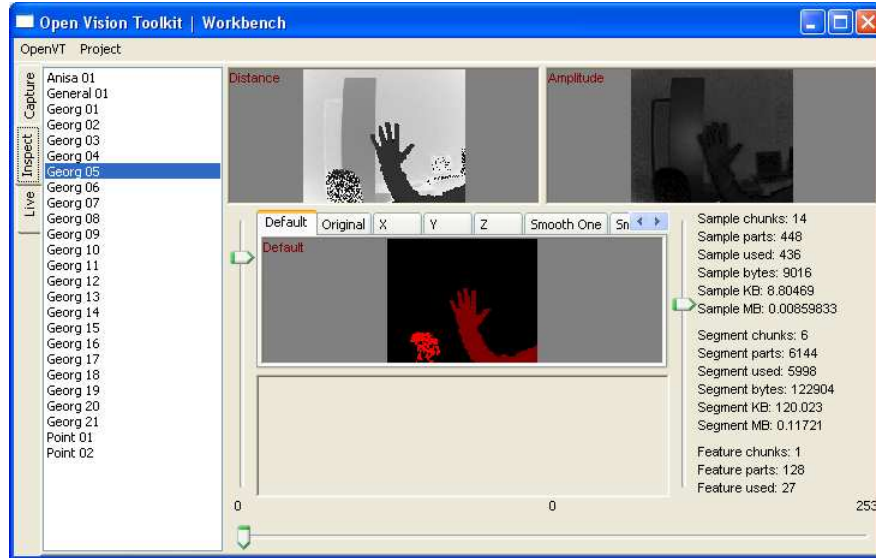


Figure E.2: *Inspect* tab of the geometry-based workbench.

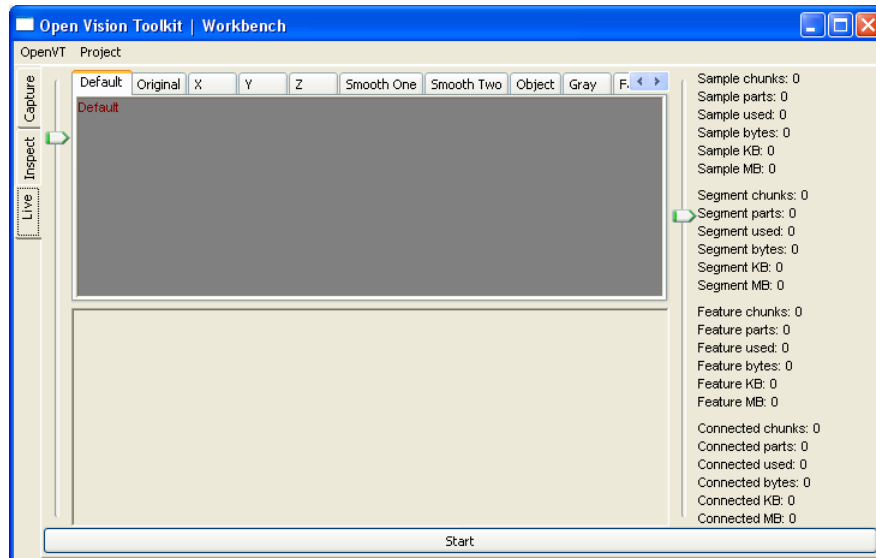


Figure E.3: *Live* tab of the geometry-based workbench.

Bibliography

- [1] K. Abe, H. Saito, and S. Ozawa. 3-d drawing system via hand motion recognition from two cameras. In *Systems, Man, and Cybernetics, 2000 IEEE International Conference on*, volume 2, pages 840–845, 2000.
- [2] A. A. Argyros and M. I. A. Lourakis. Real-time tracking of multiple skin-colored objects with a possibly moving camera. In *ECCV*, pages 368–379, 2004.
- [3] A. A. Argyros and M. I. A. Lourakis. Binocular hand tracking and reconstruction based on 2d shape matching. In *ICPR '06: Proceedings of the 18th International Conference on Pattern Recognition*, pages 207–210, Washington, DC, USA, 2006. IEEE Computer Society.
- [4] V. Athitsos and S. Sclaroff. An appearance-based framework for 3d hand shape classification and camera viewpoint estimation. In *FGR '02: Proceedings of the Fifth IEEE International Conference on Automatic Face and Gesture Recognition*, page 45, Washington, DC, USA, 2002. IEEE Computer Society.
- [5] K. A. Barhate, K. S. Patwardhan, S. D. Roy, S. Chaudhuri, and S. Chaudhuri. Robust shape based two hand tracker. In *Image Processing, 2004. ICIP '04. 2004 International Conference on*, volume 2, pages 1017–1020, Oct. 2004.
- [6] H. G. Barrow, J. M. Tenenbaum, R. C. Bolles, and H. C. Wolf. Parametric correspondence and chamfer matching: two new techniques for image matching. In *IJCAI'77: Proceedings of the 5th international joint conference on Artificial intelligence*, pages 659–663, San Francisco, CA, USA, 1977. Morgan Kaufmann Publishers Inc.
- [7] H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool. Speeded-up robust features (surf). *Comput. Vis. Image Underst.*, 110(3):346–359, 2008.
- [8] M. J. Black and A. D. Jepson. Eigenttracking: Robust matching and tracking of articulated objects using a view-based representation. In *International Journal of Computer Vision*, pages 329–342, 1998.

- [9] Y. Bulatov, S. Jambawalikar, P. Kumar, and S. Sethia. Hand recognition using geometric classifiers. In *ICBA'04: Proceedings of the 1st international conference on Biometric Authentication*, pages 753–759, 2004.
- [10] J. Canny. Finding edges and lines in images. Technical report, Cambridge, MA, USA, 1983.
- [11] J. W. Chastine, K. Nagel, Y. Zhu, and L. Yearsovich. Understanding the design space of referencing in collaborative augmented reality environments. In *GI '07: Proceedings of Graphics Interface 2007*, pages 207–214, New York, NY, USA, 2007. ACM.
- [12] D. Chik, J. Trumpf, and N. N. Schraudolph. 3d hand tracking in a stochastic approximation setting.
- [13] D. Comaniciu and P. Meer. Robust analysis of feature spaces: color image segmentation. In *CVPR '97: Proceedings of the 1997 Conference on Computer Vision and Pattern Recognition (CVPR '97)*, page 750, Washington, DC, USA, 1997. IEEE Computer Society.
- [14] T. Cover and P. Hart. Nearest neighbor pattern classification. *Information Theory, IEEE Transactions on*, 13(1):21–27, 1967.
- [15] M. De La Gorce and N. Paragios. A variational approach to monocular hand-pose estimation. *Computer Vision and Image Understanding*, 114(3):363–372, 2010.
- [16] P. De La Hamette and G. Troster. Fingermouse - architecture of an asic-based mobile stereovision smart camera. In *Wearable Computers, 2006 10th IEEE International Symposium on*, pages 121–122, Oct. 2006.
- [17] Q. Delamarre and O. Faugeras. Finding pose of hand in video images: A stereo-based approach. In *FG '98: Proceedings of the 3rd. International Conference on Face & Gesture Recognition*, page 585, Washington, DC, USA, 1998. IEEE Computer Society.
- [18] J. Deutscher, A. Blake, and I. Reid. Articulated body motion capture by annealed particle filtering. In *Computer Vision and Pattern Recognition, 2000. Proceedings. IEEE Conference on*, volume 2, pages 126–133, 2000.
- [19] M. Donoser and H. Bischof. Real time appearance based hand tracking. In *Pattern Recognition, 2008. ICPR 2008. 19th International Conference on*, pages 1–4, Dec. 2008.
- [20] J. Edelmann, A. Schilling, and S. Fleck. The dabr - a multitouch system for intuitive 3d scene navigation. In *3DTV Conference: The True Vision - Capture, Transmission and Display of 3D Video, 2009*, pages 1–4, May 2009.

- [21] A. El-Sawah, C. Joslin, N. D. Georganas, and E. M. Petriu. A framework for 3d hand tracking and gesture recognition using elements of genetic programming. In *CRV '07: Proceedings of the Fourth Canadian Conference on Computer and Robot Vision*, pages 495–502, Washington, DC, USA, 2007. IEEE Computer Society.
- [22] M. Fiorentino, R. De Amicis, G. Monno, and A. Stork. Spacedesign: A mixed reality workspace for aesthetic industrial design. In *ISMAR '02: Proceedings of the 1st International Symposium on Mixed and Augmented Reality*, page 86, Washington, DC, USA, 2002. IEEE Computer Society.
- [23] J. Fredriksson, S. B. Ryen, and M. Fjeld. Real-time 3d hand-computer interaction: optimization and complexity reduction. In *NordiCHI '08: Proceedings of the 5th Nordic conference on Human-computer interaction*, pages 133–141, New York, NY, USA, 2008. ACM.
- [24] T. Grossman, D. Wigdor, and R. Balakrishnan. Multi-finger gestural interaction with 3d volumetric displays. In *UIST '04: Proceedings of the 17th annual ACM symposium on User interface software and technology*, pages 61–70, New York, NY, USA, 2004. ACM.
- [25] T. Gump, P. Azad, K. Welke, E. Oztop, R. Dillmann, and G. Cheng. Unconstrained real-time markerless hand tracking for humanoid interaction. In *Humanoid Robots, 2006 6th IEEE-RAS International Conference on*, pages 88–93, Dec. 2006.
- [26] C. Harris and M. Stephens. A combined corner and edge detection. In *Proceedings of The Fourth Alvey Vision Conference*, pages 147–151, 1988.
- [27] R. Hartley. Theory and practice of projective rectification. *Int. J. Comput. Vision*, 35(2):115–127, 1999.
- [28] R. Hartley and A. Zisserman. *Multiple view geometry in computer vision*. Cambridge University Press, New York, NY, USA, 2000.
- [29] I. Herbst, A.-K. Braun, R. McCall, and W. Broll. Timewarp: interactive time travel with a mobile mixed reality game. In *MobileHCI '08: Proceedings of the 10th international conference on Human computer interaction with mobile devices and services*, pages 235–244, New York, NY, USA, 2008. ACM.
- [30] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.
- [31] M.-K. Hu. Visual pattern recognition by moment invariants. *Information Theory, IEEE Transactions on*, 8(2):179–187, 1962.
- [32] H. Hua, L. D. Brown, and C. Gao. Scape: supporting stereoscopic collaboration in augmented and projective environments. *Computer Graphics and Applications, IEEE*, 24(1):66–75, Jan.-Feb. 2004.

- [33] M. Isard and A. Blake. Condensation - conditional density propagation for visual tracking. *International Journal of Computer Vision*, 29:5–28, 1998.
- [34] M. Isard and A. Blake. Icondensation: Unifying low-level and high-level tracking in a stochastic framework. In *ECCV '98: Proceedings of the 5th European Conference on Computer Vision-Volume I*, pages 893–908, London, UK, 1998. Springer-Verlag.
- [35] R. E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME-Journal of Basic Engineering*, 82(Series D):35–45, 1960.
- [36] H. Kaufmann and D. Schmalstieg. Designing immersive virtual reality for geometry education. In *Virtual Reality Conference, 2006*, pages 51–58, March 2006.
- [37] C. Keskin, A. Erkan, and L. Akarun. Real time hand tracking and 3d gesture recognition for interactive interfaces using hmm. In *In Proceedings of International Conference on Artificial Neural Networks*, 2003.
- [38] H.-J. Kim, K.-C. Kwak, S. O. Chi, and Y.-J. Cho. Ar-klt based hand tracking. In *Robot and Human Interactive Communication, 2006. ROMAN 2006. The 15th IEEE International Symposium on*, pages 611–616, Sept. 2006.
- [39] H.-S. Kim, G. Kurillo, and R. Bajcsy. Hand tracking and motion detection from the sequence of stereo color image frames. In *Industrial Technology, 2008. ICIT 2008. IEEE International Conference on*, pages 1–6, April 2008.
- [40] H. Koike and M. Kobayashi. Integrating paper and digital information on enhanceddesk: a method for realtime finger tracking on an augmented desk system. *ACM Trans. Comput.-Hum. Interact.*, 8(4):307–322, 2001.
- [41] M. Kolsch and M. Turk. Robust hand detection. In *Automatic Face and Gesture Recognition, 2004. Proceedings. Sixth IEEE International Conference on*, pages 614–619, May 2004.
- [42] M. Kolsch and M. Turk. Hand tracking with flocks of features. In *Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on*, volume 2, page 1187, June 2005.
- [43] A. Kotranza, J. Quarles, and B. Lok. Mixed reality: are two hands better than one? In *VRST '06: Proceedings of the ACM symposium on Virtual reality software and technology*, pages 31–34, New York, NY, USA, 2006. ACM.
- [44] R. Lange and P. Seitz. Solid-state time-of-flight range camera. *Quantum Electronics, IEEE Journal of*, 37(3):390–397, Mar. 2001.

- [45] H. K. Lee and J. H. Kim. An hmm-based threshold model approach for gesture recognition. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 21(10):961–973, Oct. 1999.
- [46] J. Letessier and F. Bérard. Visual tracking of bare fingers for interactive surfaces. In *UIST '04: Proceedings of the 17th annual ACM symposium on User interface software and technology*, pages 119–122, New York, NY, USA, 2004. ACM.
- [47] W. Liang, Y. Jia, F. Sun, B. Ning, T. Liu, and X. Wu. Visual hand tracking using mdsa method. In *Computational Engineering in Systems Applications, IMACS Multiconference on*, volume 1, pages 255–259, Oct. 2006.
- [48] H. Liu, T.-H. Hong, M. Herman, T. Camus, and R. Chellappa. Accuracy vs efficiency trade-offs in optical flow algorithms. *Comput. Vis. Image Underst.*, 72(3):271–286, 1998.
- [49] Y. Liu and Y. Jia. A robust hand tracking and gesture recognition method for wearable visual interfaces and its applications. In *Image and Graphics, 2004. Proceedings. Third International Conference on*, pages 472–475, Dec. 2004.
- [50] S. Lu, D. Metaxas, D. Samaras, and J. Oliensis. Using multiple cues for hand tracking and model refinement. In *Computer Vision and Pattern Recognition, 2003. Proceedings. 2003 IEEE Computer Society Conference on*, volume 2, pages 443–450, June 2003.
- [51] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *IJCAI'81: Proceedings of the 7th international joint conference on Artificial intelligence*, pages 674–679, San Francisco, CA, USA, 1981. Morgan Kaufmann Publishers Inc.
- [52] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *IJCAI'81: Proceedings of the 7th International Joint Conference on Artificial Intelligence*, pages 674–679, 1981.
- [53] J. Matas, O. Chum, M. Urban, and T. Pajdla. Robust wide-baseline stereo from maximally stable extremal regions. *Image and Vision Computing*, 22(10):761–767, September 2004.
- [54] M. Mesbah. Gradient-based optical flow: a critical review. In *Signal Processing and Its Applications, 1999. ISSPA '99. Proceedings of the Fifth International Symposium on*, volume 1, pages 467–470, 1999.
- [55] Z. Mo, J. P. Lewis, and U. Neumann. Smartcanvas: a gesture-driven intelligent drawing desk system. In *IUI '05: Proceedings of the 10th international conference on Intelligent user interfaces*, pages 239–243, New York, NY, USA, 2005. ACM.

- [56] K. Oka, Y. Sato, and H. Koike. Real-time tracking of multiple fingertips and gesture recognition for augmented desk interface systems. In *FGR '02: Proceedings of the Fifth IEEE International Conference on Automatic Face and Gesture Recognition*, page 429, Washington, DC, USA, 2002. IEEE Computer Society.
- [57] N. R. Pal and S. K. Pal. A review on image segmentation techniques. *Pattern Recognition*, 26(9):1277–1294, 1993.
- [58] P. Peltonen, A. Salovaara, G. Jacucci, T. Ilmonen, C. Ardito, P. Saarikko, and V. Batra. Extending large-scale event participation with user-created mobile media on a public display. In *MUM '07: Proceedings of the 6th international conference on Mobile and ubiquitous multimedia*, pages 131–138, New York, NY, USA, 2007. ACM.
- [59] S. Perreault and P. Hebert. Median filtering in constant time. *Image Processing, IEEE Transactions on*, 16(9):2389–2394, Sept. 2007.
- [60] I. Poupyrev, M. Billinghurst, S. Weghorst, and T. Ichikawa. The go-go interaction technique: non-linear mapping for direct manipulation in vr. In *UIST '96: Proceedings of the 9th annual ACM symposium on User interface software and technology*, pages 79–80, New York, NY, USA, 1996. ACM.
- [61] G. Reitmayr and D. Schmalstieg. A wearable 3d augmented reality workspace. In *Wearable Computers, 2001. Proceedings. Fifth International Symposium on*, pages 165–166, 2001.
- [62] S. H. Rizzi, C. J. Luciano, and P. P. Banerjee. Haptic interaction with volumetric datasets using surface-based haptic libraries. In *Haptics Symposium, 2010 IEEE*, pages 243–250, March 2010.
- [63] R. Rosales, V. Athitsos, L. Sigal, and S. Sclaroff. 3d hand pose reconstruction using specialized mappings. In *Computer Vision, 2001. ICCV 2001. Proceedings. Eighth IEEE International Conference on*, volume 1, pages 378–385, 2001.
- [64] E. Rosten, R. Porter, and T. Drummond. Faster and better: A machine learning approach to corner detection. *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, 32(1):105–119, Jan. 2010.
- [65] H. A. Rowley, S. Baluja, and T. Kanade. Rotation invariant neural network-based face detection. In *CVPR '98: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, page 38, Washington, DC, USA, 1998. IEEE Computer Society.
- [66] Y. Sato and M. Saito. Real-time input of 3d pose and gestures of a user's hand and its applications for hci. In *Proc. 2001 IEEE Virtual Reality Conference*, pages 79–86, 2001.

- [67] M. Schlattman and R. Klein. Simultaneous 4 gestures 6 dof real-time two-hand tracking without any markers. In *VRST '07: Proceedings of the 2007 ACM symposium on Virtual reality software and technology*, pages 39–42, New York, NY, USA, 2007. ACM.
- [68] J. Segen and S. Kumar. Gesture vr: vision-based 3d hand interace for spatial interaction. In *MULTIMEDIA '98: Proceedings of the sixth ACM international conference on Multimedia*, pages 455–464, New York, NY, USA, 1998. ACM.
- [69] B. E. Shelton and N. R. Hedley. Using augmented reality for teaching earth-sun relationships to undergraduate geography students. page 8 pp., Sept. 2002.
- [70] J. Stewart, E. M. Raybourn, B. Bederson, and A. Druin. When two hands are better than one: enhancing collaboration using single display groupware. In *CHI '98: CHI 98 conference summary on Human factors in computing systems*, pages 287–288, New York, NY, USA, 1998. ACM.
- [71] K. N. Tran and Z. Huang. Design and implementation of a built-in camera based user interface for mobile games. In *GRAPHITE '07: Proceedings of the 5th international conference on Computer graphics and interactive techniques in Australia and Southeast Asia*, pages 25–31, New York, NY, USA, 2007. ACM.
- [72] E. Ueda, Y. Matsumoto, M. Imai, and T. Ogasawara. Hand pose estimation using multi-viewpoint silhouette images. In *Intelligent Robots and Systems, 2001. Proceedings. 2001 IEEE/RSJ International Conference on*, volume 4, pages 1989–1996, 2001.
- [73] A. Utsumi and J. Ohya. Multiple-hand-gesture tracking using multiple cameras. volume 1, page 478, Los Alamitos, CA, USA, 1999. IEEE Computer Society.
- [74] O. Veksler. Fast variable window for stereo correspondence using integral images. In *Computer Vision and Pattern Recognition, 2003. Proceedings. IEEE Computer Society Conference on*, volume 1, pages 556–561, June 2003.
- [75] P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *Computer Vision and Pattern Recognition, 2001. CVPR 2001. Proceedings of the 2001 IEEE Computer Society Conference on*, volume 1, pages 511–518, 2001.
- [76] J. Wall. Demo i microsoft surface and the single view platform. In *CTS '09: Proceedings of the 2009 International Symposium on Collaborative Technologies and Systems*, pages xxxi–xxxii, Washington, DC, USA, 2009. IEEE Computer Society.

- [77] L. Wang, M. Liao, M. Gong, R. Yang, and D. Nistér. High-quality real-time stereo using adaptive cost aggregation and dynamic programming. In *3D Data Processing, Visualization, and Transmission, Third International Symposium on*, pages 798–805, June 2006.
- [78] X. Wang, X. Zhang, and G. Dai. Tracking of deformable human hand in real time as continuous input for gesture-based interaction. In *IUI '07: Proceedings of the 12th international conference on Intelligent user interfaces*, pages 235–242, New York, NY, USA, 2007. ACM.
- [79] Q. Yang, L. Wang, R. Yang, S. Wang, M. Liao, and D. Nistér. Real-time global stereo matching using hierarchical belief propagation. In *BMVC'06: Proceedings of the 17th British Machine Vision Conference*, volume 3, pages 989–998, 2006.
- [80] R. Yang and M. Pollefeys. Multi-resolution real-time stereo on commodity graphics hardware. In *Computer Vision and Pattern Recognition, 2003. Proceedings. 2003 IEEE Computer Society Conference on*, volume 1, pages 211–217, June 2003.
- [81] Y. Yao, D. Wu, and Y. Liu. Collaborative education ui in augmented reality from remote to local. In *Education Technology and Computer Science, 2009. ETCS '09. First International Workshop on*, volume 2, pages 670–673, March 2009.
- [82] S. Zagoranski and S. Divjak. Use of augmented reality in education. In *EUROCON 2003. Computer as a Tool. The IEEE Region 8*, volume 2, pages 339–342, Sept. 2003.
- [83] Z. Zhang. Iterative point matching for registration of free-form curves and surfaces. *International Journal of Computer Vision*, 13(2):119–152, October 1994.
- [84] S. Zhong and F. Hao. Hand tracking by particle filtering with elite particles mean shift. In *Frontier of Computer Science and Technology, 2008. FCST '08. Japan-China Joint Workshop on*, pages 163–167, Dec. 2008.